

Konzeption und Implementierung eines KI-gestützten GitLab-Assistenten zur Verbesserung der Qualität von Software-Tickets

21. Juli 2026

Inhaltsverzeichnis

1	Einleitung	5
2	Theoretische und technische Grundlagen	8
2.1	Requirements Engineering und Anforderungen in Softwareentwicklungsprozessen	8
2.2	Tickets, User Stories und Bug Reports als Arbeitsartefakte	9
2.3	Qualität von Ticketinformationen	11
2.4	Generative KI und Large Language Models	12
2.5	LLM-basierte Agenten, Werkzeugnutzung und Opencode	13
2.6	GitLab, Webhooks und technische Integrationsplattform	14
2.7	Risiken und Validierungsbedarf bei generativer KI und Webhooks	16
2.8	Zwischenfazit und Anforderungen an Dev-Assist	18
3	Anforderungsanalyse	19
3.1	Vorgehen und Grundlagen der Anforderungsanalyse	19
3.2	Beschreibung des bisherigen Prozesses	20
3.3	Zielgruppen des Systems	21
3.4	Fachliche Anforderungen an die Ticketqualität	22
3.5	Funktionale Anforderungen	24
3.6	Nicht-funktionale Anforderungen	26
3.7	Abgrenzung der Anforderungen	27
3.8	Priorisierung der Anforderungen	28
3.9	Zwischenfazit	28
4	Konzeption der Systemarchitektur	30
4.1	Architekturziele und Entwurfsprinzipien	30
4.2	Gesamtarchitektur des Webhook-Systems	31
4.3	Ereignisbasierter Ablauf über GitLab-Webhooks	32
4.4	Zentrale Komponenten und Verantwortlichkeiten	33
4.5	Datenfluss vom GitLab-Issue bis zum aktualisierten Ticket	34
4.6	Prompt-, Agenten- und Antwortverarbeitung	35
4.7	Kontextpersistenz, Publish-Funktion und Kommentarbereinigung	36
4.8	Fehlerbehandlung und Robustheit	36
4.9	Sicherheitskonzept	37
4.10	Bild-, Screenshot- und Repository-Kontext	38
4.11	Validierung und Testkonzept der Architektur	38
4.12	Zwischenfazit	39
5	Implementierung	40
5.1	Aufbau des TypeScript-Projekts	40
5.2	Konfiguration, Startverhalten und Logging	41

5.3	Express-Server und HTTP-Routen	42
5.4	Webhook-Verarbeitung, Deduplication und Aktivierung	43
5.5	GitLab-Integration über glab und REST-Fallback	44
5.6	Prozesssteuerung der Analyse	45
5.7	OpenCode-Agenten-Integration und Prompt-Design	46
5.8	JSON-Antwortverarbeitung und Laufzeitvalidierung	47
5.9	Umgang mit Rückfragen und Kommentarverlauf	47
5.10	Repository-Kontext und Grenzen der Bildverarbeitung	48
5.11	Kontextpersistenz und Publish-Kommando	49
5.12	Simulation, lokale Ausführung und Tests	49
5.13	Zwischenfazit	50
6	Qualitätssicherung und Evaluation	52
6.1	Zielsetzung und Untersuchungsrahmen	52
6.2	Teststrategie	53
6.3	Testumgebung, Durchführung und Gesamtergebnis	54
6.4	Unit- und Komponententests	55
6.4.1	Webhook-Parsing, Aktivierung und Kommandos	55
6.4.2	Webhook-Authentifizierung und Kommentarbereinigung	56
6.4.3	Schema-Validierung und Ausgabeformatierung	57
6.4.4	Prompt, Rückfragen und Opencode-Ausgabe	57
6.4.5	Repository-Kontext, Process und Publish	58
6.5	Simulation ohne reale GitLab- oder Opencode-Abhängigkeit	58
6.6	Webhook-Randfälle und Robustheitsbefunde	59
6.7	Bewertungskriterien für die inhaltliche Ticketqualität	60
6.8	Nachverfolgbarkeit der funktionalen Anforderungen	62
6.9	Nachverfolgbarkeit der nicht-funktionalen Anforderungen	64
6.10	Grenzen und Bedrohungen der Aussagekraft	65
6.11	Zwischenfazit	66
7	Ergebnisse	69
7.1	Ergebnisrahmen	69
7.2	Beschreibung des finalen Prototyps	70
7.2.1	Schnittstellen und Aktivierung	71
7.2.2	Kontextaufbereitung und Analyse	71
7.2.3	Rückfragen, Vorschläge und Kontextdateien	72
7.2.4	Kontrollierter Publish-Schritt	73
7.3	Beispielhafter Ablauf	73
7.3.1	Erstellung und Aktivierung des Issues	74
7.3.2	Kontextabruf und erste Analyse	74
7.3.3	Antwort und erneute Analyse	74
7.3.4	Strukturierter Vorschlag	75
7.3.5	Freigabe und Übernahme	76
7.4	Ergebnisartefakte und Übergabepunkte	76
7.5	Erfüllung der funktionalen Anforderungen	77
7.6	Erfüllung der nicht-funktionalen Anforderungen	78
7.7	Verdichtung der Ergebnisse	80
7.8	Zwischenfazit	80

8	Diskussion	83
8.1	Einordnung des Ergebnisbeitrags	83
8.2	Nutzen im Entwicklungsprozess	84
8.2.1	Strukturierung und frühe Sichtbarkeit von Informationslücken .	84
8.2.2	Entlastung bei wiederkehrender Aufbereitungsarbeit	85
8.2.3	Prozessintegration und kontrollierte Freigabe	85
8.3	Vergleich mit manueller Ticketpflege	86
8.4	Grenzen KI-generierter Tickervorschläge	87
8.4.1	Halluzinationen und falsche Annahmen	87
8.4.2	Begrenzter und potenziell veralteter Kontext	88
8.4.3	Fehlender semantischer und externer Nachweis	88
8.5	Risiken und verantwortbarer Einsatz	89
8.5.1	Prompt Injection und begrenzte Autonomie	89
8.5.2	Datenschutz und Vertraulichkeit	89
8.5.3	Technische und organisatorische Betriebsrisiken	90
8.6	Wartbarkeit und Erweiterbarkeit	90
8.7	Gesamtbewertung	91
8.8	Zwischenfazit	92
9	Fazit und Ausblick	95
9.1	Zusammenfassung der Ergebnisse	95
9.2	Beantwortung der Forschungsfragen	96
9.2.1	Erforderliche Informationen für umsetzbare und überprüfbare GitLab-Tickets	96
9.2.2	Kontrollierte Integration eines KI-gestützten Assistants in GitLab	97
9.2.3	Übergreifende Antwort	98
9.3	Beitrag und Grenzen der Arbeit	99
9.4	Priorisierter Ausblick	99
9.4.1	Priorität 1: Vertragskorrektheit und belastbare Evaluation . . .	100
9.4.2	Bessere Metriken zur Ticketqualität	100
9.4.3	Priorität 2: Produktiver Betrieb, Monitoring und Governance .	101
9.4.4	Priorität 3: Funktionale Erweiterungen	101
9.5	Schlussfolgerung	102
	Quellen	104

1 Einleitung

In modernen Softwareentwicklungsprozessen werden Anforderungen, Fehlerberichte und ergänzende Hinweise häufig in Ticket- oder Issue-Tracking-Systemen dokumentiert. Diese Artefakte sind nicht nur technische Aufgabenlisten, sondern Koordinationspunkte zwischen meldenden Personen, Produktverantwortlichen und Entwicklerinnen und Entwicklern. Studien zu Bug Reports zeigen, dass Bug-Tracking-Systeme eine zentrale Rolle in der Zusammenarbeit zwischen Nutzenden und Entwicklungsteams übernehmen und dass Rückfragen in Tickets häufig dazu dienen, fehlende Informationen nachzuliefern (vgl. Breu et al., 2010, S. 301 und S. 303). Gleichzeitig unterscheiden sich die Informationen, die meldende Personen bereitstellen, deutlich von den Informationen, die Entwicklerinnen und Entwickler für eine schnelle Bearbeitung benötigen. Besonders Reproduktionsschritte, Stack Traces und Testfälle gelten als hilfreich, sind für meldende Personen aber schwer bereitzustellen (vgl. Bettenburg et al., 2008, S. 308 f.).

Diese Diskrepanz betrifft nicht nur klassische Fehlerberichte, sondern auch Anforderungen in agilen Entwicklungsprozessen. Anforderungen werden dort häufig in knappen, natürlichsprachlichen Formen wie User Stories oder Tickets erfasst. Ihre Qualität hängt davon ab, ob Ziel, Kontext, erwartetes Verhalten, Randbedingungen und Akzeptanzkriterien so beschrieben sind, dass sie von Entwicklungsteams verstanden, umgesetzt und überprüft werden können. Lucassen et al. beschreiben User Stories als weit verbreitete Notation im agilen Requirements Engineering, weisen jedoch zugleich darauf hin, dass sie in der Praxis häufig Qualitätsdefekte aufweisen. Das Quality User Story Framework fasst diese Qualität unter anderem über syntaktische, semantische und pragmatische Kriterien (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Für Fehlerberichte zeigen Chaparro et al. ähnlich, dass Beschreibungen für die Bearbeitung insbesondere beobachtetes Verhalten, erwartetes Verhalten und Schritte zur Reproduktion enthalten sollten, solche Informationen jedoch oft fehlen oder nur unvollständig vorliegen (vgl. Chaparro et al., 2017, S. 396 f.).

Damit entsteht ein praktisches Problem an der Schnittstelle zwischen Kommunikation und Entwicklung: Ein Ticket kann fachlich relevant sein, aber dennoch nicht in einer Form vorliegen, die unmittelbar für Planung, Implementierung und Validierung geeignet ist. Entwicklungsteams müssen dann fehlende Informationen nachfragen, vorhandene Kommentare interpretieren, Screenshots oder Anhänge einordnen und aus verstreutem Kontext eine umsetzbare Anforderung ableiten. Wissenschaftliche Arbeiten und Werkzeuge zeigen, dass automatisierte Unterstützung bei der Bewertung und Strukturierung solcher Informationen grundsätzlich möglich ist. CUEZILLA bewertet die Qualität von Bug Reports und schlägt ergänzende Informationen vor, AQUASA erkennt Qualitätsdefekte in User Stories und DeMiBuD identifiziert insbesondere fehlendes erwartetes Verhalten sowie fehlende Reproduktionsschritte in Bug Descriptions. Diese Ansätze leisten damit jeweils unterschiedliche Beiträge zur automatisierten Qualitätsunterstützung. Für diese Arbeit bleibt darüber hinaus die Frage offen, wie eine

solche Unterstützung in einen konkreten, laufenden Entwicklungsworkflow eingebunden werden kann (vgl. Bettenburg et al., 2008, S. 308 f.; Lucassen et al., 2016, S. 383 f. und S. 386 f.; Chaparro et al., 2017, S. 401 ff.).

Generative KI und große Sprachmodelle eröffnen für diese Aufgabe neue Möglichkeiten. Brown et al. zeigen am Beispiel GPT-3, dass ein großes Sprachmodell Aufgaben ohne Gewichtsaktualisierung oder aufgabenspezifisches Fine-Tuning bearbeiten kann, wenn Aufgabenbeschreibung und Beispiele im Prompt bereitgestellt werden (vgl. Brown et al., 2020, S. 1 und S. 3). Daraus lässt sich für diese Arbeit ableiten, dass große Sprachmodelle grundsätzlich dafür geeignet sein können, natürlichsprachliche Ticketinformationen flexibel zu verarbeiten. Für die Anforderungsaufbereitung ist insbesondere relevant, dass solche Modelle nicht nur Texte zusammenfassen, sondern Inhalte klassifizieren, fehlende Informationen identifizieren, Rückfragen formulieren und bestehende Beschreibungen in strukturierte Formate überführen können. Damit eignen sie sich prinzipiell als Assistenzsysteme, die unstrukturierte Ticketinformationen analysieren und daraus einen konsistenteren Vorschlag für Titel, Beschreibung, Akzeptanzkriterien oder offene Fragen ableiten.

Über die reine Textgenerierung hinaus werden große Sprachmodelle zunehmend als handlungsfähige Komponenten in Werkzeugketten betrachtet. Der ReAct-Ansatz verbindet sprachliche Zwischenschritte mit aufgabenspezifischen Aktionen, sodass ein Modell Informationen aus externen Quellen einbeziehen und sein Vorgehen nachvollziehbarer machen kann (vgl. Yao et al., 2023, S. 1 f.). Toolformer zeigt ergänzend, dass Sprachmodelle den Einsatz externer Werkzeuge über APIs als Teil der Problemlösung nutzen können (vgl. Schick et al., 2023, S. 1). Für einen Developer Assistant ist dieser Gedanke zentral: Das System soll nicht isoliert einen Text erzeugen, sondern auf Ereignisse in GitLab reagieren, Ticketdaten und Kommentare abrufen, eine Analyse anstoßen und das Ergebnis wieder im Entwicklungskontext bereitstellen. GitLab-Webhooks ermöglichen hierfür eine ereignisbasierte Integration, bei der externe Anwendungen über Änderungen wie Issue-Updates oder neue Kommentare informiert werden können (vgl. GitLab Docs, o. J., o. S.).

Der Einsatz generativer KI in einem solchen Prozess ist jedoch nicht ohne Risiken. Generative KI kann überzeugend formulierte, aber falsche oder nicht belegte Inhalte erzeugen; NIST beschreibt dieses Risiko als Confabulation beziehungsweise Hallucination (vgl. NIST, 2024, S. 4 und S. 6). Für ein System, das Anforderungen für Entwicklerinnen und Entwickler aufbereitet, wäre dies besonders problematisch, weil unzutreffende Annahmen direkt in Umsetzungsvorschläge, Akzeptanzkriterien oder technische Entscheidungen einfließen könnten. Hinzu kommt, dass LLM-integrierte Anwendungen für Angriffe wie Prompt Injection anfällig sein können, wenn unkontrollierte Eingaben das Verhalten des Systems beeinflussen (vgl. NIST, 2024, S. 11). Ein Developer Assistant muss daher nicht nur gute Vorschläge erzeugen, sondern seine Ausgaben technisch begrenzen, validieren und in einen kontrollierten Freigabeprozess einbetten.

Vor diesem Hintergrund befasst sich die vorliegende Bachelorarbeit mit der Konzeption und Entwicklung eines KI-gestützten Developer Assistants zur automatisierten Anforderungsaufbereitung in Softwareentwicklungsprozessen. Der prototypische Assistant, im Folgenden Dev-Assist genannt, reagiert über GitLab-Webhooks auf neu erstellte oder aktualisierte Tickets sowie auf relevante Kommentare. Anschließend analysiert er Titel, Beschreibung, Diskussion und weitere Ticketinformationen mithilfe genera-

tiver KI und überführt den vorhandenen Kontext in ein standardisiertes Format. Je nach Informationslage soll das System entweder gezielte Rückfragen stellen oder einen strukturierten Vorschlag für die weitere Bearbeitung erzeugen. Der aufbereitete Kontext dient dabei als Grundlage für nachgelagerte Entwicklungsprozesse und soll die Übergabe zwischen Anforderung, Umsetzung und Validierung verbessern.

Die Arbeit untersucht damit zwei eng verbundene Fragestellungen. Erstens ist zu klären, welche Informationen ein GitLab-Ticket enthalten muss, damit es von Entwicklerinnen und Entwicklern nachvollziehbar umgesetzt und überprüft werden kann. Zweitens ist zu untersuchen, wie ein KI-gestützter Assistant technisch so in einen GitLab-basierten Entwicklungsworkflow integriert werden kann, dass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen. Dazu gehören neben der Webhook-Verarbeitung und der Kommunikation mit GitLab-APIs auch die Strukturierung der KI-Ausgabe, die Erkennung ungültiger Antworten, der Umgang mit Bild- und Kommentar-Kontext sowie die Frage, welche zusätzlichen Skills oder Validierungsschritte erforderlich sind, um eine möglichst fehlerarme Umsetzung zu unterstützen.

Ziel der Arbeit ist nicht, menschliche Abstimmung in der Anforderungsanalyse zu ersetzen. Vielmehr soll untersucht werden, wie ein Assistenzsystem wiederkehrende Aufbereitungsaufgaben übernehmen und Entwicklungsteams dadurch entlasten kann. Der Beitrag der Arbeit liegt deshalb in einem umgesetzten Prototyp sowie in der Bewertung, welche Architektur- und Validierungsentscheidungen notwendig sind, damit generative KI in einem realistischen Ticket-Workflow praktisch nutzbar wird. Die folgenden Kapitel führen zunächst in die Grundlagen von Requirements Engineering, Ticketqualität, generativer KI und agentenbasierten Werkzeugketten ein. Anschließend werden Konzeption, Implementierung und Validierung des Dev-Assist-Prototyps beschrieben. Abschließend werden die Ergebnisse eingeordnet, Grenzen des Ansatzes diskutiert und mögliche Weiterentwicklungen aufgezeigt.

2 Theoretische und technische Grundlagen

Dieses Kapitel schafft die fachliche und technische Grundlage für die Konzeption des KI-gestützten GitLab-Assistenten. Im Mittelpunkt stehen drei Fragen: Erstens ist zu klären, welche Eigenschaften Software-Tickets besitzen müssen, damit sie für Entwicklungsteams nutzbar sind. Zweitens ist zu beschreiben, welche Rolle GitLab-Issues, Kommentare und Webhooks in einem ereignisbasierten Entwicklungsworkflow einnehmen. Drittens werden Large Language Models, KI-Agenten, Opencode sowie die verwendete Node.js-/TypeScript-/Express-Umgebung eingeordnet, damit die spätere Architektur des Prototyps nachvollziehbar begründet werden kann.

2.1 Requirements Engineering und Anforderungen in Softwareentwicklungsprozessen

Requirements Engineering bezeichnet den Teil der Softwareentwicklung, in dem Anforderungen erhoben, analysiert, dokumentiert, geprüft und verwaltet werden. Anforderungen bilden die Grundlage dafür, welche Funktionalität ein System bereitstellen soll, welche Qualitätsmerkmale es erfüllen muss und unter welchen Randbedingungen es betrieben wird. Damit sind Anforderungen nicht nur fachliche Beschreibungen, sondern auch verbindliche Kommunikationsartefakte zwischen Auftraggebern, Nutzenden, Produktverantwortlichen, Entwicklung und Qualitätssicherung.

Die Bedeutung dieses Tätigkeitsbereichs zeigt sich auch in normativen Grundlagen. Die öffentlich zugängliche Katalogseite zu ISO/IEC/IEEE 29148 beschreibt Requirements Engineering als Prozessbereich, der Anforderungen für Systeme und Softwareprodukte über den Lebenszyklus hinweg hervorbringt, zugehörige Informationsprodukte definiert und Vorgaben für deren Inhalte und Formate bereitstellt (vgl. ISO/IEC/IEEE, 2018, o. S.). Für diese Arbeit wird daraus nur die allgemeine Einordnung des Standards herangezogen; detaillierte normative Abschnittsvorgaben des vollständigen Standards werden nicht vorausgesetzt. Besonders relevant ist daran, dass Anforderungen nicht isoliert als einzelne Texte betrachtet werden können. Sie stehen in einem Prozesszusammenhang: Eine Anforderung muss verstanden, umgesetzt, überprüft und bei Änderungen nachvollziehbar angepasst werden können.

Eine gute Anforderung sollte daher mehrere Qualitätsmerkmale erfüllen. Sie muss verständlich formuliert sein, damit alle Beteiligten denselben Sachverhalt meinen. Sie muss eindeutig genug sein, damit Entwicklungsteams nicht mehrere widersprüchliche Interpretationen ableiten. Sie muss prüfbar sein, damit Umsetzung und Akzeptanz nicht nur subjektiv bewertet werden. Außerdem muss sie ausreichend vollständig sein, damit

zentrale fachliche und technische Informationen nicht erst während der Implementierung rekonstruiert werden müssen. Die öffentlich zugängliche Katalogseite zu ISO/IEC 25010 ordnet Qualitätsmodelle unter anderem als Hilfsmittel ein, um Anforderungen zu spezifizieren, ihre Vollständigkeit zu validieren, Testziele abzuleiten und Akzeptanzkriterien zu bestimmen (vgl. ISO/IEC, 2023, o. S.). Auch wenn ISO/IEC 25010 primär ein Produktqualitätsmodell beschreibt, ist der Bezug zur Anforderungsarbeit deutlich: Qualität muss so beschrieben werden, dass sie später bewertet werden kann.

In agilen Entwicklungsprozessen werden Anforderungen häufig nicht in umfangreichen Spezifikationsdokumenten, sondern in kleineren Artefakten wie User Stories, Tasks oder Tickets festgehalten. Diese Form reduziert den Dokumentationsaufwand und unterstützt iterative Entwicklung, erhöht aber zugleich die Abhängigkeit von klarer Kommunikation. Inayat et al. beschreiben agile Requirements Engineering als eine stärker kollaborative Form der Anforderungsarbeit, in der Planung, Ausführung und Begründung von Anforderungen eng mit Kommunikation im Team verbunden sind (vgl. Inayat et al., 2015, S. 915 ff.). Für den Kontext dieser Arbeit bedeutet das: Wenn Anforderungen in GitLab-Issues und Kommentaren entstehen, dann entscheidet die Qualität dieser Artefakte darüber, ob ein Entwicklungsteam ohne zusätzliche Reibung weiterarbeiten kann.

Lucassen et al. zeigen am Beispiel von User Stories, dass solche kurzen, natürlichsprachlichen Anforderungen in der Praxis weit verbreitet sind, aber häufig Qualitätsdefekte aufweisen. Das Quality User Story Framework unterscheidet Qualitätskriterien unter anderem in syntaktische, semantische und pragmatische Aspekte (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Syntaktische Qualität betrifft dabei die Form einer Anforderung, semantische Qualität die inhaltliche Bedeutung und Konsistenz, pragmatische Qualität die Nutzbarkeit für die Beteiligten. Diese Einordnung lässt sich auf Software-Tickets übertragen: Ein Ticket kann sprachlich lesbar sein, aber dennoch für Entwicklerinnen und Entwickler unzureichend bleiben, wenn erwartetes Verhalten, Kontext, Akzeptanzkriterien oder technische Randbedingungen fehlen.

Für die vorliegende Arbeit ergibt sich daraus ein grundlegendes Verständnis: Ein GitLab-Issue ist nicht automatisch eine gute Anforderung. Es ist zunächst ein Container für Titel, Beschreibung, Kommentare, Metadaten und Anhänge. Erst wenn diese Informationen so strukturiert sind, dass sie fachlich verstanden, technisch umgesetzt und später überprüft werden können, erfüllt das Ticket die Rolle einer verwertbaren Anforderung. Der geplante Developer Assistant setzt genau an dieser Stelle an: Er soll vorhandene Ticketinformationen analysieren, fehlende Bestandteile sichtbar machen und vorhandenen Kontext in eine besser nutzbare Form überführen.

2.2 Tickets, User Stories und Bug Reports als Arbeitsartefakte

Software-Tickets dienen in Entwicklungsprozessen als Arbeitsartefakte. Sie beschreiben Aufgaben, Fehler, Änderungswünsche oder fachliche Anforderungen und verbinden diese Informationen mit Status, Zuständigkeiten, Kommentaren und weiteren Metadaten. In GitLab werden solche Artefakte typischerweise als Issues geführt. Sie können eine

kurze Problem- oder Anforderungsbeschreibung enthalten, durch Kommentare ergänzt werden und im Verlauf der Bearbeitung aktualisiert werden. Damit sind Tickets sowohl Dokumentationsform als auch Prozessschnittstelle.

Zwischen User Stories, Bug Reports und allgemeinen Tickets bestehen Unterschiede, die für die Arbeit wichtig sind. Eine User Story beschreibt meist einen gewünschten fachlichen Nutzen aus Sicht einer Rolle, häufig in der Form "Als ... möchte ich ..., damit ...". Ein Bug Report beschreibt dagegen eine Abweichung zwischen beobachtetem und erwartetem Systemverhalten. Ein allgemeines Ticket kann beide Formen enthalten oder auch eine technische Aufgabe beschreiben. Gemeinsam ist diesen Artefakten, dass sie in natürlicher Sprache formuliert werden und häufig von Personen mit unterschiedlichem technischen Hintergrund erstellt oder ergänzt werden. Dadurch entstehen Informationslücken und Interpretationsspielräume.

Bug Reports sind in der Forschung gut untersucht und eignen sich deshalb als Referenz für Ticketqualität. Bettenburg et al. zeigen, dass Bug Reports für Entwicklerinnen und Entwickler wichtige Informationen enthalten, aber stark in ihrer Qualität variieren. Besonders deutlich ist die Diskrepanz zwischen Informationen, die Entwicklungsteams benötigen, und Informationen, die meldende Personen bereitstellen. In der Studie wurden Reproduktionsschritte, Stack Traces und Testfälle von Entwicklerseite als hilfreich bewertet, während genau diese Angaben für Reporterinnen und Reporter schwer bereitzustellen sind (vgl. Bettenburg et al., 2008, S. 308 f.). Diese Erkenntnis ist auf GitLab-Issues übertragbar: Auch dort können meldende Personen ein reales Problem beschreiben, ohne alle Informationen zu liefern, die für die Umsetzung oder Fehleranalyse erforderlich sind.

Breu et al. betonen, dass Bug-Tracking-Systeme eine zentrale Rolle in der Zusammenarbeit zwischen Nutzenden und Entwicklungsteams spielen. Ihre Analyse von Fragen in 600 Bug Reports aus Mozilla- und Eclipse-Projekten zeigt, dass die aktive Beteiligung der Nutzenden über das reine Melden eines Fehlers hinausgeht (vgl. Breu et al., 2010, S. 301). Besonders relevant ist die Kategorie "Missing Information": Entwicklerinnen und Entwickler fragen unter anderem nach Reproduktionsschritten, Build-Nummern, Betriebssystemen, Testfällen, Beispielen, Programmausgaben und Screenshots (vgl. Breu et al., 2010, S. 303). Ein Ticket ist damit nicht nur ein statischer Text, sondern häufig ein Gesprächsverlauf, in dem fehlende Informationen nach und nach ergänzt werden.

Für einen GitLab-Assistenten ist dieser Gesprächscharakter zentral. Wenn ein Issue erstellt wird, liegt der vollständige Entwicklungskontext oft noch nicht vor. Kommentare können zusätzliche Hinweise, Rückfragen, Screenshots oder Entscheidungen enthalten. Dadurch entsteht ein verteilter Kontext: Ein Teil steht im Titel, ein Teil in der Beschreibung, ein Teil in späteren Kommentaren und ein weiterer Teil möglicherweise in Bildern oder Links. Ein Assistenzsystem, das Ticketqualität verbessern soll, darf daher nicht nur die ursprüngliche Beschreibung betrachten. Es muss den gesamten verfügbaren Ticketkontext einbeziehen.

Lucassen et al. ergänzen diese Perspektive für agile Anforderungen. User Stories sind zwar als kurze Anforderungen praktikabel, ihre Qualität hängt jedoch davon ab, ob sie inhaltlich und sprachlich ausreichend präzise sind (vgl. Lucassen et al., 2016, S. 383 f.). Übertragen auf GitLab-Issues bedeutet dies: Kürze ist kein Qualitätsmerkmal an sich. Ein knappes Ticket kann gut sein, wenn es Ziel, Kontext, erwartetes Ergebnis und

Prüfkriterien eindeutig beschreibt. Es kann aber auch unbrauchbar sein, wenn es nur eine vage Idee enthält. Der Developer Assistant muss deshalb nicht nur "mehr Text" erzeugen, sondern relevante Informationen strukturieren und Lücken gezielt benennen.

2.3 Qualität von Ticketinformationen

Die Qualität von Ticketinformationen lässt sich aus mehreren Perspektiven betrachten. Aus fachlicher Sicht muss ein Ticket beschreiben, welches Problem gelöst oder welche Funktion umgesetzt werden soll. Aus technischer Sicht müssen Informationen vorhanden sein, die eine Umsetzung oder Analyse ermöglichen. Aus Sicht der Qualitätssicherung müssen erwartetes Verhalten und Akzeptanzkriterien prüfbar sein. Aus Sicht des Projektmanagements sollte erkennbar sein, welche Aufgabe zu erledigen ist, welche Abhängigkeiten bestehen und wann die Bearbeitung abgeschlossen werden kann.

Für Bug Reports identifizieren Chaparro et al. drei besonders wichtige Informationstypen: Observed Behavior, Steps to Reproduce und Expected Behavior. Ein wirksamer Bug Report sollte also beschreiben, was tatsächlich passiert ist, welche Schritte zu diesem Verhalten führen und welches Verhalten stattdessen erwartet wurde (vgl. Chaparro et al., 2017, S. 396 f.). Diese Struktur ist auch für allgemeine Tickets hilfreich. Bei Fehlern ist sie unmittelbar anwendbar. Bei neuen Anforderungen entspricht sie sinngemäß der Frage, welcher Ist-Zustand oder Bedarf besteht, welches Zielverhalten erreicht werden soll und anhand welcher Kriterien die Umsetzung nachvollzogen werden kann.

Chaparro et al. zeigen außerdem, dass diese Informationen häufig fehlen. Sie entwickelten mit DeMIBuD einen Ansatz, der fehlendes Expected Behavior und fehlende Steps to Reproduce in Bug Descriptions erkennen kann (vgl. Chaparro et al., 2017, S. 397 und S. 401 ff.). Für die vorliegende Arbeit ist weniger die konkrete technische Umsetzung von DeMIBuD entscheidend, sondern das Grundprinzip: Ticketqualität lässt sich teilweise automatisiert prüfen, wenn relevante Informationsbestandteile bekannt sind. Dev-Assist folgt einem ähnlichen Ziel, soll jedoch nicht nur fehlende Bestandteile erkennen, sondern im GitLab-Workflow Rückfragen oder strukturierte Vorschläge erzeugen.

Die Qualität von Reproduktionsschritten verdient besondere Aufmerksamkeit. Chaparro et al. untersuchen in einer späteren Arbeit die Qualität der Steps to Reproduce und beschreiben, dass schlechte Reproduktionsschritte zu zusätzlichem manuellem Aufwand bei Triaging und Fehlerbehebung führen. Ihr Ansatz Euler identifiziert und bewertet Reproduktionsschritte und gibt Rückmeldung zur Verbesserung des Bug Reports (vgl. Chaparro et al., 2019, S. 152). Daraus folgt für Software-Tickets: Nicht nur das Vorhandensein eines Abschnitts ist relevant, sondern auch dessen Nutzbarkeit. Reproduktionsschritte müssen konkret, nachvollziehbar und in einer plausiblen Reihenfolge beschrieben sein.

Song und Chaparro stellen mit BEE ein Werkzeug vor, das Bug Reports automatisch analysiert und Rückmeldung zu Observed Behavior, Expected Behavior und Steps to Reproduce gibt. BEE erkennt unter anderem, ob ein Issue einen Bug, eine Erweiterung oder eine Frage beschreibt, markiert die Struktur innerhalb einer Bug Description und erkennt fehlende Elemente (vgl. Song/Chaparro, 2020, S. 1551 f.). Diese Arbeit zeigt,

dass strukturierte Analyse von Tickettexten nicht nur theoretisch möglich ist, sondern praktisch als Werkzeug in Entwicklungsplattformen eingebunden werden kann. Für Dev-Assist ist daran besonders relevant, dass Ticketanalyse nicht als isolierte Textklassifikation verstanden wird, sondern als Unterstützung für Reporterinnen, Reporter und Entwicklungsteams.

Aus diesen Arbeiten lassen sich konkrete Qualitätsdimensionen für GitLab-Tickets ableiten. Ein Ticket sollte einen aussagekräftigen Titel besitzen, der den Gegenstand knapp benennt. Die Beschreibung sollte Problem oder Ziel, Kontext, erwartetes Ergebnis, relevante Schritte, technische Randbedingungen und Akzeptanzkriterien enthalten. Kommentare sollten nicht nur lose Diskussionen bleiben, sondern in den Ticketkontext einfließen, wenn sie entscheidungsrelevante Informationen enthalten. Anhänge und Screenshots sollten so referenziert sein, dass ihre Bedeutung erkennbar ist. Außerdem sollte ein Ticket zwischen gesicherten Informationen, Annahmen und offenen Fragen unterscheiden.

Gerade an dieser Unterscheidung setzt ein KI-gestützter Assistent an. Wenn Informationen fehlen, sollte er nicht frei spekulieren, sondern Rückfragen formulieren. Wenn genügend Kontext vorhanden ist, kann er einen strukturierten Vorschlag erzeugen. Wenn Kommentare widersprüchlich sind, sollte dies als Unsicherheit sichtbar werden. Ticketqualität entsteht also nicht allein durch sprachliche Glättung, sondern durch die kontrollierte Verarbeitung von Kontext.

2.4 Generative KI und Large Language Models

Large Language Models sind Sprachmodelle, die auf sehr großen Textmengen trainiert werden und natürliche Sprache verstehen, fortsetzen und erzeugen können. Die Grundlage vieler moderner Modelle bildet die Transformer-Architektur. Vaswani et al. schlagen den Transformer als Architektur vor, die auf Attention-Mechanismen basiert und auf rekurrente oder konvolutionale Strukturen verzichtet (vgl. Vaswani et al., 2017, S. 1 f.). Für diese Arbeit ist keine detaillierte mathematische Beschreibung der Transformer-Architektur erforderlich. Entscheidend ist, dass Transformer-basierte Modelle lange Eingaben verarbeiten, Beziehungen zwischen Textbestandteilen gewichten und dadurch komplexe Sprachaufgaben lösen können.

Brown et al. zeigen am Beispiel GPT-3, dass große Sprachmodelle Aufgaben aus Instruktionen und wenigen Beispielen im Prompt bearbeiten können, ohne dass für jede Aufgabe ein eigenes Fine-Tuning oder eine Gewichtsaktualisierung erforderlich ist. Sie bezeichnen diesen Mechanismus als In-Context Learning (vgl. Brown et al., 2020, S. 1 und S. 3). Für Dev-Assist ist das relevant, weil Ticketanalyse eine variable Aufgabe ist. Jedes Issue enthält andere Formulierungen, andere Kommentare, andere fachliche Inhalte und unterschiedliche Informationslücken. Ein starres Regelwerk wäre schwer zu pflegen. Ein LLM kann dagegen auf Basis eines Prompts angewiesen werden, Ticketinformationen zu prüfen, Rückfragen zu stellen oder einen Vorschlag in einem standardisierten Format zu erzeugen.

Zhao et al. beschreiben Large Language Models als Ergebnis einer Entwicklung von klassischen Sprachmodellen zu großen, vortrainierten Modellen, die durch Skalierung neue

Fähigkeiten zeigen können. Der Survey ordnet LLMs unter anderem nach Pre-Training, Anpassung, Nutzung und Evaluation ein (vgl. Zhao et al., 2023, S. 1). Für diese Arbeit ist vor allem die Nutzungsseite relevant. Ein LLM wird nicht selbst trainiert, sondern über eine Agentenschnittstelle in einen bestehenden Workflow eingebunden. Die Qualität des Ergebnisses hängt deshalb stark von Promptgestaltung, Kontextauswahl, Ausgabeformat und Validierung ab.

Prompt Engineering bezeichnet die Gestaltung von Eingaben, mit denen das Modell zu einer gewünschten Aufgabe angeleitet wird. Zhao et al. beschreiben Prompt Engineering als manuelles Erstellen geeigneter Prompts, um Fähigkeiten von LLMs für bestimmte Aufgaben hervorzurufen (vgl. Zhao et al., 2023, S. 44). Im Kontext dieser Arbeit bedeutet das: Der Prompt muss dem Modell nicht nur sagen, dass ein Ticket analysiert werden soll. Er muss auch definieren, welche Informationen relevant sind, welche Ausgabeform erwartet wird, wann Rückfragen gestellt werden sollen und wie mit Bildern oder Kommentaren umzugehen ist.

Gleichzeitig darf die Leistungsfähigkeit von LLMs nicht überschätzt werden. Sie erzeugen plausible Sprache, besitzen aber kein garantiert korrektes Verständnis des tatsächlichen Systems. Sie können aus unvollständigem Kontext Annahmen ableiten, die fachlich nicht belegt sind. Deshalb ist in dieser Arbeit wichtig, LLMs nicht als autonome Entscheider zu behandeln. Sie dienen als Assistenzkomponente, die Vorschläge formuliert und Informationslücken sichtbar macht. Die Verantwortung für endgültige Ticketänderungen bleibt beim Menschen beziehungsweise beim kontrollierten Publish-Prozess.

Für die spätere Implementierung folgt daraus eine technische Anforderung: Die Ausgabe des Modells muss maschinenlesbar und prüfbar sein. Wenn Dev-Assist eine Entscheidung zwischen Rückfrage und Vorschlag treffen soll, reicht ein freier Fließtext nicht aus. Die Antwort muss ein definiertes Format besitzen, etwa mit Feldern für `hasQuestions`, `questions`, `proposedTitle` und `proposedDescription`. Nur so kann das System die Antwort weiterverarbeiten, Fehler erkennen und robuste Tests schreiben.

2.5 LLM-basierte Agenten, Werkzeugnutzung und Opencode

Ein LLM-basierter Agent unterscheidet sich von einem einfachen Chatbot dadurch, dass er nicht nur eine Antwort auf eine Nutzereingabe erzeugt, sondern in einen Handlungskontext eingebettet ist. Er kann Informationen aufnehmen, Zwischenschritte ausführen, Werkzeuge nutzen und Ergebnisse wieder in eine Umgebung zurückspielen. Xi et al. beschreiben LLM-basierte Agenten als Systeme, in denen große Sprachmodelle als Grundlage für Agenten dienen. Ihr allgemeines Framework besteht aus den Komponenten Brain, Perception und Action (vgl. Xi et al., 2023, S. 1). Übertragen auf Dev-Assist bedeutet dies: Das Modell verarbeitet den Ticketkontext, nimmt GitLab-Issue-Daten und Kommentare als Wahrnehmung auf und erzeugt als Handlung Rückfragen oder strukturierte Vorschläge.

Yao et al. verbinden im ReAct-Ansatz sprachliches Schlussfolgern mit Aktionen. Das Modell erzeugt Reasoning Traces und aufgabenspezifische Aktionen in verschränkter

Form, wodurch es Informationen aus externen Quellen oder Umgebungen einbeziehen kann (vgl. Yao et al., 2023, S. 1 f.). Für die vorliegende Arbeit ist daran nicht die exakte ReAct-Implementierung entscheidend, sondern das Prinzip: Ein KI-System im Entwicklungsworkflow sollte Kontext aus der Umgebung einbeziehen und nicht nur auf eine isolierte Texteingabe reagieren. Dev-Assist ruft deshalb Issue-Daten, Kommentare und Bildreferenzen ab, bevor eine Analyse gestartet wird.

Schick et al. zeigen mit Toolformer, dass Sprachmodelle lernen können, externe Werkzeuge über einfache APIs einzusetzen. Toolformer entscheidet unter anderem, welche APIs aufgerufen werden, wann sie aufgerufen werden, welche Argumente verwendet werden und wie Ergebnisse in die weitere Vorhersage einfließen (vgl. Schick et al., 2023, S. 1). Auch hier steht für diese Arbeit nicht das konkrete Verfahren im Vordergrund, sondern der Architekturgedanke: Leistungsfähige LLM-Anwendungen entstehen nicht nur durch Modellantworten, sondern durch die Kombination aus Modell, Werkzeugen, Datenquellen und kontrollierten Schnittstellen.

OpenCode wird in dieser Arbeit als Agentenschnittstelle eingesetzt. Die offizielle Dokumentation beschreibt Agents als spezialisierte KI-Assistenten, die für bestimmte Aufgaben und Workflows konfiguriert werden können, einschließlich eigener Prompts, Modelle und Tool-Zugriffe (vgl. OpenCode Docs, o. J.a, Abschnitt "Agents"). Die SDK-Dokumentation beschreibt den JS/TS-Client als typischere Schnittstelle, mit der Integrationen gebaut und OpenCode programmatisch gesteuert werden können (vgl. OpenCode Docs, o. J.b, Abschnitt "SDK"). Für Dev-Assist ist diese Eigenschaft zentral, weil der GitLab-Webhook nicht interaktiv im Terminal arbeitet, sondern serverseitig eine Agentensession starten, einen Prompt senden und die Antwort weiterverarbeiten muss.

Die OpenCode-Tool-Dokumentation beschreibt zudem, dass Tools dem LLM Aktionen in der Codebasis ermöglichen und dass Tool-Verhalten über Berechtigungen gesteuert werden kann (vgl. OpenCode Docs, o. J.c, Abschnitt "Tools"). Im Rahmen dieser Arbeit wird diese Werkzeugperspektive vor allem konzeptionell relevant: Ein Agent braucht begrenzte, nachvollziehbare Fähigkeiten. Für den Dev-Assist-Prototyp bedeutet das, dass die eigentliche GitLab-Verarbeitung nicht beliebig durch das Modell erfolgt. Stattdessen ruft die Anwendung kontrolliert Daten ab, übergibt einen definierten Kontext an den Agenten und verarbeitet die Antwort nach festen Regeln.

Damit ergibt sich eine klare Rollenverteilung. GitLab liefert den Ereignis- und Ticketkontext. Die Anwendung entscheidet, ob ein Ereignis relevant ist. OpenCode stellt die Agentenschnittstelle bereit. Das LLM erzeugt Rückfragen oder Vorschläge. Die Anwendung validiert die Antwort und veröffentlicht sie kontrolliert als Kommentar oder übernimmt sie nach einem Publish-Kommando. Diese Trennung ist wichtig, weil sie den Handlungsspielraum des Modells begrenzt und die spätere Testbarkeit erhöht.

2.6 GitLab, Webhooks und technische Integrationsplattform

GitLab ist im Kontext dieser Arbeit die zentrale Entwicklungs- und Integrationsplattform. Issues bilden die primären Ticketartefakte. Kommentare beziehungsweise Notes

ergänzen den Verlauf und enthalten Rückfragen, Hinweise oder Freigabekommandos. Discussions strukturieren Gesprächsverläufe in Threads. Webhooks verbinden GitLab mit externen Systemen, indem sie Ereignisse als HTTP-Anfragen an konfigurierte Endpunkte senden.

Die GitLab-Dokumentation beschreibt Webhooks als Mechanismus, der GitLab mit anderen Tools und Systemen über Echtzeitbenachrichtigungen verbindet. Bei wichtigen Ereignissen sendet GitLab Informationen an externe Anwendungen, sodass Automatisierungsworkflows auf Merge Requests, Code Pushes oder Issue-Updates reagieren können (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook events"). Die Webhook-Events-Dokumentation ergänzt, dass Webhooks HTTP-POST-Anfragen mit detaillierten Informationen an konfigurierte Endpunkte senden, wenn bestimmte Ereignisse eintreten. Dazu gehören unter anderem Kommentare auf Issues (vgl. GitLab Docs, o. J.b, Abschnitt "Comment events").

Für Dev-Assist sind besonders Issue- und Note-Events relevant. Ein Issue-Event kann ausgelöst werden, wenn ein Issue erstellt oder aktualisiert wird. Ein Note-Event kann entstehen, wenn ein Kommentar hinzugefügt wird. Dadurch kann der Assistent auf zwei Arten aktiviert werden: durch ein neu erstelltes oder aktualisiertes Ticket, das `@dev-assist` erwähnt, oder durch einen Kommentar, der den Assistenten anspricht. Diese Ereignisorientierung passt zur Zielsetzung, weil der Assistent nicht dauerhaft pollend nach Änderungen suchen muss, sondern auf konkrete GitLab-Ereignisse reagieren kann.

Die GitLab Issues API zeigt, dass Issues unter anderem Titel, Beschreibung, Status, Autorenschaft, Labels, Assignees und weitere Metadaten enthalten können. Die Beschreibung eines Issues ist ein eigenes Feld und kann aktualisiert werden (vgl. GitLab Docs, o. J.c, o. S.). Die Notes API ermöglicht das Abrufen von Kommentaren zu einem Issue, einschließlich Sortierung und Filterung nach Kommentartypen (vgl. GitLab Docs, o. J.d, o. S.). Die Discussions API erlaubt das Erstellen, Ergänzen, Aktualisieren und Löschen von Issue-Threads und Notizen (vgl. GitLab Docs, o. J.e, o. S.). Diese APIs bilden die technische Grundlage dafür, dass Dev-Assist Ticketkontext lesen, Rückfragen oder Vorschläge posten und später Vorschläge übernehmen kann.

Technisch wird der Prototyp als Node.js-/TypeScript-/Express-Anwendung umgesetzt. Node.js ist eine freie, quelloffene und plattformübergreifende JavaScript-Laufzeitumgebung, mit der unter anderem Server, Webanwendungen, Kommandozeilenwerkzeuge und Skripte erstellt werden können (vgl. Node.js Docs, o. J.a, o. S.). Die HTTP-Dokumentation zeigt, dass Node.js HTTP-Server und -Clients bereitstellt und Anfragen sowie Antworten niedrigschwellig verarbeiten kann (vgl. Node.js Docs, o. J.b, o. S.). Für einen Webhook-Endpunkt ist diese Grundlage geeignet, weil GitLab Webhook-Ereignisse als HTTP-POST-Anfragen sendet.

Express ergänzt Node.js um ein minimalistisches und flexibles Webframework. Die Express-Dokumentation beschreibt Express als Framework für Webanwendungen und APIs mit HTTP-Hilfsmethoden und Middleware-Unterstützung (vgl. Express Docs, o. J., o. S.). Für Dev-Assist ist Express zweckmäßig, weil der Webhook-Endpunkt eingehende Requests entgegennehmen, Header prüfen, JSON-Payloads verarbeiten und schnell eine HTTP-Antwort an GitLab zurückgeben muss. Die eigentliche Analyse kann anschließend asynchron weiterlaufen, damit GitLab nicht auf lang laufende KI-

Verarbeitung warten muss.

TypeScript wird eingesetzt, um die dynamische JavaScript-Laufzeit um statische Typprüfung zu ergänzen. Das TypeScript-Handbook beschreibt TypeScript als statischen Typechecker für JavaScript-Programme, der vor der Ausführung prüft, ob Typen korrekt verwendet werden (vgl. TypeScript Docs, o. J., o. S.). Für den Prototyp ist das besonders relevant, weil GitLab-Payloads, API-Antworten und Agentenantworten strukturierte Daten enthalten, aber aus externen Quellen stammen. Typisierung unterstützt Wartbarkeit und Testbarkeit, ersetzt jedoch keine Laufzeitvalidierung. Deshalb muss die Anwendung zusätzlich prüfen, ob die vom Agenten gelieferten JSON-Strukturen tatsächlich dem erwarteten Format entsprechen.

Aus technischer Sicht ergibt sich damit ein Integrationsmuster: GitLab sendet ein Webhook-Ereignis an Express. Die Anwendung validiert grundlegende Eigenschaften des Requests, liest Issue- und Kommentar-Kontext über GitLab-Schnittstellen, startet eine Opencode-Agentenanalyse und schreibt das Ergebnis als Kommentar zurück. Bei einem Publish-Kommando wird ein zuvor erzeugter Vorschlag übernommen. Diese Kette verbindet Ereignisverarbeitung, API-Integration, KI-Analyse und kontrollierte Änderung des Tickets.

2.7 Risiken und Validierungsbedarf bei generativer KI und Webhooks

Der Einsatz generativer KI in einem Ticket-Workflow erzeugt Nutzen, aber auch Risiken. Das wichtigste fachliche Risiko besteht darin, dass ein Modell plausibel klingende, aber falsche oder nicht belegte Informationen erzeugt. Das National Institute of Standards and Technology beschreibt dieses Risiko als Confabulation beziehungsweise Hallucination und meint damit überzeugend dargestellte, aber fehlerhafte oder falsche Inhalte (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Huang et al. beschreiben Halluzinationen bei LLMs ähnlich als plausible, aber nicht faktische Inhalte, die die Zuverlässigkeit realer Informationssysteme beeinträchtigen können (vgl. Huang et al., 2023, S. 1 f.).

Für Dev-Assist ist dieses Risiko besonders relevant, weil Ticketvorschläge unmittelbare Auswirkungen auf Entwicklungsarbeit haben können. Wenn ein Assistent unbelegte Annahmen als Akzeptanzkriterien formuliert, kann daraus eine falsche Implementierung entstehen. Wenn er technische Ursachen erfindet, kann die Fehlersuche in eine falsche Richtung gelenkt werden. Wenn er fehlende Informationen nicht erkennt, wirkt das Ticket vollständiger, als es tatsächlich ist. Deshalb muss der Prototyp so konzipiert werden, dass er bei unzureichendem Kontext Rückfragen stellt und Vorschläge nicht automatisch ohne Freigabe übernimmt.

Ein zweites Risiko betrifft Prompt Injection. Liu et al. untersuchen Prompt-Injection-Angriffe gegen LLM-integrierte Anwendungen und zeigen, dass reale Anwendungen anfällig sein können; in ihrer Untersuchung waren 31 von 36 betrachteten Anwendungen gegenüber dem entwickelten Angriff anfällig (vgl. Liu et al., 2023, S. 1). OWASP beschreibt Prompt Injection als Schwachstelle, bei der Nutzereingaben das Verhalten oder die Ausgabe eines LLM in unbeabsichtigter Weise verändern können. Dabei un-

terscheidet OWASP direkte und indirekte Prompt Injection, wobei indirekte Angriffe über externe Quellen wie Dateien oder Webseiten erfolgen können (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection").

In einem GitLab-Assistenten können potenziell alle vom Modell gelesenen Inhalte Angriffs- oder Störsignale enthalten: Issue-Beschreibungen, Kommentare, Codeblöcke, Screenshots mit Text oder externe Links. Ein Kommentar könnte zum Beispiel versuchen, die Systemanweisungen des Assistenten zu überschreiben oder ihn dazu zu bringen, ungeprüfte Änderungen vorzunehmen. Daraus folgt, dass Ticketinhalte nicht als vertrauenswürdige Befehle behandelt werden dürfen. Sie sind Daten, die analysiert werden sollen. Entscheidungen über GitLab-Änderungen müssen in der Anwendung und nicht frei im Modell liegen.

Ein weiteres Sicherheitsproblem betrifft Webhooks selbst. Webhook-Endpunkte sind von außen erreichbare HTTP-Schnittstellen. Wenn ein Endpunkt unzureichend abgesichert ist, könnten fremde Systeme gefälschte Ereignisse senden. Nach aktueller GitLab-Dokumentation sollten neue Webhooks bevorzugt mit einem Signing Token abgesichert werden. GitLab berechnet damit eine HMAC-SHA256-Signatur über den Payload und übermittelt sie im Header `webhook-signature`, sodass der Empfänger Authentizität und Integrität der Anfrage prüfen kann. Ein Secret Token im Header `X-Gitlab-Token` bietet schwächere Garantien und wird für neue Webhooks nicht empfohlen (vgl. GitLab Docs, o. J.a, Abschnitt "Signing tokens"). Zusätzlich sollte der Zeitstempel `webhook-timestamp` geprüft werden, um Replay-Angriffe zu erschweren. Für den Prototyp ist daher wichtig, Signaturprüfung, relevante Header, erlaubte Ereignistypen und formal plausible Payloads gemeinsam zu berücksichtigen. Außerdem müssen Bot-Endlosschleifen verhindert werden: Wenn der Assistent auf seine eigenen Kommentare reagiert, kann eine unkontrollierte Folge von Webhook-Ereignissen entstehen.

Auch die Autonomie des Systems muss begrenzt werden. OWASP führt "Excessive Agency" als Risiko auf, wenn LLM-basierte Systeme über zu weitreichende Funktionalitäten, Berechtigungen oder Autonomie verfügen (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency"). Für Dev-Assist spricht dies für einen Vorschlagsmodus: Das System analysiert, fragt nach oder schlägt eine strukturierte Beschreibung vor. Die endgültige Übernahme erfolgt erst durch ein explizites Publish-Kommando. Dadurch bleibt die menschliche Kontrolle erhalten, und der Assistent kann in den Entwicklungsprozess integriert werden, ohne ungeprüft produktive Ticketinhalte zu verändern.

Validierungsbedarf besteht daher auf mehreren Ebenen. Eingehende Webhook-Payloads müssen formal plausibel sein. GitLab-API-Antworten müssen robust verarbeitet werden. Agentenantworten müssen als JSON parsebar sein und dem erwarteten Schema entsprechen. Fehlende oder widersprüchliche Informationen müssen als solche behandelt werden. Fehler beim Bildabruf oder bei der Vision-Verarbeitung dürfen nicht den gesamten Workflow blockieren. Logging muss nachvollziehbar machen, warum ein Ereignis ignoriert, analysiert oder veröffentlicht wurde. Diese Anforderungen bilden eine direkte Brücke zur späteren Anforderungsanalyse und Systemarchitektur.

2.8 Zwischenfazit und Anforderungen an Dev-Assist

Die Grundlagen zeigen, dass Software-Tickets eine zentrale Rolle als Arbeits- und Kommunikationsartefakte einnehmen. Anforderungen und Bug Reports müssen so dokumentiert sein, dass sie verstanden, umgesetzt und überprüft werden können. Forschung zu Bug Reports zeigt jedoch, dass relevante Informationen häufig fehlen oder unstrukturiert vorliegen. Besonders wichtig sind beobachtetes Verhalten, erwartetes Verhalten, Reproduktionsschritte, Kontextinformationen und klare Prüfkriterien. Gleichzeitig entstehen in GitLab-Issues Informationen nicht nur in der Beschreibung, sondern auch in Kommentaren, Diskussionen und Anhängen.

Large Language Models bieten eine Möglichkeit, solche natürlichsprachlichen und verteilten Informationen flexibel zu analysieren. Durch In-Context Learning und Prompt Engineering können sie angewiesen werden, Ticketinhalte zu strukturieren, Rückfragen zu formulieren oder Vorschläge zu erzeugen. LLM-basierte Agenten und Werkzeugnutzung zeigen darüber hinaus, dass solche Modelle in technische Workflows eingebunden werden können. Opencode stellt dafür im Prototyp die Agentenschnittstelle bereit, während GitLab über Webhooks und APIs den Ereignis- und Datenkontext liefert.

Aus den Risiken ergibt sich jedoch, dass Dev-Assist nicht als autonomer Entscheider konzipiert werden darf. Halluzinationen, Prompt Injection, fehlerhafte Payloads und zu weitreichende Handlungsrechte erfordern technische Grenzen. Der Assistent muss den Kontext analysieren, aber zwischen fehlender Information und gesicherter Information unterscheiden. Er muss Rückfragen stellen, wenn keine ausreichende Grundlage für einen Vorschlag besteht. Er muss Vorschläge in einem strukturierten Format liefern, das maschinell validiert werden kann. Und er darf Änderungen am Ticket nur über einen kontrollierten Publish-Prozess übernehmen.

Für die nachfolgende Anforderungsanalyse lassen sich daraus zentrale Anforderungen ableiten. Dev-Assist muss `@dev-assist` in Issues und Kommentaren erkennen, GitLab-Kontext über APIs abrufen, Issue-Beschreibung und Diskussion gemeinsam auswerten, Bild- und Screenshot-Kontext berücksichtigen, fehlende Informationen identifizieren, entweder Rückfragen oder strukturierte Vorschläge erzeugen, Agentenantworten zur Laufzeit validieren, eigene Bot-Ereignisse ignorieren und sicher auf fehlerhafte Eingaben reagieren. Diese Anforderungen werden im nächsten Kapitel konkretisiert und anschließend in eine Systemarchitektur überführt.

3 Anforderungsanalyse

Nachdem Kapitel 2 die fachlichen und technischen Grundlagen beschrieben hat, konkretisiert dieses Kapitel die Anforderungen an Dev-Assist. Im Mittelpunkt stehen die Anforderungen, die sich aus dem GitLab-basierten Ticketprozess, aus der Ticketqualitätsforschung und aus den Risiken generativer KI für den Prototyp ergeben. Die Anforderungsanalyse bildet damit die Brücke zwischen Grundlagen und Systemarchitektur.

Dev-Assist soll unstrukturierte oder unvollständige GitLab-Issues nicht automatisch in fertige Entwicklungsaufgaben verwandeln, sondern deren Aufbereitung unterstützen. Das System soll erkennen, wann es angesprochen wird, vorhandenen Ticketkontext auswerten, fehlende Informationen sichtbar machen, strukturierte Vorschläge erzeugen und die Übernahme nur nach einem expliziten Publish-Kommando ermöglichen. Die Anforderungen betreffen deshalb sowohl die Qualität der entstehenden Tickets als auch die robuste, nachvollziehbare und kontrollierte Integration in GitLab.

3.1 Vorgehen und Grundlagen der Anforderungsanalyse

Die Anforderungen werden aus vier Quellen abgeleitet. Erstens werden die Erkenntnisse aus Kapitel 2 zu Ticketqualität, Informationslücken und Risiken von Large Language Models herangezogen. Zweitens wird der bisherige manuelle GitLab-Prozess betrachtet. Drittens fließt die projektinterne Zielbeschreibung ein, nach der Dev-Assist über `@dev-assist` aktiviert wird, bei fehlendem Kontext Rückfragen stellt, bei ausreichendem Kontext einen Vorschlag erzeugt und diesen erst nach `@dev-assist publish` übernimmt. Viertens wird der aktuelle Prototyp berücksichtigt, um die Anforderungen mit den technischen Rahmenbedingungen abzugleichen.

Requirements Engineering umfasst nach der öffentlichen Katalogseite zu ISO/IEC/IEEE 29148 die systematische Behandlung von Anforderungen über den Lebenszyklus hinweg und beschreibt zugehörige Informationsprodukte und Prozesse (vgl. ISO/IEC/IEEE, 2018, o. S.). Für Dev-Assist bedeutet das, dass Anforderungen nicht nur Funktionen beschreiben, sondern auch benötigte Informationen, Verarbeitungsschritte und Systemgrenzen. Da GitLab-Issues in einem agilen Entwicklungskontext entstehen, ist außerdem die kollaborative Dimension relevant: Agile Requirements Engineering ist stark mit Kommunikation, gemeinsamer Klärung und kontinuierlicher Anpassung verbunden (vgl. Inayat et al., 2015, S. 915 ff.). Dev-Assist muss sich daher in bestehende Kommunikation einfügen, statt sie durch einen isolierten Analysevorgang zu ersetzen.

Die fachlichen Anforderungen orientieren sich an Arbeiten zu User Stories und Bug Reports. Lucassen et al. unterscheiden syntaktische, semantische und pragmatische Quali-

tätsaspekte von User Stories (vgl. Lucassen et al., 2016, S. 386 f.). Ein Ticketvorschlag darf deshalb nicht nur formal gegliedert sein, sondern muss inhaltlich verständlich, konsistent und für Entwicklerinnen und Entwickler nutzbar bleiben. Chaparro et al. zeigen für Bug Descriptions, dass besonders beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte häufig fehlen (vgl. Chaparro et al., 2017, S. 396 f.). Diese Befunde werden hier nicht als unmittelbarer Nachweis für alle Ticketarten verstanden. Für Dev-Assist wird daraus abgeleitet, dass GitLab-Tickets Ziel, Kontext, erwartetes Ergebnis, Akzeptanzkriterien, offene Fragen und relevante Randbedingungen möglichst explizit enthalten sollten.

Die technische Seite ergibt sich aus GitLab-Webhooks, GitLab-APIs und dem Prototyp-Stack. Webhooks ermöglichen externe Reaktionen auf GitLab-Ereignisse; Issues API, Notes API und Discussions API stellen Ticket- und Kommentarverlauf bereit (vgl. GitLab Docs, o. J.a, o. S.; GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.; GitLab Docs, o. J.e, o. S.). Dev-Assist muss daher Ereignisse entgegennehmen, relevante Ereignisse erkennen, Kontext abrufen und Ergebnisse nach GitLab zurückschreiben können. Gleichzeitig sind LLM-Ausgaben fehleranfällig. NIST beschreibt Halluzinationen beziehungsweise Confabulations als überzeugend oder sicher präsentierte, aber fehlerhafte oder falsche Inhalte (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Validierung, Rückfragen und menschliche Freigabe sind deshalb notwendige Bestandteile des Systementwurfs.

3.2 Beschreibung des bisherigen Prozesses

Der bisherige Prozess beginnt mit einem GitLab-Issue. Eine meldende Person beschreibt eine Anforderung, einen Fehler oder eine technische Aufgabe. Manche Tickets enthalten bereits Zielbeschreibung, Akzeptanzkriterien und Validierungshinweise; andere bestehen nur aus einer Idee, einem Screenshot, einer Fehlermeldung oder einem Gesprächsfragment. Damit ähnelt der Prozess den in der Forschung beschriebenen Bug-Tracking-Situationen, in denen meldende Personen häufig andere Informationen bereitstellen als Entwicklerinnen und Entwickler für die Bearbeitung benötigen (vgl. Bettenburg et al., 2008, S. 308 f.).

Fehlen Informationen, stellen Entwicklerinnen und Entwickler Rückfragen in den Kommentaren. Breu et al. zeigen, dass solche Rückfragen in Bug Reports unter anderem Reproduktionsschritte, Umgebung, Beispiele, Ausgaben und Screenshots betreffen können (vgl. Breu et al., 2010, S. 303). Übertragen auf GitLab-Issues bedeutet dies, dass die eigentliche Anforderung häufig nicht vollständig in der ursprünglichen Beschreibung steht, sondern schrittweise aus Titel, Beschreibung, Kommentaren, Antworten, Anhängen und Entscheidungen entsteht.

Dieser Ablauf erzeugt typische Probleme: Kontext ist verteilt, Wartezeiten entstehen durch manuelle Klärung, Rückfragen erfolgen unsystematisch, und nach der Klärung muss das Ticket oft erneut zusammengefasst und strukturiert werden. Ohne diesen Aufräumschritt bleibt das Ticket für Planung, Umsetzung und spätere Nachvollziehbarkeit schwer lesbar.

Dev-Assist soll den Prozess nicht ersetzen, sondern an dieser Stelle unterstützen. Das

System wird nur aktiv, wenn ein Issue oder Kommentar mit `@dev-assist` beginnt. Danach analysiert es den vorhandenen Kontext, stellt bei fehlenden Informationen Rückfragen oder erzeugt bei ausreichendem Kontext einen strukturierten Vorschlag. Erst nach `@dev-assist publish` wird dieser Vorschlag übernommen. Fachliche Verantwortung und Freigabe bleiben damit beim Menschen.

Der gewünschte Zielprozess umfasst fünf Schritte:

1. Eine Person erstellt oder aktualisiert ein GitLab-Issue und spricht Dev-Assist mit `@dev-assist` an.
2. Dev-Assist prüft das Ereignis, liest den verfügbaren Issue- und Kommentar-Kontext und startet die Analyse.
3. Bei unzureichendem Kontext stellt Dev-Assist gezielte Rückfragen im Kommentarverlauf.
4. Bei ausreichendem Kontext erzeugt Dev-Assist einen strukturierten Ticketvorschlag mit Titel, Ziel, Umfang, Anforderungen, Akzeptanzkriterien, offenen Fragen, Risiken und Validierungsschritten.
5. Nach `@dev-assist publish` werden Dev-Assist-bezogene Kommentare bereinigt und Titel sowie Beschreibung des Issues durch den strukturierten Vorschlag ersetzt.

Dev-Assist wird damit als Prozesswerkzeug verstanden. Es verbessert die Qualität des Ticketartefakts, ohne GitLab als Arbeitsort zu verlassen oder eine zusätzliche Benutzeroberfläche einzuführen.

3.3 Zielgruppen des Systems

Dev-Assist richtet sich an alle Rollen, die am Entstehen und Nutzen von GitLab-Tickets beteiligt sind. Ihre Bedarfe unterscheiden sich und führen zu unterschiedlichen Anforderungen an Rückfragen, Strukturierung, Prüfbarkeit und Betrieb.

Zielgruppe	Bedarf im Prozess	Konsequenz für Dev-Assist
Meldende Personen und fachliche Stakeholder	Niedrige Einstiegshürde beim Formulieren von Anforderungen oder Fehlern	Dev-Assist darf keine perfekte Erstbeschreibung voraussetzen und muss verständliche Rückfragen stellen.
Product Owner und Projektverantwortliche	Besser strukturierte Tickets, klare Abgrenzung und nachvollziehbare Akzeptanzkriterien	Der Vorschlag muss Ziel, Nutzen, Scope, Out-of-Scope und Definition of Done sichtbar machen.

Zielgruppe	Bedarf im Prozess	Konsequenz für Dev-Assist
Entwicklerinnen und Entwickler	Bearbeitbare Aufgaben mit ausreichend Kontext und prüfbaren Kriterien	Der erzeugte Tickettext muss Umsetzung, Validierung und offene Annahmen klar trennen.
Qualitätssicherung	Prüfbarkeit der Umsetzung und klare erwartete Ergebnisse	Akzeptanzkriterien und Validierungsschritte müssen explizit aufgeführt werden.
Administratorinnen und Betreiber	Sicherer und wartbarer Betrieb der Webhook-Anwendung	Konfiguration, Logging, Fehlerverhalten und Signaturprüfung müssen nachvollziehbar sein.

Für meldende Personen ist wichtig, dass Dev-Assist nicht mit technischen Detailfragen beginnt. Die projektinterne Ausrichtung sieht vor, dass der Assistent nach Anforderungen, Nutzerbedarf, Akzeptanzkriterien, Scope und Erfolgskriterien fragt, nicht nach konkreten Dateien oder Implementierungsdetails. Diese Abgrenzung ist fachlich sinnvoll, weil meldende Personen bestimmte für Entwickler hilfreiche Informationen oft nicht ohne Weiteres ermitteln oder verwertbar bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.).

Für Entwicklerinnen und Entwickler zählt die spätere Nutzbarkeit. Der Vorschlag muss beschreiben, was gebaut oder geändert werden soll, ohne unbelegte Implementierungsentscheidungen zu treffen. Technische Hinweise sind nur aufzunehmen, wenn sie aus dem Ticketkontext hervorgehen oder ausdrücklich als Randbedingung genannt wurden. Für Product Owner und Qualitätssicherung sind Ziel, erwartetes Verhalten und Akzeptanzkriterien zentral, da nur so Priorisierung, Umsetzung und Abnahme möglich sind. Diese Logik passt auch zu Chaparro et al., die erwartetes Verhalten und Reproduktionsschritte als wichtige Bestandteile von Bug Descriptions beschreiben (vgl. Chaparro et al., 2017, S. 396 f.).

Für Betreiberinnen und Betreiber muss Dev-Assist kontrollierbar bleiben. Das System verarbeitet externe Webhook-Anfragen und nutzt generative KI. Daher müssen Secrets konfigurierbar sein, Ereignisse protokolliert werden, Signaturprüfungen für produktive Umgebungen erzwingbar sein und Fehler ohne unkontrollierte Issue-Änderungen behandelt werden.

3.4 Fachliche Anforderungen an die Ticketqualität

Das zentrale fachliche Ziel ist ein entwicklungsfähiges Ticket. In dieser Arbeit gilt ein Ticket als entwicklungsfähig, wenn eine Entwicklerin oder ein Entwickler die Aufgabe verstehen, umsetzen und überprüfen kann, ohne wesentliche Informationen aus verstreuten Kommentaren rekonstruieren zu müssen.

Dafür muss der Titel den Gegenstand knapp beschreiben, das Ziel den Nutzen oder das zu lösende Problem benennen und der Umfang klar abgegrenzt sein. Scope und Out-of-

Scope verhindern, dass Tickets zu groß werden oder verschiedene Themen vermischen. Funktionale Anforderungen und Akzeptanzkriterien müssen explizit formuliert werden. Lucassen et al. betonen, dass User-Story-Qualität nicht nur an der Form, sondern auch an Bedeutung und Nutzbarkeit gemessen werden muss (vgl. Lucassen et al., 2016, S. 386 f.). Eine bloße sprachliche Umformulierung reicht daher nicht aus; Dev-Assist soll vorhandene Inhalte in prüfbare Aussagen überführen.

Offene Fragen, Risiken und Annahmen müssen sichtbar bleiben. Wenn Informationen fehlen, darf das System sie nicht durch scheinbar sichere Aussagen ersetzen. Das ist besonders wichtig, weil LLMs überzeugend wirkende, aber fehlerhafte Inhalte erzeugen können (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Unsichere Ableitungen müssen daher als Annahmen oder offene Punkte erkennbar sein.

Aus diesen Qualitätszielen ergibt sich folgende Mindeststruktur:

Bestandteil	Zweck
Zusammenfassung	Kurze Einordnung des Ticketinhalts und der Quelle des Vorschlags
Titel	Knappes, suchbares und handlungsorientiertes Issue-Thema
Ziel	Beschreibung des angestrebten Ergebnisses oder Nutzens
Scope	Inhalte, die ausdrücklich Teil des Tickets sind
Out-of-Scope	Inhalte, die nicht Teil des Tickets sind
User Stories	Rollenbezogene Beschreibung des erwarteten Nutzens, sofern sinnvoll ableitbar
Funktionale Anforderungen	Konkrete Verhaltensanforderungen an das System
Technische Hinweise	Nur explizit genannte technische Randbedingungen, keine erfundenen Implementierungsdetails
Umsetzungsschritte	Grobe, entwicklungsorientierte Arbeitsschritte ohne unnötige Tiefendetails
Definition of Done	Bedingungen, unter denen das Ticket als abgeschlossen gelten kann
Akzeptanzkriterien	Prüfpunkte für Abnahme und Qualitätssicherung
Offene Fragen	Fehlende Informationen, die nicht sicher abgeleitet werden können
Risiken und Annahmen	Unsicherheiten, Abhängigkeiten und mögliche Fehlinterpretationen
Validierungsschritte	Vorschläge, wie Umsetzung oder Fehlerbehebung geprüft werden kann

Die Struktur ist bewusst umfangreicher als eine minimale User Story. Sie soll Tickets nicht künstlich aufblähen, sondern sichtbar machen, welche Informationen bereits vorhanden sind und welche fehlen. Wenn ein Abschnitt nicht sinnvoll gefüllt werden kann, muss Dev-Assist die Lücke benennen, statt leere Qualität zu simulieren.

3.5 Funktionale Anforderungen

Die funktionalen Anforderungen beschreiben, was Dev-Assist leisten muss. Die Nummerierung FA-1 bis FA-11 dient der Nachverfolgbarkeit in Konzeption, Implementierung und Evaluation.

FA-1: Explizite Aktivierung durch @dev-assist.

Dev-Assist darf nur reagieren, wenn der erste inhaltliche Text eines Issues oder Kommentars mit @dev-assist beginnt. Führende Leerzeichen und einfache Markdown-Formatierungen wie Überschriften, Listenmarker oder Fettschreibung sollen toleriert werden. Dadurch analysiert das System keine beliebige GitLab-Kommunikation und reagiert nicht auf zufällige Erwähnungen im Fließtext.

FA-2: Verarbeitung relevanter GitLab-Ereignisse.

Das System muss Work-item-/Issue-Ereignisse und Kommentarereignisse aus GitLab-Webhooks entgegennehmen können. Relevant ist ein Ereignis, wenn Issue-Beschreibung oder Kommentar Dev-Assist am Anfang ansprechen. GitLab-Webhooks liefern Ereignisse als HTTP-Anfragen an externe Anwendungen; GitLab dokumentiert dafür unter anderem Work-item-/Issue-bezogene Ereignisse und Comment Events (vgl. GitLab Docs, o. J.a, o. S.; GitLab Docs, o. J.b, Abschnitte "Work item events" und "Comment events"). Dev-Assist muss diese Payloads in ein internes, einheitliches Format überführen.

FA-3: Unterscheidung zwischen Analyse- und Publish-Kommando.

Nach der Mention muss Dev-Assist zwischen Analyseanforderung und Publish-Kommando unterscheiden. Beginnt der restliche Kommentar mit **publish**, wird keine neue Analyse gestartet, sondern der zuletzt erzeugte strukturierte Kontext übernommen. Alle anderen Aktivierungen lösen die Analyse aus.

FA-4: Abruf des relevanten Ticketkontexts.

Die Webhook-Payload reicht für die Analyse nicht immer aus. Dev-Assist muss deshalb, soweit möglich, das aktuelle Issue und die zugehörigen Kommentare über GitLab abrufen. Die Issues API stellt Issue-Daten bereit, die Notes API den Kommentarverlauf (vgl. GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.). Der Kommentarverlauf ist wichtig, weil Informationen häufig nachträglich ergänzt werden.

FA-5: Analyse von Beschreibung und Kommentaren.

Dev-Assist muss Titel, Beschreibung, Kommentare und gegebenenfalls zusätzlichen Rohtext gemeinsam auswerten. Das System soll vorhandene und fehlende Informationen erkennen und zwischen Anforderungen, Akzeptanzkriterien, technischen Randbedingungen, offenen Fragen und Annahmen unterscheiden. Diese Anforderung folgt aus dem Charakter von Tickets als verteilte Arbeitsartefakte.

FA-6: Rückfragen bei fehlenden Informationen.

Wenn wesentliche Informationen fehlen, muss Dev-Assist gezielte Rückfragen formulieren. Sie sollen fachliche Ziele, Nutzerbedarf, Scope, Akzeptanzkriterien, Randfälle und Erfolgskriterien betreffen, nicht interne Implementierungsdetails. Die Fragen müssen

spezifisch genug sein, um den Ticketkontext tatsächlich zu verbessern. Diese Anforderung knüpft an die Forschung zu fehlenden Informationen in Bug Reports an (vgl. Breu et al., 2010, S. 303; Chaparro et al., 2017, S. 396 f.).

FA-7: Erzeugung strukturierter Ticketvorschläge.

Wenn Hauptzweck, zentrale Anforderungen und Umfang ausreichend klar sind, muss Dev-Assist einen strukturierten Vorschlag nach Abschnitt 3.4 erzeugen. Der Vorschlag muss als neue Issue-Beschreibung nutzbar sein und darf ungesicherte Informationen nicht als Tatsachen darstellen. Offene Punkte werden als offene Fragen oder Annahmen markiert. Dieser Modus knüpft an Qualitätshilfen wie BEE an, das Bug Reports anhand zentraler Bestandteile wie beobachtetem Verhalten, erwartetem Verhalten und Reproduktionsschritten strukturiert; die breitere Dev-Assist-Struktur ist eine eigene Anforderung dieser Arbeit (vgl. Song/Chaparro, 2020, S. 1551 f.).

FA-8: Maschinenlesbares Analyseformat.

Die KI-Ausgabe muss in einem definierten JSON-Format erfolgen. Das Format muss Felder für Zusammenfassung, Quellenbasis, Umsetzungsticket, Akzeptanzkriterien, technische Hinweise, offene Fragen, Risiken und Validierungsschritte enthalten. Das System muss die Antwort parsen und strukturell prüfen, bevor daraus ein Kommentar oder Kontextdokument entsteht.

FA-9: Veröffentlichung als GitLab-Kommentar.

Das Analyseergebnis muss in GitLab sichtbar werden. Bei fehlendem Kontext postet Dev-Assist Rückfragen, bei ausreichendem Kontext einen vollständigen strukturierten Vorschlag. Der Kommentar muss deutlich machen, dass die Übernahme erst durch `@dev-assist publish` erfolgt.

FA-10: Persistenz des strukturierten Kontexts.

Nach der Analyse muss Dev-Assist den strukturierten Kontext lokal in einem reproduzierbaren Pfad ablegen, etwa unter `.dev-assist/issues/<projectId>/<issueId>/context.md`. Verfügbare Titel oder Metadaten sollen zusätzlich maschinenlesbar gespeichert werden. Diese Kontextdateien bilden die Grundlage für den Publish-Schritt und mögliche spätere Entwicklungsprozesse.

FA-11: Kontrollierte Übernahme durch Publish.

Das Publish-Kommando muss den zuvor erzeugten Kontext lesen, Dev-Assist-bezogene Kommentare identifizieren, diese bereinigen und anschließend Titel sowie Beschreibung des GitLab-Issues aktualisieren. Publish darf nur nach erfolgreicher Analyse erfolgen. Die Anforderung begrenzt die Systemautonomie und passt zu OWASPs Risiko "Excessive Agency", bei dem LLM-basierte Systeme zu weitreichende Berechtigungen oder Handlungsspielräume erhalten (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency").

3.6 Nicht-funktionale Anforderungen

Die nicht-funktionalen Anforderungen beschreiben Qualitätsmerkmale für Betrieb, Wartung und Vertrauen. Da Dev-Assist Webhooks, GitLab-API-Zugriffe und generative KI verbindet, sind diese Anforderungen besonders relevant.

NFA-1: Wartbarkeit durch klare Modulgrenzen.

Webhook-Verarbeitung, GitLab-Anbindung, Mention-Erkennung, KI-Analyse, Formatierung, Kontextpersistenz und Publish-Logik müssen getrennt nachvollziehbar sein. Klare Modulgrenzen erleichtern Änderungen, etwa den Austausch des KI-Providers oder die Erweiterung der GitLab-Operationen.

NFA-2: Testbarkeit ohne echte GitLab- oder KI-Abhängigkeit.

Dev-Assist muss lokal testbar sein, ohne reale GitLab-Projekte oder kostenpflichtige KI-Aufrufe zu benötigen. Dafür braucht das System einen Mock-Modus für KI-Analysen und isolierbare Funktionen für Parser, Mention-Erkennung, Formatierung, Cleanup und Webhook-Authentifizierung. Wegen der Nichtdeterministik von LLM-Ausgaben müssen die deterministischen Systemteile besonders gut abgesichert werden.

NFA-3: Robustheit gegenüber unvollständigem Kontext und externen Fehlern.

Das System muss mit fehlgeschlagenen GitLab-API-Aufrufen, unvollständigen Payloads, fehlenden Kommentaren oder ungültigen KI-Antworten umgehen können. Fehler sollen protokolliert und, soweit sinnvoll, mit Fallback-Daten behandelt werden. Wenn eine Analyse vollständig fehlschlägt, darf kein fehlerhafter Publish-Kontext entstehen.

NFA-4: Nachvollziehbarkeit durch Logging.

Relevante Verarbeitungsschritte müssen über Konsolenausgaben nachvollziehbar sein: eingehende Requests, erkannte und ignorierte Ereignisse, Analysebeginn und -ende, Fehler, gepostete Kommentare, Kontextdateien, gelöschte Kommentare und aktualisierte Issues. Da keine separaten Logdateien vorgesehen sind, ist die Konsole die zentrale Beobachtungsquelle. Logging muss konkret sein, darf aber keine Secrets ausgeben.

NFA-5: Begrenzung von KI-Risiken.

LLM-Ausgaben dürfen nicht ungeprüft zu produktiven Änderungen werden. Halluzinationen und Confabulations sind für generative KI ein bekanntes Risiko (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Daher müssen KI-Antworten strukturell validiert, offene Fragen erhalten und Vorschläge erst nach expliziter Freigabe übernommen werden. Der Prompt muss zudem klarstellen, dass Ticketinhalte Daten und keine Systemanweisungen sind, da OWASP Prompt Injection als Risiko beschreibt, bei dem Eingaben das Verhalten eines LLM unbeabsichtigt verändern können (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection").

NFA-6: Sicherheit der Webhook-Schnittstelle.

Dev-Assist muss eine Signatur- oder Tokenprüfung für Webhook-Anfragen unterstützen und produktiv erzwingbar machen. Lokale Entwicklung soll auch ohne eingerichtetes GitLab-Secret möglich bleiben. Für Tests kann ein toleranter Modus sinnvoll sein; pro-

duktiv muss die Integrität der Anfrage abgesichert werden.

NFA-7: Vermeidung von Bot-Endlosschleifen.

Dev-Assist muss verhindern, dass eigene Kommentare neue Analysen auslösen. Dazu verarbeitet das System nur führende `@dev-assist`-Mentions, generierte Vorschläge beginnen nicht mit dieser Mention, und doppelte Ereignisse müssen innerhalb eines kurzen Zeitfensters ignoriert werden können.

NFA-8: Schnelle Webhook-Antwort und asynchrone Verarbeitung.

GitLab-Webhooks sollten zeitnah mit einem erfolgreichen HTTP-Status beantwortet werden, während KI-Analysen länger dauern können (vgl. GitLab Docs, o. J.a, o. S.). Dev-Assist muss Webhook-Anfragen daher schnell mit einem erfolgreichen `2xx`-Status, etwa `200` oder `201`, beantworten und die Verarbeitung asynchron oder nachgelagert ausführen. So werden Timeouts und erneut gesendete Ereignisse vermieden.

NFA-9: Konfigurierbarkeit des Betriebs.

Port, Log-Level, Mention-String, GitLab-Basis-URL, GitLab-Authentifizierung, KI-Provider, Modell, Timeout, Kontextverzeichnis und Signaturpflicht müssen über Umgebungsvariablen steuerbar sein. Secrets gehören in die `.env` und nicht in den Code. Diese Anforderung unterstützt Mock-Betrieb, `glab`, PAT-Fallback und spätere Modellerweiterungen.

NFA-10: Prozessintegrierte Bedienbarkeit.

Dev-Assist soll keine zusätzliche grafische Oberfläche benötigen. Die Bedienung erfolgt über GitLab-Issues und Kommentare, sodass der Arbeitsfluss vertraut bleibt. Dashboards, Metriken oder separate Administrationsoberflächen sind mögliche Erweiterungen, aber nicht Teil des Kernumfangs.

3.7 Abgrenzung der Anforderungen

Dev-Assist soll nicht zu einem allgemeinen Entwicklungsagenten ausgeweitet werden. Die Arbeit konzentriert sich auf die Verbesserung von Software-Tickets.

Erstens führt Dev-Assist keine autonome Implementierung durch. Das System erzeugt strukturierte Tickets, Rückfragen und Vorschläge, schreibt aber keinen Produktivcode, erstellt keine Merge Requests und entscheidet nicht eigenständig über technische Umsetzung. Zweitens ersetzt Dev-Assist keine fachliche Priorisierung. Ob ein Ticket wichtig ist, wann es umgesetzt wird und welche Produktentscheidung dahintersteht, bleibt Aufgabe der verantwortlichen Personen. Drittens ersetzt der Assistent keine menschliche Prüfung: Der Publish-Schritt ist bewusst explizit.

Viertens bewertet Dev-Assist nicht die gesamte Qualität eines Projekts. Es geht nicht um Codequalität, Architekturqualität oder Produktmetriken, sondern um Struktur und Nutzbarkeit einzelner Tickets. Fünftens garantiert das System keine vollständige semantische Wahrheit. LLM-Ausgaben bleiben Vorschläge; Unsicherheiten müssen sichtbar bleiben, und die Verantwortung verbleibt beim Team.

Sechstens ist eine umfassende Bild- oder Screenshot-Interpretation nicht Kern der An-

forderungsanalyse. Screenshots können Hinweise liefern und spätere Erweiterungen ermöglichen, für den Kernprozess steht jedoch die Strukturierung vorhandener textueller Informationen aus Beschreibung und Kommentaren im Vordergrund.

3.8 Priorisierung der Anforderungen

Für den Prototyp werden die Anforderungen nach ihrer Bedeutung für den Kernworkflow priorisiert. Muss-Anforderungen sind notwendig, Soll-Anforderungen verbessern Qualität, Betrieb oder Erweiterbarkeit, und Kann-Anforderungen sind sinnvoll, aber nicht Voraussetzung für den Kernnachweis.

Priorität	Anforderungen	Begründung
Muss	FA-1 bis FA-11, NFA-1 bis NFA-10	Ohne expliziten Trigger, Analyse, Rückfragen, Vorschlag, Persistenz, Publish, Validierung, Logging, Konfiguration, Sicherheitsgrenzen und Schleifenvermeidung ist der Zielprozess nicht belastbar erfüllt.
Soll	Erweiterte Behandlung von Bild- und Screenshot-Kontext, zusätzliche GitLab-Metadaten, ausführlichere Fehlerdiagnosen	Diese Punkte verbessern den praktischen Nutzen, sind aber nicht erforderlich, um den Kernworkflow nachzuweisen.
Kann	Dashboard, quantitative Qualitätsmetriken, Rollen- und Rechtekonzept über GitLab hinaus	Diese Erweiterungen sind fachlich interessant, aber für den prototypischen Nachweis nicht erforderlich.

Die Priorisierung spiegelt die Zielsetzung wider: Dev-Assist soll zeigen, dass ein KI-gestützter GitLab-Assistent Ticketinformationen in einem kontrollierten Workflow verbessern kann. Entscheidend sind daher Trigger-Erkennung, Kontextanalyse, strukturierte Ausgabe und menschlich kontrollierte Übernahme, nicht zusätzliche Oberflächen oder umfassende Produktivbetriebsfunktionen.

3.9 Zwischenfazit

Die Anforderungsanalyse zeigt, dass Dev-Assist zwei Ebenen verbinden muss. Fachlich soll das System unvollständige oder unstrukturierte GitLab-Issues in besser nutzbare Entwicklungsartefakte überführen. Dafür muss es fehlende Informationen erkennen,

Rückfragen stellen, strukturierte Vorschläge erzeugen und Annahmen sichtbar machen. Technisch muss Dev-Assist GitLab-Ereignisse verarbeiten, Ticketkontext abrufen, KI-Ausgaben validieren, Ergebnisse zurückschreiben und den Publish-Schritt kontrolliert ausführen.

Besonders wichtig ist die Begrenzung der Systemautonomie. Die Forschung zu Ticketqualität begründet den Nutzen strukturierter Anforderungen, Akzeptanzkriterien und Rückfragen. Die Risiken generativer KI begründen, warum Dev-Assist keine Änderungen ohne Freigabe vornehmen darf. Daraus ergibt sich der zentrale Entwurfsansatz: Dev-Assist ist ein Assistenzsystem zur Ticketaufbereitung, kein autonomer Entscheider.

Die Anforderungen dieses Kapitels bilden die Grundlage für die Systemarchitektur im nächsten Kapitel. Dort wird beschrieben, wie Webhook-Handler, GitLab-API-Anbindung, Agenten-Analyse, Validierung, Kontextdateien, Kommentarverarbeitung und Publish-Funktion zusammenspielen.

4 Konzeption der Systemarchitektur

Nachdem Kapitel 3 die fachlichen und nicht-funktionalen Anforderungen an Dev-Assist abgeleitet hat, beschreibt dieses Kapitel die daraus folgende Systemarchitektur. Im Mittelpunkt steht die konzeptionelle Struktur des Prototyps: notwendige Komponenten, Datenflüsse, Begrenzung und Validierung von KI-Ausgaben sowie die Absicherung gegen Änderungen an GitLab-Tickets ohne menschliche Freigabe.

Dev-Assist verbindet GitLab als ereignisbasierte Entwicklungsplattform mit Issues, Kommentaren, Webhooks und APIs mit einem Large-Language-Model-basierten Analyseprozess, der natürlichsprachliche Ticketinformationen in strukturierte Vorschläge überführt. Die zentrale Aufgabe besteht darin, diese Verbindung kontrolliert, nachvollziehbar und testbar zu gestalten. Das System ist daher nicht als autonomer Agent konzipiert, sondern als begrenzter Webhook-Dienst: Es prüft Ereignisse, sammelt Kontext, stößt eine Analyse an, persistiert Ergebnisse und übernimmt Änderungen erst nach einem expliziten Publish-Kommando.

4.1 Architekturziele und Entwurfsprinzipien

Die Architektur orientiert sich an den Anforderungen aus Kapitel 3. Dev-Assist soll ohne zusätzliche Benutzeroberfläche in einem bestehenden GitLab-Prozess nutzbar sein; die Interaktion findet vollständig über Issues und Kommentare statt. Technisch führt dies zu einer API-orientierten Architektur: Ein Express-Server nimmt HTTP-Anfragen entgegen, verarbeitet GitLab-Webhook-Payloads und ruft nachgelagerte Dienste für GitLab-Kommunikation, KI-Analyse, Kontextpersistenz und Publish-Verarbeitung auf.

Ein erstes Architekturziel ist die klare Trennung der Verantwortlichkeiten. Softwarearchitektur beschreibt nach Bass et al. zentrale Elemente, Beziehungen und daraus entstehende Systemeigenschaften (vgl. Bass et al., 2021, Kap. 1). Für Dev-Assist bedeutet dies, dass Webhook-Routing, Authentifizierung, Mention-Erkennung, GitLab-API-Zugriff, KI-Prompting, Antwortvalidierung, Markdown-Rendering, Kontextdateien und Publish-Logik getrennt umgesetzt werden. Diese Aufteilung unterstützt Wartbarkeit, Tests und Änderbarkeit.

Ein zweites Ziel ist die ereignisbasierte Integration. GitLab-Webhooks informieren externe Anwendungen bei relevanten Ereignissen, etwa Issue-Updates oder neuen Kommentaren (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook events"). Dev-Assist muss daher nicht zyklisch nach neuen Tickets suchen, sondern reagiert auf konkrete Ereignisse. Dieses Muster entspricht lose gekoppelten Integrationsarchitekturen, in denen Nachrichten nachgelagerte Verarbeitungsschritte auslösen (vgl. Hohpe/Woolf, 2003, Abschnitte

"Messaging" und "Message Channel").

Ein drittes Ziel ist kontrollierte Autonomie. Das Modell darf Ticketinformationen analysieren und Vorschläge erzeugen, aber keine produktiven Ticketänderungen selbst durchführen. Diese Grenze ist notwendig, weil generative KI fehlerhafte oder unbelegte Inhalte erzeugen kann. NIST beschreibt dieses Risiko als "Confabulation"; "hallucinations" und "fabrications" werden dort als umgangssprachliche Begriffe für überzeugend präsentierte, aber falsche oder irreführende Inhalte eingeordnet (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). OWASP warnt zudem vor "Excessive Agency", wenn LLM-basierte Systeme zu weitreichende Berechtigungen erhalten (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency"). Dev-Assist nutzt deshalb einen zweistufigen Prozess: Zunächst entsteht ein Vorschlag als Kommentar und Kontextdatei; erst `@dev-assist publish` übernimmt ihn in Titel und Beschreibung des Issues.

Ein viertes Ziel ist Nachvollziehbarkeit. Da der Prototyp keine gesonderte grafische Oberfläche und keine Datei-Logs vorsieht, werden relevante Verarbeitungsschritte über Konsolenausgaben sichtbar gemacht. Dazu gehören eingehende Requests, ignorierte oder verarbeitete Webhooks, Signaturentscheidungen, GitLab-Zugriffe, KI-Aufrufe, Kontextdateien, Kommentare und Issue-Updates.

4.2 Gesamtarchitektur des Webhook-Systems

Dev-Assist ist als reine Serveranwendung ohne Frontend konzipiert. Der Express-Server initialisiert Konfiguration und Log-Level, prüft bei Nutzung des Opencode-Pfads die CLI-Verfügbarkeit und startet die HTTP-Anwendung. Optional kann ein Cloudflare-Tunnel den lokalen Webhook-Endpunkt für GitLab erreichbar machen. Die Anwendung stellt drei zentrale Routen bereit: einen Health-Endpunkt, den GitLab-Webhook-Endpunkt sowie manuelle Issue-Endpunkte für Process und Publish.

Die Gesamtarchitektur lässt sich in sechs Schichten gliedern:

Schicht	Hauptaufgabe	Zentrale Projektmodule
HTTP- und Routing-Schicht	Requests, JSON-Parsing, Rohkörper-Erfassung, Logging, Antwort an GitLab	<code>src/app.ts</code> , <code>src/server.ts</code> , <code>src/routes/gitlabWebhooks.ts</code> , <code>src/routes/issues.ts</code> , <code>src/routes/health.ts</code>
Webhook-Prüfung	Signatur-/Tokenprüfung, Payload-Parsing, Mention- und Kommandoerkennung, Deduplication	<code>src/services/gitlab/auth.ts</code> , <code>src/services/gitlab/parser.ts</code> , <code>src/services/gitlab/mention.ts</code> , <code>src/services/gitlab/commands.ts</code>
GitLab-Anbindung	Lesen von Issues und Kommentaren, Schreiben und Löschen von Notizen, Aktualisieren von Issues	<code>src/services/gitlab/client.ts</code> , <code>src/services/gitlab/cleanup.ts</code>

Schicht	Hauptaufgabe	Zentrale Projektmodule
Analyse- und Agentenschicht	Prompt-Aufbau, Opencode- oder Mock-Analyse, JSON-Parsing, Schema-Prüfung, Rückfragen	<code>src/services/ai/service.ts</code> , <code>src/services/ai/instructions.ts</code> , <code>src/services/ai/schema.ts</code> , <code>src/services/ai/clarifications.ts</code> , <code>opencode.json</code>
Ausgabe- und Persistenzschicht	GitLab-Markdown, Kontextdateien	<code>src/services/ai/formatter.ts</code> , <code>src/services/context/writer.ts</code> , <code>src/services/context/reader.ts</code>
Prozesssteuerung	Orchestrierung von Analyse und Publish	<code>src/services/processing/processor.ts</code> , <code>src/services/processing/publisher.ts</code>

Die HTTP-Schicht kennt nur den Transport und delegiert fachliche Entscheidungen. Die GitLab-Anbindung kapselt Zugriffe über `glab` oder Personal Access Token, die Analyse-Schicht kapselt Mock- und Opencode-Pfad, und die Publish-Schicht liest den persistierten Kontext, ohne Modelllogik zu kennen. Die Abhängigkeiten verlaufen damit kontrolliert: Routen rufen die Prozessschicht auf, diese nutzt GitLab-, AI-, Repository-Summary- und Kontextdienste. Die KI-Schicht schreibt nicht selbst nach GitLab, sondern liefert nur eine strukturierte Antwort.

4.3 Ereignisbasierter Ablauf über GitLab-Webhooks

Der primäre Ablauf beginnt mit einem GitLab-Ereignis. Relevante Ereignisse sind Issue- beziehungsweise Work-Item-Ereignisse und Kommentarereignisse. GitLab dokumentiert Work-Item-Ereignisse für erstellte, bearbeitete, geschlossene oder wieder geöffnete Work Items; bei Issues wird `object_kind` als `issue` übermittelt (vgl. GitLab Docs, o. J.b, Abschnitt "Work item events"). Kommentare werden über Comment Events beziehungsweise Notes abgebildet.

Der Webhook-Endpunkt `/webhooks/gitlab/issues` ist so ausgelegt, dass GitLab schnell eine erfolgreiche Antwort erhält. Die Route prüft die Anfrage, parst das Payload, entscheidet über Relevanz und antwortet im Prototyp mit `202 Accepted`, während die eigentliche Verarbeitung asynchron weiterläuft. GitLab empfiehlt für Webhook-Receiver eine schnelle Antwort, typischerweise 200 oder 201, sowie die Auslagerung längerer Verarbeitungsschritte; zugleich gelten in der Webhook-Historie Statuscodes von 200 bis 299 als erfolgreich (vgl. GitLab Docs, o. J.a, Abschnitte "Webhook receiver requirements" und "View webhook request history"). `202 Accepted` ist damit ein plausibler `2xx`-Status, aber eine projektspezifische Implementierungsentscheidung.

Ein Analyseereignis durchläuft folgende Schritte:

1. GitLab sendet ein Issue- oder Note-Event an den Webhook-Endpunkt.
2. Express liest das JSON-Payload und speichert den Rohkörper für die Signaturprüfung.
3. Die Webhook-Authentifizierung prüft Signatur oder zulässigen Entwicklungsmodus.

dus.

4. Der Parser überführt die Payload in ein internes `ParsedWebhook`-Format mit Ereignistyp, Projekt-ID, Issue-IID, Titel, Beschreibung, Kommentartext und Kommando.
5. Mention-Gate und Command-Parser prüfen, ob Dev-Assist explizit angesprochen wurde und ob Analyse oder Publish ausgelöst werden soll.
6. Eine In-Memory-Deduplication verhindert mehrfache Verarbeitung identischer oder schnell wiederholter Ereignisse.
7. Je nach Kommando startet `processFromWebhook` oder `publishIssue`.
8. Die Route gibt unmittelbar einen erfolgreichen 2xx-Status zurück; im Prototyp ist dies 202 `Accepted`.

Damit wird zwischen Transportbestätigung und fachlichem Ergebnis unterschieden. Die 2xx-Antwort bedeutet nur, dass Dev-Assist das Ereignis angenommen hat. Ob Analyse, Kommentar oder GitLab-Zugriff erfolgreich waren, wird anschließend über Logging und GitLab-Kommentare sichtbar. Für lokale Tests und manuelle Nachverarbeitung existieren zusätzlich `/api/issues/:projectId/:issueId/process` und `/api/issues/:projectId/:issueId/publish`, beide nutzen dieselbe Prozess- und Publish-Logik wie der Webhook-Pfad.

4.4 Zentrale Komponenten und Verantwortlichkeiten

Der Express-Server bildet den technischen Einstiegspunkt. Er initialisiert die Anwendung, liest Konfiguration und loggt eingehende Requests. Die JSON-Middleware erfasst zusätzlich den Rohkörper der Anfrage, weil die GitLab-Signing-Token-Prüfung den unveränderten Body als Teil der signierten Nachricht benötigt. Die Verifikation darf daher nicht nur auf geparstem JSON beruhen, sondern muss ursprüngliche Body-Zeichenkette und Webhook-Header berücksichtigen.

Der GitLab-Webhook-Router ist die erste fachliche Entscheidungsstelle. Er ruft die Signaturprüfung auf, parst die Payload, wendet Deduplication an, ignoriert irrelevante oder selbst erzeugte Ereignisse und startet den passenden Hintergrundprozess. KI-Logik und Prompt-Erstellung liegen bewusst außerhalb des Routers.

Parser, Mention-Gate und Command-Parser bilden die Aktivierungsschicht. Der Parser vereinheitlicht Issue- und Note-Events: Beim Issue-Event stehen Titel und Beschreibung in `object_attributes`, beim Note-Event der Kommentar in `object_attributes.note` und die zugehörige Issue-Information im `issue`-Objekt. Das Mention-Gate stellt sicher, dass Dev-Assist nur auf explizite Ansprache reagiert; der Command-Parser unterscheidet Analyse und `publish`.

Der GitLab-Client kapselt die Kommunikation mit GitLab. Primär wird `glab` genutzt, alternativ ein Token-basierter API-Zugriff. Die Issues API erlaubt Lesen und Aktualisieren von Issues, einschließlich Titel und Beschreibung (vgl. GitLab Docs, o. J.c, o. S.). Die Notes API erlaubt Abrufen und Erstellen von Issue-Kommentaren (vgl. GitLab

Docs, o. J.d, o. S.). Die Discussions API bildet die Grundlage für Thread-Operationen, auch wenn der Prototyp überwiegend mit Notes arbeitet (vgl. GitLab Docs, o. J.e, o. S.).

Der Processor orchestriert den Analysepfad. Er lädt nach Möglichkeit das aktuelle Issue, Kommentare und eine Repository-Zusammenfassung aus GitLab. Fehlen diese Daten, nutzt er das Webhook-Payload als Fallback. Die Repository-Zusammenfassung kann zusätzlichen technischen Kontext liefern, bleibt aber unterstützend: Fachliche Anforderungen müssen weiterhin aus dem Ticketkontext stammen.

Die AI-Service-Komponente erstellt den Prompt und ruft je nach Konfiguration Mock-Modus oder Opencode auf. Opencode wird über den Agenten `dev-assist-analyzer` eingebunden; die Opencode-Dokumentation beschreibt Agents als spezialisierte KI-Assistenten für Aufgaben und Workflows (vgl. OpenCode Docs, o. J.a, Abschnitt "Agents"). Die detaillierten Regeln, das erwartete JSON-Format und Füllregeln liegen in `src/services/ai/instructions.ts`, sodass Promptregeln und Laufzeitverarbeitung synchron bleiben.

Die Schema-Komponente definiert die erwartete Antwortstruktur mit Feldern wie `summary`, `sourceBasis`, `implementationTicket`, `acceptanceCriteria`, `technicalNotes`, `openQuestions`, `risks` und `validationSteps`. Erst diese maschinenlesbare Struktur ermöglicht deterministische Weiterverarbeitung. Der Formatter rendert gültige Analysen als GitLab-Markdown, entweder als Rückfragekommentar oder als vollständigen strukturierten Vorschlag. Context Writer und Reader speichern den Vorschlag unter `.dev-assist/issues/<project>` und optionale Metadaten in `context.json`. Der Publisher liest diese Dateien, bereinigt Dev-Assist-bezogene Kommentare und aktualisiert anschließend Titel und Beschreibung des Issues.

4.5 Datenfluss vom GitLab-Issue bis zum aktualisierten Ticket

Der Datenfluss gliedert sich in Analyse und Publish. Diese Trennung ist der wichtigste Schutzmechanismus gegen ungeprüfte Änderungen.

In der Analysephase entsteht ein internes Kontextobjekt aus Projektinformationen, Issue-Daten, Kommentaren, auslösendem Rohtext und optionaler Repository-Zusammenfassung. Da das Webhook-Payload nicht immer alle aktuellen Daten enthält, versucht Dev-Assist, Issue und Kommentare über die GitLab-API nachzuladen. Die Notes API unterstützt das Abrufen von Issue-Komentaren einschließlich Sortierung (vgl. GitLab Docs, o. J.d, o. S.). Der Prompt kombiniert feste Analyseanweisungen, Titel und Beschreibung, relevante Kommentare einschließlich beantworteter Rückfragen sowie gegebenenfalls Repository-Kontext. `clarifications.ts` verhindert, dass bereits beantwortete Dev-Assist-Fragen erneut gestellt werden.

Nach der Modellantwort folgt die strukturelle Prüfung. Der Opencode-Output wird extrahiert, als JSON geparkt und gegen Pflichtfelder geprüft. Nur schema-konforme Antworten werden als `RequirementAnalysis` weiterverarbeitet. Bei ungültigem Output wird kein verwertbarer Publish-Kontext erzeugt; stattdessen versucht das System,

einen Fehlerkommentar in GitLab zu posten.

Ist die Analyse gültig, entscheidet der Processor anhand offener Fragen, ob ein Rückfragekommentar oder ein vollständiger Vorschlag gepostet wird. In beiden Fällen wird der strukturierte Kontext lokal persistiert, sodass der spätere Publish-Schritt denselben Stand verwendet, der im Kommentar sichtbar war.

Die Publish-Phase beginnt mit einem neuen GitLab-Kommentar oder einem manuellen API-Aufruf. Der Command-Parser erkennt `publish`, der Publisher liest `context.md` und `context.json`, bereitet die finale Issue-Beschreibung auf, extrahiert den vorgeschlagenen Titel und bereinigt Dev-Assist-bezogene Notizen. Anschließend aktualisiert er das Issue über die GitLab Issues API, die Titel, Beschreibung und weitere Felder ändern kann (vgl. GitLab Docs, o. J.c, Abschnitt "Update an issue"). Damit schreibt nicht das Modell produktiv nach GitLab, sondern die Anwendung nach expliziter menschlicher Freigabe.

4.6 Prompt-, Agenten- und Antwortverarbeitung

Dev-Assist verwendet kein freies Chatformat, sondern ein streng vorgegebenes JSON-Schema. Dieses Schema bildet die spätere Ticketstruktur ab und zwingt die Analyse in Felder, die geprüft und gerendert werden können.

Die Anweisungen fokussieren Ziel, Nutzerwert, Anforderungen, Scope, Akzeptanzkriterien und Definition of Done. Der Assistent soll keine Fragen zu aktuellen Dateien, Komponenten, Bibliotheken oder Implementierungsdetails stellen, weil meldende Personen solche Informationen oft nicht zuverlässig liefern können. Vorhandener Repository-Kontext darf für technische Hinweise genutzt werden; fehlt er, darf das Modell keine Architekturdetails erfinden.

Der Prompt ist damit eine Kontrollfläche, reicht allein aber nicht aus. Prompt Injection kann unkontrollierte Eingaben so gestalten, dass sie das Verhalten eines LLM beeinflussen (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection"). Dev-Assist behandelt Ticketbeschreibungen und Kommentare deshalb als Daten, nicht als Systemanweisungen. Die festen Analyseanweisungen kommen aus der Anwendung, und die Modellantwort wird erst nach JSON-Parsing und Schema-Prüfung akzeptiert.

Opencode dient als austauschbare Agentenschnittstelle. Die Anwendung startet den Agenten über die CLI, übergibt den Prompt und liest die Antwort aus dem JSON-Stream oder über einen Export-Fallback. Das lokale `opencode.json` definiert `dev-assist-analyzer` für Issue-Analysen und `repo-summary` für Repository-Zusammenfassungen. Beide Agenten erhalten keine direkten GitLab-Berechtigungen; GitLab-Zugriffe bleiben bei der Anwendung.

Der Mock-Modus ermöglicht lokale Entwicklung und Tests ohne echte Modellaufrufe. Dadurch können Parser, Renderer, Kontextdateien, Publish-Logik und Tests unabhängig von nicht-deterministischem oder externem Modellverhalten geprüft werden.

4.7 Kontextpersistenz, Publish-Funktion und Kommentarbereinigung

Die Kontextpersistenz bildet den Übergabepunkt zwischen Analyse und Publish, erzeugt ein nachvollziehbares Artefakt außerhalb von GitLab und verhindert, dass Publish erneut eine KI-Analyse ausführen muss. Übernommen wird der zuvor erzeugte und gespeicherte Vorschlag.

`context.md` enthält den strukturierten Vorschlag als Markdown, unter anderem Zusammenfassung, Implementation Ticket, Scope, User Stories, funktionale Anforderungen, technische Hinweise, Definition of Done, Akzeptanzkriterien, offene Fragen, Risiken und Validierungsschritte. `context.json` enthält optionale Metadaten wie den vorgeschlagenen Titel. Markdown bleibt für Menschen lesbar, JSON für gezielte maschinelle Verarbeitung geeignet.

Der Publish-Schritt liest diese Dateien, entfernt Abschnitte, die nicht in die finale Issue-Beschreibung gehören, übernimmt bevorzugt den Titel aus den Metadaten und aktualisiert anschließend das GitLab-Issue. Vorher bereinigt die Cleanup-Komponente Dev-Assist-bezogene Kommentare. Sie löscht keine Systemnotes und keine fachlich unabhängigen Nutzerkommentare. Löschbar sind Dev-Assist-generierte Kommentare sowie Kommentare, die eindeutig Teil der Dev-Assist-Interaktion sind, etwa Assistenzkommandos mit Mention oder Markierungen wie `## Dev-Assist:` oder `# Dev-Assist Context`.

Nach Publish soll die relevante Anforderung nicht mehr über Rückfragen, Vorschläge und Bestätigungskommentare verteilt sein. Stattdessen stehen Titel und Beschreibung strukturiert im Issue; der Kommentarverlauf wird nicht mehr als primäre Anforderungsquelle benötigt.

4.8 Fehlerbehandlung und Robustheit

Die Architektur muss mit Fehlern externer Systeme umgehen: GitLab-API-Aufrufe können fehlschlagen, `glab` kann nicht authentifiziert sein, Tokens können fehlen, Open-code kann nicht installiert sein, Modellaufrufe können timeouten und Modellantworten können ungültig sein. Dev-Assist behandelt diese Fälle differenziert.

Beim Abruf des vollständigen GitLab-Kontexts nutzt der Processor einen Fallback. Wenn `getIssue` oder `listNotes` fehlschlägt, wird eine Warnung geloggt und mit den Daten aus dem Webhook-Payload weitergearbeitet. Die Analyse kann dadurch eingeschränkt sein, bleibt aber oft möglich.

Bei KI-Fehlern ist die Architektur strenger. Erzeugt der AI-Service keine gültige Analyse, wird kein normaler Kontext geschrieben. Das System versucht stattdessen, einen Fehlerkommentar zu posten und wirft den Fehler weiter. So wird verhindert, dass ungültiger Modelloutput später durch Publish übernommen wird.

GitLab-Schreiboperationen werden teilweise best effort behandelt. Ein fehlgeschlagener Analysekommentar kann neben einer lokal geschriebenen Kontextdatei stehen; beim Pu-

blish blockiert das Fehlschlagen einzelner Kommentar-Löschungen nicht zwangsläufig die Aktualisierung der Issue-Beschreibung. Solche Fälle werden geloggt.

Deduplication schützt gegen Mehrfachverarbeitung. Da Webhooks wiederholt eintreffen oder erneut aus der Ereignishistorie gesendet werden können (vgl. GitLab Docs, o. J.a, Abschnitt "Manage webhooks"), speichert Dev-Assist für eine konfigurierbare Zeit Schlüssel aus Projekt, Issue, Ereignistyp und Ereignis-ID. Wiederholte Ereignisse erhalten einen erfolgreichen 2xx-Status, werden aber nicht erneut verarbeitet. Weitere Robustheit entsteht durch Konfiguration über Umgebungsvariablen, etwa für Port, Mention, GitLab-URL, `glab`, Token, Signaturpflicht, KI-Provider, Timeout, Kontextverzeichnis und Deduplication-TTL.

4.9 Sicherheitskonzept

Das Sicherheitskonzept betrifft Webhook-Authentizität, Schutz vor ungewollter Verarbeitung, Begrenzung der KI-Autonomie und Umgang mit Secrets.

Webhook-Endpunkte sind öffentliche HTTP-Schnittstellen und müssen vor gefälschten Ereignissen geschützt werden. GitLab empfiehlt für neue Webhooks Signing Tokens. Dabei sendet GitLab die Header `webhook-id`, `webhook-timestamp` und, sofern ein Signing Token konfiguriert ist, `webhook-signature`. Die HMAC-SHA256-Signatur wird über `{message_id}.{timestamp}.{body}` berechnet; `message_id` und `timestamp` stammen aus den Headern, `body` ist der unveränderte JSON-Rohkörper. Der ältere Secret-Token-Mechanismus über `X-Gitlab-Token` bietet schwächere Garantien und wird für neue Webhooks nicht empfohlen (vgl. GitLab Docs, o. J.a, Abschnitt "Signing tokens"). Dev-Assist erfasst deshalb den Rohkörper, wertet `webhook-id`, `webhook-timestamp` und `webhook-signature` aus, prüft die Signatur vor der Verarbeitung und validiert den Zeitstempel gegen Replay-Angriffe. Die Verifikation ist in einem separaten Auth-Modul gekapselt.

Für lokale Entwicklung unterstützt der Prototyp einen Modus, in dem fehlende Signing Secrets zugelassen und als Warnung geloggt werden. Produktiv lässt sich die Signaturpflicht erzwingen. Zusätzlich verhindert das Mention-Gate, dass Dev-Assist auf beliebige Projektkommentare reagiert; selbst verfasste oder eindeutig generierte Dev-Assist-Kommentare werden ignoriert, um Bot-Schleifen zu vermeiden.

Die KI-Autonomie bleibt begrenzt. Das Modell erhält keine direkte GitLab-Schreibberechtigung, sondern liefert nur strukturierte Antworten. Die Anwendung prüft, rendert und entscheidet über weitere Systemhandlungen; produktive Änderungen erfolgen erst nach `@dev-assist publish`. Damit adressiert die Architektur sowohl Halluzinationsrisiken als auch OWASPs Risiko übermäßiger Handlungsfähigkeit (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6; OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency"). Secrets wie GitLab-Token, Webhook-Secrets und Modellkonfigurationen liegen nicht im Code, sondern in `.env`; Logging darf keine Secrets ausgeben.

4.10 Bild-, Screenshot- und Repository-Kontext

Bild- und Screenshot-Verarbeitung ist in der Gliederung als möglicher Bestandteil genannt, aber im aktuellen Prototyp nicht als eigenständige Bildanalyse-Komponente umgesetzt. Kapitel 3 grenzt bereits ab, dass umfassende Bild- oder Screenshot-Interpretation nicht zum Kern der Anforderungsanalyse gehört. Die Architektur behandelt diesen Punkt daher als Erweiterungsstelle.

Für einen produktiven Ausbau wären zusätzliche Schritte notwendig: Erkennung von Anhängen oder Markdown-Bildreferenzen, gesicherter Abruf über GitLab, Größen- und Dateitypprüfung, optionale OCR- oder Vision-Verarbeitung, Kennzeichnung unsicherer Bildinterpretationen und Integration der Ergebnisse in den Prompt. Bildinhalte müssten wie Textkommentare als Daten, nicht als Systemanweisungen behandelt werden.

Der aktuell umgesetzte Kontextausbau liegt bei der Repository-Zusammenfassung. Wenn der Opencode-Pfad aktiv ist, ruft Dev-Assist Projektmetadaten, Programmiersprachen, Dateibaum und ausgewählte Schlüsseldateien aus GitLab ab. Der `repo-summary`-Agent erzeugt daraus eine kompakte Markdown-Zusammenfassung mit Technologie-Stack, Projektstruktur, Befehlen, Architektur, wichtigen Dateien und Konventionen. Sie wird gecacht und unter `.dev-assist/repo-summary-<projectId>.md` gespeichert. Diese Zusammenfassung ersetzt nicht den Ticketkontext, sondern ergänzt ihn, damit technische Hinweise realistischer werden, ohne fachliche Ziele zu erfinden.

4.11 Validierung und Testkonzept der Architektur

Die Architektur wurde so gewählt, dass zentrale Entscheidungen isoliert testbar sind. Da die KI-Komponente selbst nicht vollständig deterministisch ist, konzentrieren sich Tests auf deterministische Systemteile: Parsing, Authentifizierung, Cleanup, Prompt-Aufbau, Rendering, Repository-Summary und Opencode-Ausgabeparsing.

Die GitLab-Payload-Tests prüfen, ob Issue- und Note-Events korrekt erkannt, Publish-Kommandos abgeleitet und selbst erzeugte Dev-Assist-Kommentare ignoriert werden. Authentifizierungstests prüfen Verhalten ohne Secret, bei fehlender Signatur, bei gültiger HMAC-Signatur und bei ungültiger Signatur. Cleanup-Tests stellen sicher, dass Systemnotes und normale Nutzerkommentare erhalten bleiben, während Dev-Assist-bezogene Kommentare gelöscht werden können. Formatter-Tests prüfen Rückfragen, Kontextdokumente und die Darstellung schwacher oder fehlender Abschnitte.

Prompt- und Clarification-Tests sichern, dass Repository-Zusammenfassungen in den Prompt gelangen und beantwortete Dev-Assist-Fragen nicht erneut gestellt werden. Opencode-Tests prüfen die Extraktion von Analyseergebnissen aus JSON-Stream und Export-Fallback. Repository-Summary-Tests sichern die Sammlung von Projektmetadaten, Sprachen, Dateibaum und Schlüsseldateien ab.

Damit wird nicht das Modell als deterministisch vorausgesetzt, sondern die Anwendung um das Modell herum robust gemacht. Dev-Assist nutzt generative KI als Assistenzkomponente, während Systemgrenzen, Datenflüsse und Freigabeschritte durch deterministische Anwendungsteile kontrolliert bleiben.

4.12 Zwischenfazit

Die Systemarchitektur von Dev-Assist verbindet GitLab-Webhooks, GitLab-APIs, Opencode-basierte Analyse, lokale Kontextpersistenz und einen kontrollierten Publish-Prozess. Die wichtigste Entwurfsentscheidung ist die Trennung zwischen Analyse und Übernahme: Das System kann Tickets analysieren, Rückfragen stellen und strukturierte Vorschläge erzeugen, aber produktive Änderungen erfolgen erst nach expliziter menschlicher Freigabe.

Die Architektur erfüllt damit die Anforderungen aus Kapitel 3. Sie verarbeitet relevante GitLab-Ereignisse, kapselt die Aktivierungslogik, ruft Ticket- und Kommentar-Kontext ab, validiert KI-Ausgaben, persistiert strukturierte Vorschläge und aktualisiert Issues erst im Publish-Schritt. Signaturprüfung, Deduplication, Bot-Schleifenschutz, Schema-Validierung, Logging und Mock-Modus begrenzen die Risiken eines KI-gestützten Webhook-Systems. Für Kapitel 5 bildet dieses Kapitel den konzeptionellen Rahmen für die konkrete Umsetzung in TypeScript, Express, GitLab-Client, Opencode-Integration, Prompt-Modulen, Kontextdateien und Tests.

5 Implementierung

Nachdem Kapitel 4 die Architektur von Dev-Assist beschrieben hat, erläutert dieses Kapitel die konkrete Umsetzung des Prototyps. Im Mittelpunkt stehen TypeScript, Express, GitLab-Anbindung, OpenCode-Integration, Promptgestaltung, JSON-Verarbeitung, Kontextpersistenz und Publish-Logik. Die Darstellung orientiert sich am tatsächlichen Projektstand und verweist daher auf vorhandene Module, Tests und Konfigurationsdateien.

Dev-Assist ist als reiner API-Dienst umgesetzt. Das System stellt keine eigene grafische Oberfläche bereit, sondern wird über GitLab-Issues, Kommentare, Webhooks sowie manuelle HTTP- beziehungsweise CLI-Aufrufe bedient. Damit ergänzt es den bestehenden GitLab-Prozess dort, wo Ticketinformationen entstehen. GitLab beschreibt Webhooks als Mechanismus, mit dem externe Anwendungen bei Ereignissen wie Issue-Aktualisierungen oder neuen Kommentaren informiert werden können (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook events"). Dev-Assist nutzt diesen Mechanismus, prüft eingehende Ereignisse auf Relevanz und stößt anschließend Analyse oder Veröffentlichung eines strukturierten Ticketvorschlags an.

Die Implementierung folgt drei Leitgedanken: Transportlogik, GitLab-Zugriffe, KI-Analyse, Formatierung, Kontextdateien und Publish-Schritt sind getrennt; generative KI erstellt nur Vorschläge und schreibt nicht selbst nach GitLab; externe Eingaben wie Webhook-Payloads, Kommentare und Modellantworten werden geparkt, eingegrenzt und validiert. Damit greift die Umsetzung die Anforderungen an Wartbarkeit, Testbarkeit, Robustheit und kontrollierte Autonomie aus Kapitel 3 und 4 auf.

5.1 Aufbau des TypeScript-Projekts

Das Projekt ist als Node.js-/TypeScript-Anwendung organisiert. `package.json` definiert den Dienst als ECMAScript-Modulprojekt und enthält die zentralen Skripte: `npm run dev` startet den Entwicklungsserver mit `tsx watch src/server.ts`, `npm run build` kompiliert nach `dist/`, `npm test` führt `tsx -test tests/**/*.test.ts` aus, und `npm run publish-issue - <projectId>/<issueId>` ermöglicht einen manuellen Publish-Schritt. Damit ist der Prototyp lokal im Entwicklungsmodus und als kompiliertes Node.js-Programm betreibbar.

TypeScript wird eingesetzt, weil Dev-Assist viele strukturierte externe Daten verarbeitet: GitLab-Webhooks, GitLab-API-Antworten, Agentenantworten, Kontextmetadaten und Konfigurationswerte. Das TypeScript-Handbook beschreibt TypeScript als statischen Typechecker für JavaScript, der Fehler vor der Laufzeit auffinden soll (vgl. TypeScript Docs, o. J., o. S.). Für Dev-Assist schließt das Laufzeitfehler nicht aus, hilft aber, interne Datenformen klar zu beschreiben, etwa in `src/services/gitlab/client.ts`

für Issues und Notes oder in `src/services/ai/schema.ts` für KI-Antworten.

`tsconfig.json` nutzt `NodeNext` für Modul- und Auflösungsstrategie, `ES2022` als Zielplattform, `strict: true` für strenge Typprüfung und `outDir: ./dist` für die Build-Ausgabe. Der Quellcode liegt unter `src/`, die Tests unter `tests/`, die OpenCode-Konfiguration im Projektwurzelverzeichnis und die Thesis-Dokumentation unter `doc/`.

Die Projektstruktur ist klein und modular gehalten:

Bereich	Aufgabe
<code>src/server.ts</code> , <code>src/app.ts</code> <code>src/config.ts</code> <code>src/routes/</code>	Serverstart, Middleware, Routen und Shutdown-Verhalten Einlesen und Normalisieren von Umgebungsvariablen Health-Check, GitLab-Webhooks und manuelle Issue-Aktionen
<code>src/services/gitlab/</code>	Webhook-Parsing, Mention-Erkennung, Authentifizierung, GitLab-Client und Kommentarbereinigung
<code>src/services/ai/</code>	Promptaufbau, Mock- und OpenCode-Analyse, Antwortschema, Formatierung und Klärungslogik
<code>src/services/context/</code>	Schreiben und Lesen von <code>context.md</code> und <code>context.json</code>
<code>src/services/processing/</code>	Orchestrierung von Analyse und Publish
<code>src/services/repositorySummary.ts</code>	Zusammenfassung von Repository-Kontext
<code>tests/</code>	Automatisierte Tests für deterministische Systemteile

Die Abhängigkeiten entsprechen dem Prototypumfang. `express` bildet den HTTP-Server, `dotenv` lädt die `.env`, `typescript`, `tsx` und Node-Typdefinitionen unterstützen Entwicklung und Tests. Die reale KI-Integration erfolgt derzeit über die OpenCode-CLI, obwohl `@opencode-ai/sdk` als Abhängigkeit vorhanden ist. Maßgeblich ist damit der CLI-Pfad: Die Anwendung startet `opencode run`, liest dessen Ausgabe und extrahiert daraus die strukturierte Analyse.

5.2 Konfiguration, Startverhalten und Logging

Die Laufzeitkonfiguration wird zentral in `src/config.ts` aufgebaut. Das Modul lädt die `.env` aus dem aktuellen Arbeitsverzeichnis und stellt eine typisierte `Config`-Struktur bereit. Wichtige Einstellungen sind Port, Log-Level, Mention-String, Bot-Benutzername, GitLab-Basis-URL, GitLab-Authentifizierungsmodus, Webhook-Signing-Secret, KI-Provider, Modell, Timeout, Kontextverzeichnis und Deduplication-Zeitfenster.

Für die lokale Entwicklung ist `AI_PROVIDER=mock` vorgesehen. Dieser Modus benötigt keine externen Modellzugänge und erzeugt eine deterministische Beispielanalyse. Für reale Analysen unterstützt der Prototyp `AI_PROVIDER=opencode`; dann wird ein konfiguriertes Modell über die OpenCode-CLI verwendet. Die GitLab-Anbindung kann über `glab` oder tokenbasierte REST-Aufrufe erfolgen. Standardmäßig ist `GITLAB_USE_GLAB=true`,

weil der Dienst auf einem bereits authentifizierten lokalen GitLab-CLI-Kontext aufbauen soll. GitLab beschreibt `glab` als offenes CLI-Werkzeug, mit dem GitLab-Funktionen direkt aus dem Terminal genutzt werden können (vgl. GitLab Docs, o. J.f, o. S.).

`src/server.ts` liest die Konfiguration, setzt das Log-Level und prüft bei aktivem OpenCode-Provider, ob die OpenCode-CLI im Pfad erreichbar ist. Anschließend startet der Server die von `createApp()` erzeugte Express-Anwendung auf dem konfigurierten Port. Optional kann `START_TUNNEL=true` gesetzt werden; dann startet `src/services/tunnel.ts` einen Cloudflare-Tunnel und gibt die öffentliche Webhook-URL für `/webhooks/gitlab/issues` aus.

Das Logging bleibt bewusst einfach. `src/utils/logger.ts` erzeugt Konsolenzeilen mit Zeitstempel, Log-Level, Nachricht und optionalem Kontextobjekt. Warnungen und Fehler gehen nach `stderr`, Informations- und Debug-Ausgaben nach `stdout`. Diese Strategie passt zum Prototyp, weil lokale Simulationen nachvollziehbar bleiben und keine zusätzliche Logging-Infrastruktur nötig ist. Für produktiven Betrieb wären strukturierte Logweiterleitung, Maskierung sensibler Daten, Request-Korrelation und Monitoring naheliegende Erweiterungen.

5.3 Express-Server und HTTP-Routen

Die Express-Anwendung wird in `src/app.ts` aufgebaut. Express eignet sich für den Prototyp, weil es Routing und Middleware für HTTP-Anwendungen bereitstellt. Die Express-Dokumentation beschreibt Routing als Zuordnung von Endpunkten und HTTP-Methoden zu Handlern; Middleware-Funktionen haben Zugriff auf Request, Response und die nächste Funktion im Request-Response-Zyklus (vgl. Express Docs, o. J.a, o. S.; Express Docs, o. J.b, o. S.). Dev-Assist nutzt diese Eigenschaften, um JSON-Payloads einzulesen, den Rohkörper für Webhook-Prüfungen zu erfassen und Requests an fachliche Routen zu delegieren.

Die wichtigste Middleware ist `express.json()` mit einem Limit von `1mb` und einer `verify`-Funktion. Sie speichert den ursprünglichen Request-Body als `rawBody`, damit HMAC-Prüfungen auf dem tatsächlich empfangenen Byteinhalt beruhen und nicht auf einem nachträglich serialisierten JSON-Objekt. Danach protokolliert eine Request-Logging-Middleware Methode, Pfad, GitLab-Event-Header, Statuscode und Dauer.

Die Anwendung registriert drei Routenbereiche:

Route	Zweck
<code>/health</code>	einfacher Health-Endpunkt mit Status, Servicename und Zeitstempel
<code>/webhooks/gitlab/issues</code>	primärer Webhook-Endpunkt für Issue- und Kommentarereignisse
<code>/api/issues/:projectId/:issueId/subscribe</code>	manuelle Issue- und Publish-Endpunkte
<code>/api/issues/:projectId/:issueId/publish</code>	

Die manuellen Issue-Routen in `src/routes/issues.ts` verwenden dieselbe Prozess-

und Publish-Logik wie der Webhook-Pfad. Ein Process-Aufruf startet `processIssue`, ein Publish-Aufruf startet `publishIssue` und kann zusätzliche GitLab-Felder wie `state_event` oder Labels aus dem Request-Body übernehmen.

Der Webhook-Endpunkt akzeptiert Ereignisse möglichst schnell mit `202 Accepted`, während Analyse oder Veröffentlichung asynchron ausgeführt werden. Diese Trennung ist wichtig, weil eine OpenCode-Analyse länger dauern kann als GitLab auf eine Webhook-Antwort warten sollte. GitLab empfiehlt für Webhook-Receiver schnelle Antworten mit `200` oder `201`, keine Verarbeitung im selben Request und Beachtung möglicher Duplikate (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook receiver requirements"). Dev-Assist folgt der Zielrichtung durch schnelle `2xx`-Antwort und ausgelagerte Verarbeitung; die Wahl von `202 Accepted` kennzeichnet die Annahme zur späteren Bearbeitung, weicht aber von den ausdrücklich genannten Beispielstatuscodes ab.

5.4 Webhook-Verarbeitung, Deduplication und Aktivierung

Die Webhook-Verarbeitung liegt in `src/routes/gitlabWebhooks.ts`. Bei einem POST auf `/webhooks/gitlab/issues` durchläuft der Request Signaturprüfung, Payload-Parsing, Deduplication, Bot- und Generierungsfiler, Mention- und Kommandoentscheidung sowie den Start von Analyse oder Publish.

Zunächst ruft die Route `verifyWebhookRequest` aus `src/services/gitlab/auth.ts` auf. Ist kein Signing Secret konfiguriert, akzeptiert der Prototyp die Anfrage im Entwicklungsmodus und protokolliert eine Warnung. Für produktive GitLab-Webhook-Authentifizierung ist das Signing-Token-Verfahren relevant: GitLab sendet `webhook-id`, `webhook-timestamp` und bei konfiguriertem Signing Token `webhook-signature`; die Signatur wird über `{message_id}.{timestamp}.{body}` aus Message-ID, Zeitstempel und rohem JSON-Body berechnet. Der Zeitstempel sollte auf Aktualität geprüft werden, um Replay-Angriffe zu erschweren. Falls `GITLAB_REQUIRE_SIGNATURE=true` gesetzt ist, führt eine fehlgeschlagene Prüfung zu `401 Unauthorized`; andernfalls wird im Entwicklungsmodus weiterverarbeitet. GitLab empfiehlt Signing Tokens, da sie Authentizität und Integrität per HMAC-SHA256 besser absichern als ein reiner Secret-Token-Header. Für ältere Installationen bleibt `X-Gitlab-Token` relevant, da GitLab Signing Tokens als in GitLab 19.0 eingeführt und in GitLab 19.1 allgemein verfügbar dokumentiert (vgl. GitLab Docs, o. J.a, Abschnitte "Create a webhook", "Signing tokens" und "Delivery headers").

Danach überführt `parseGitLabWebhook` die Payload in ein internes `ParsedWebhook`-Objekt. Bei Issue-Events liest der Parser Projekt-ID, Issue-IID, Titel, Beschreibung und Aktion aus `object_attributes`. Bei Note-Events liest er den Kommentar aus `object_attributes.note` und die zugehörigen Issue-Daten aus dem `issue`-Objekt. GitLab beschreibt Comment Events als Ereignisse für neue oder bearbeitete Kommentare und legt Note-Daten in `object_attributes` sowie Zielinformationen unter anderem im `issue`-Objekt ab (vgl. GitLab Docs, o. J.b, Abschnitt "Comment events").

Die Aktivierung erfolgt über `src/services/gitlab/mention.ts`. Standardmäßig wird `@dev-assist` erkannt; `hasMention` prüft die Mention, `stripMention` entfernt sie für die

Kommandoanalyse. Einfache Markdown- und Formatierungszeichen am Anfang werden toleriert. Die aktuelle Erkennung ist großzügiger als die strengere Zielregel aus Kapitel 3, nach der die Mention auf der ersten Inhaltszeile stehen sollte: Für Issue-Events werden Titel und Beschreibung geprüft, für Note-Events der Kommentartext. Diese Toleranz erhöht die Bedienbarkeit, sollte produktiv aber bewusst gegen die strengere Aktivierungsregel abgewogen werden.

`src/services/gitlab/commands.ts` bestimmt anschließend das Kommando. Beginnt der Text nach Entfernen der Mention mit `publish`, wird das Ereignis als Publish-Kommando interpretiert; andernfalls startet eine Analyseanforderung. Zusätzlich verhindert eine im Arbeitsspeicher gehaltene Deduplication Mehrfachverarbeitung. Sie speichert für ein konfigurierbares Zeitfenster einen Schlüssel aus Projekt-ID, Issue-IID, Ereignisart und Ereignis-ID beziehungsweise Objekt-ID. Wiederholte Ereignisse werden mit `202 Accepted` beantwortet, aber nicht erneut verarbeitet. Für mehrere Instanzen oder Neustarts wäre eine persistente oder verteilte Deduplication erforderlich.

Schließlich ignoriert der Router selbst erzeugte oder selbst verfasste Kommentare. Dafür vergleicht der Parser den GitLab-Autor mit dem konfigurierten Bot-Benutzernamen und erkennt generierte Kommentare über Marker wie `## Dev-Assist: generated by Dev-Assist` oder `# Dev-Assist Context`. So interpretiert Dev-Assist eigene Vorschläge oder Rückfragen nicht erneut als Nutzeranweisung.

5.5 GitLab-Integration über glab und REST-Fallback

Die GitLab-Anbindung ist in `src/services/gitlab/client.ts` gekapselt. Das Modul stellt ein einheitliches Interface bereit, unabhängig davon, ob die Ausführung über `glab` oder direkte REST-Aufrufe mit Personal Access Token erfolgt. Dadurch bleibt die Prozesslogik frei von Details wie URL-Kodierung, CLI-Argumenten, JSON-Ausgabe, Pagination oder HTTP-Headern.

Der primäre Pfad nutzt `glab api`. Das Kommando wird mit `execFile` gestartet, die Ausgabe als JSON gelesen und geparkt. Projektpfade werden auf `/projects/<id>/...` normalisiert. Bei selbstverwalteten GitLab-Instanzen kann `GITLAB_GLAB_HOSTNAME` gesetzt werden; das Modul bereinigt den Wert auf einen Hostnamen. Ist `GITLAB_TOKEN` vorhanden, kann derselbe Client REST-Aufrufe mit `PRIVATE-TOKEN` ausführen. Ohne `glab` und ohne Token sind Schreiboperationen nicht zuverlässig möglich, was der Client beim Start protokolliert.

Für den Ticketkontext nutzt der Client `getIssue` und `listNotes`. Die GitLab Issues API dient zum Lesen und Aktualisieren von Issues; die Notes API stellt Endpunkte für Issue-Kommentare bereit, darunter Abrufen, Erstellen und Löschen von Notes (vgl. GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.). Für Repository-Kontext existieren `getProject`, `getRepositoryLanguages`, `getRepositoryTree` und `getRepositoryFile`. Für die Rückgabe an GitLab gibt es `createNote`, `deleteNote` und `updateIssue`.

`updateIssue` nimmt ein Objekt mit GitLab-Feldern entgegen, normalisiert Arrays für Labels oder Assignees und setzt Werte entweder per REST-JSON oder per `glab -field`. So kann der Publish-Schritt neben Titel und Beschreibung auch optionale Zu-

satzaktionen ausführen, etwa `state_event=close` oder `add_labels=ready`. GitLab dokumentiert beim Issue-Update unter anderem `description`, `title`, `state_event`, `add_labels` und `remove_labels` als mögliche Parameter (vgl. GitLab Docs, o. J.c, Abschnitt "Update an issue").

Fehler werden abhängig vom Schritt behandelt. In `processor.ts` wird ein fehlgeschlagener Kontextabruf protokolliert und mit vorhandenen Webhook-Daten weitergearbeitet. Bei Publish ist das Verhalten strenger: Fehlt die Kontextdatei oder schlägt die Issue-Aktualisierung fehl, bricht der Publish-Schritt ab. Das ist sinnvoll, weil eine eingeschränkte Analyse als Vorschlag kontrollierbar bleibt, ein fehlerhafter Publish aber produktive Ticketinhalte verändern würde.

5.6 Prozesssteuerung der Analyse

Die zentrale Orchestrierung liegt in `src/services/processing/processor.ts.processIssue` nimmt Projekt-ID, Issue-IID und optionalen Zusatztext aus dem Webhook entgegen, erstellt einen Logger mit Projekt-, Issue- und Phasenkontext und versucht, das aktuelle Issue sowie die Kommentare über den GitLab-Client zu laden. Falls dies fehlschlägt, wird mit einem Fallback aus dem Webhook-Text weitergearbeitet.

Anschließend wird der Repository-Summary-Provider aufgerufen. Ist eine Zusammenfassung verfügbar, wird sie in den Prompt integriert, damit der Agent technische Hinweise realistischer formulieren kann, ohne fachliche Anforderungen zu ersetzen. Danach entsteht ein Kontextobjekt mit Projekt, Issue, Kommentaren, Rohtext und optionaler Repository-Zusammenfassung.

Der Aufruf `ai.analyzeTicket(ctx)` bildet die Schnittstelle zur KI-Schicht. Schlägt die Analyse vollständig fehl, versucht der Processor, einen Fehlerkommentar in GitLab zu posten, und wirft den Fehler weiter. So entsteht keine Kontextdatei aus einer fehlgeschlagenen oder unparsebaren KI-Antwort.

Nach erfolgreicher Analyse entscheidet der Processor, welcher Kommentar gepostet wird. Bei mindestens zwei substantiellen offenen Fragen wird `renderClarificationComment` verwendet; andernfalls erzeugt `renderRequirementAnalysis` einen vollständigen strukturierten Vorschlag. Diese Heuristik verhindert, dass einzelne offene Punkte den gesamten Vorschlag blockieren, während mehrere relevante Fragen als Hinweis auf eine noch instabile Ticketgrundlage behandelt werden.

Unabhängig vom Erfolg des GitLab-Kommentars versucht die Anwendung, den strukturierten Kontext lokal zu persistieren. `writeContextFile` schreibt `.dev-assist/issues/<projectId>` und speichert, sofern vorhanden, den vorgeschlagenen Titel in `context.json`. `processFromWebhook` verbindet Parser und Prozesslogik, liest Projekt-ID, Issue-IID und auslösenden Text aus dem `ParsedWebhook`, prüft die Mention defensiv erneut und ruft `processIssue` auf.

5.7 OpenCode-Agenten-Integration und Prompt-Design

Die KI-Schicht liegt in `src/services/ai/service.ts`. `createAiService` stellt ein einheitliches Interface bereit. Im Mock-Modus wird eine deterministische Beispielanalyse erzeugt; im OpenCode-Modus wird ein Agentenlauf gestartet. OpenCode beschreibt Agents als spezialisierte KI-Assistenten, die für konkrete Aufgaben und Workflows mit eigenen Prompts, Modellen und Werkzeugzugriffen konfiguriert werden können (vgl. OpenCode Docs, o. J.a, Abschnitt "Agents"). Dev-Assist nutzt dieses Konzept kontrolliert: Der Agent erhält vorbereiteten Kontext und soll ausschließlich ein JSON-Objekt zurückgeben.

Die OpenCode-Konfiguration steht in `opencode.json`. Dort sind `dev-assist-analyzer` für die Ticketanalyse und `repo-summary` für Repository-Zusammenfassungen definiert. Basisprompts liegen in `.opencode/prompts/requirement-analysis.md` und `.opencode/prompts/r` der umfangreiche Analyseprompt wird jedoch zentral in `src/services/ai/instructions.ts` gepflegt. Dieses Modul enthält Persona, Kernregeln, JSON-Schema-Beispiel, Befüllungsregeln und Klärungshinweise.

`buildUserPrompt` kombiniert die Analyseanweisungen mit GitLab-Issue, optionaler Repository-Zusammenfassung, bereits beantworteten Dev-Assist-Rückfragen und den letzten Kommentaren. Am Ende steht die Aufforderung, ausschließlich ein JSON-Objekt auszugeben. Damit erhält das Modell denselben Regeltext unabhängig vom Ticket, während Ticketkontext und Systemregeln getrennt bleiben und frühere Antworten auf Rückfragen hervorgehoben werden.

Prompt Injection bleibt ein Risiko. OWASP beschreibt Prompt Injection als Schwachstelle, bei der Eingaben das Verhalten eines LLM unbeabsichtigt beeinflussen können (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection"). Dev-Assist begrenzt dieses Risiko nicht nur durch Prompting, sondern auch technisch: Die Anwendung entscheidet, welche Daten in den Prompt gelangen, ruft nur einen vordefinierten Agenten auf, akzeptiert nur schema-konforme JSON-Antworten und gibt dem Agenten keine direkte GitLab-Schreibberechtigung.

Im OpenCode-Pfad kopiert `analyzeWithOpencode` erforderliche Dateien nach `.opencode/runtime`, startet `opencode run` mit `-format json`, `-agent dev-assist-analyzer`, dem konfigurierten Modell und dem vollständigen Prompt, sammelt `stdout` und `stderr`, setzt einen Timeout und beendet zu lange laufende Prozesse. Die OpenCode-CLI-Dokumentation beschreibt `opencode run`, Agent-Auswahl, Formatoptionen und Exportfunktionen als CLI-Funktionen (vgl. OpenCode Docs, o. J.d, o. S.).

Die Ausgabe wird robust verarbeitet. `parseOpencodeAnalysisOutput` entfernt ANSI-Steuerzeichen, extrahiert JSON-Objekte, berücksichtigt `<task_result>`-Blöcke und durchsucht JSONL-Events. Wenn der JSON-Stream keine finale Textantwort enthält, versucht die Implementierung anhand einer Session-ID einen Export über `opencode export`. Dieser Export-Fallback ist durch `tests/ai/opencode.test.ts` abgesichert.

5.8 JSON-Antwortverarbeitung und Laufzeitvalidierung

Die Struktur der KI-Antwort ist in `src/services/ai/schema.ts` definiert. `RequirementAnalysis` enthält `summary`, `sourceBasis`, `implementationTicket`, `acceptanceCriteria`, `technicalNotes`, `openQuestions`, `risks` und `validationSteps`. Das verschachtelte `implementationTicket` enthält Titel, Ziel, Scope, Out-of-Scope, User Stories, funktionale Anforderungen, technischen Ansatz, Umsetzungsschritte und Definition of Done.

Diese Struktur zwingt den Agenten, zwischen Zusammenfassung, Umsetzungsticket, Akzeptanzkriterien, offenen Fragen und Risiken zu unterscheiden. Dadurch wird die Antwort maschinenlesbar und nachgelagert renderbar. Freier LLM-Fließtext wäre für diesen Workflow ungeeignet, weil die Anwendung dann nicht zuverlässig Titel, Beschreibung, Rückfrage oder Validierungsschritt unterscheiden könnte.

Die Laufzeitvalidierung erfolgt über `parseAnalysisJson` und `validateRequirementAnalysis`. Zunächst werden mögliche Markdown-Codezäune entfernt, dann wird JSON geparkt. Anschließend prüft `validateRequirementAnalysis`, ob alle Pflichtfelder und die Pflichtfelder des verschachtelten `implementationTicket` vorhanden sind. Diese Validierung ist bewusst einfach und ohne zusätzliche Bibliothek umgesetzt. Sie prüft die Grundstruktur, aber nicht jedes Array-Element und jeden Feldtyp so streng wie eine vollständige JSON-Schema- oder Zod-Validierung.

Für den Prototyp ist das nachvollziehbar, weil der Schwerpunkt auf der Pipeline aus Prompt, Parsing, Pflichtfeldern, Rendering, Kontextdatei und Publish liegt. Gleichzeitig bleibt es eine Grenze: TypeScript-Typen gelten zur Entwicklungszeit, nicht für externe Laufzeitdaten. Eine produktive Weiterentwicklung sollte daher eine strengere Schema-Validierung ergänzen.

Die Formatierung übernimmt `src/services/ai/formatter.ts.renderClarificationComment` erstellt Rückfragekommentare, wenn wichtige Informationen fehlen. `renderRequirementAnalysis` erzeugt das vollständige Kontextdokument mit Abschnitten wie Summary, Goal, Scope, User Stories, Functional Requirements, Technical Approach, Acceptance Criteria, Open Questions, Risks and Assumptions und Validation Steps. Schwache Listenwerte wie "not specified", "to be confirmed" oder "unknown" werden durch "Not enough information available yet." ersetzt, damit unsichere Informationen nicht scheinbar konkret erscheinen. Dies passt zum Umgang mit Halluzinationsrisiken: NIST beschreibt Confabulation beziehungsweise Hallucination als überzeugend präsentierte, aber falsche oder irreführende Inhalte (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6).

5.9 Umgang mit Rückfragen und Kommentarverlauf

Ein wichtiges Implementierungsdetail ist die Behandlung bereits beantworteter Rückfragen. Ohne diese Logik könnte das Modell bei erneuter Analyse dieselben Fragen wiederholen, obwohl sie im Kommentarverlauf bereits beantwortet wurden. `src/services/ai/clarification` adressiert dieses Problem.

Das Modul erkennt Dev-Assist-Rückfragekommentare über Marker wie `Dev-Assist: More information needed` und extrahiert Fragen aus Bullet- oder Nummerierungszeilen. Danach sucht es spätere Nutzerkommentare mit Dev-Assist-Mention. Diese Antworten werden priorisiert in den Prompt aufgenommen. Auch Antworten wie "no info" gelten als Antwort; das Modell soll die Frage dann nicht erneut stellen, sondern fehlende Informationen als "not specified" oder "to be confirmed" weiterführen.

Nach der Analyse entfernt `removeAnsweredOpenQuestions` offene Fragen, die bereits beantworteten Fragen stark ähneln. Dafür nutzt das Modul Tokenisierung mit Stoppwortliste und einen Ähnlichkeitswert. Die Tests in `tests/aiPrompt.test.ts` sichern sowohl die Aufnahme der Antworten in den Prompt als auch das Entfernen ähnlich formulierter offener Fragen ab. Die Lösung ist kein perfekter Dialogspeicher, aber deterministisch testbar und für den Prototyp ausreichend.

5.10 Repository-Kontext und Grenzen der Bildverarbeitung

Dev-Assist kann optional eine Repository-Zusammenfassung in den Prompt aufnehmen. `src/services/repositorySummary.ts` sammelt über den GitLab-Client Projektmetadata, Programmiersprachen, den Repository-Baum und ausgewählte Schlüsseldateien wie `package.json`, `tsconfig.json`, `README.md`, `AGENTS.md` oder Build-Dateien. Der Kontext ist begrenzt: maximal 300 Baumeinträge, 8 Schlüsseldateien und 8000 Zeichen pro Datei. So wird verhindert, dass große Repositories den Prompt überladen.

Die Daten werden dem `repo-summary`-Agenten übergeben. Der Prompt fordert eine knappe Markdown-Zusammenfassung zu Technology Stack, Project Structure, Key Commands, Architecture, Important Files und Conventions. Die resultierende Zusammenfassung wird im Speicher gecacht und unter `.dev-assist/repo-summary-<projectId>.md` persistiert.

Diese Zusammenfassung soll keine fachlichen Anforderungen erfinden, sondern technische Hinweise plausibler machen. Wenn ein Ticket eine Änderung an einem API-Dienst beschreibt und die Repository-Zusammenfassung Express, TypeScript und bestimmte Befehle zeigt, kann der Agent technische Umsetzungshinweise vorsichtiger einordnen. Fehlt die Zusammenfassung, läuft die Analyse ohne Zusatzkontext weiter.

Die Behandlung von Bildern und Screenshots ist dagegen nicht als eigenständige Bildanalyse umgesetzt. Markdown-Links oder Kommentartext können zwar im Prompt landen, es gibt aber keine Komponente für Bilddownload, Dateitypprüfung, OCR oder Vision-Analyse. Bildverarbeitung ist damit eine Erweiterungsstelle. Das ist relevant, weil Screenshots zusätzliche Sicherheits- und Validierungsfragen aufwerfen, etwa Zugriff auf Anhänge, Größenlimits, sensible Inhalte, OCR-Fehler und Prompt Injection über Bildtext.

5.11 Kontextpersistenz und Publish-Kommando

Die Trennung zwischen Analyse und Publish begrenzt die Systemautonomie. OWASP beschreibt "Excessive Agency" als Risiko, wenn LLM-basierte Systeme zu weitreichende Berechtigungen oder Handlungsspielräume besitzen (vgl. OWASP, 2025b, Abschnitt "LLM06:2025 Excessive Agency"). Dev-Assist begegnet diesem Risiko, indem die KI-Schicht keinen GitLab-Publish ausführt. Sie erzeugt nur eine strukturierte Analyse, die erst nach einem expliziten Publish-Kommando übernommen wird.

Die Persistenz liegt in `src/services/context/writer.ts` und `src/services/context/reader.ts`. `writeContextFile` erzeugt `.dev-assist/issues/<projectId>/<issueId>/`, schreibt dort `context.md` und optional `context.json`. `context.md` enthält den gerenderten Vorschlag, `context.json` derzeit vor allem den vorgeschlagenen Titel. Beim Publish liest `readContextFile` den Markdown-Inhalt und `readContextMetadata` die Metadaten. Fehlt die Metadaten-datei, wird mit der Beschreibung allein weitergearbeitet.

Der Publish-Schritt befindet sich in `src/services/processing/publisher.ts`. Zunächst wird der gespeicherte Kontext gelesen. Danach entfernt `renderPublishedDescription` Abschnitte, die nicht in die finale Issue-Beschreibung gehören, etwa den separaten Titelabschnitt. Der Titel wird bevorzugt aus `context.json` übernommen, andernfalls aus dem Markdown extrahiert. So landet er im GitLab-Issue-Titel und nicht redundant im Beschreibungstext.

Vor der Aktualisierung bereinigt der Publisher den Kommentarverlauf. Er lädt die Notes und ruft `filterDeletableNotes` aus `src/services/gitlab/cleanup.ts` auf. Nicht gelöscht werden Systemnotes und normale Nutzerkommentare ohne Dev-Assist-Bezug. Löschar sind Kommentare mit Mention sowie generierte Dev-Assist-Kommentare mit bekannten Markern. Einzelne Löschfehler blockieren den Publish nicht, sondern werden protokolliert.

Anschließend aktualisiert `updateIssue` Titel und Beschreibung des GitLab-Issues. Optional können Zusatzfelder wie Labels oder Statuswechsel übergeben werden. Diese Möglichkeit nutzen der manuelle HTTP-Endpunkt und `src/cli/publish-issue.ts`. Der CLI-Helfer interpretiert ein Argument wie `123/42 close,ready` als Projekt-ID, Issue-IID und optionale Zusatzaktionen. Ohne vorherigen Process-Schritt fehlt die Kontextdatei; ohne explizites Publish-Kommando wird kein Vorschlag übernommen. Damit bleibt die endgültige Änderung an eine bewusste Benutzerhandlung gebunden.

5.12 Simulation, lokale Ausführung und Tests

Der Prototyp kann lokal ohne echte GitLab- oder Modellabhängigkeit geprüft werden. Der Mock-Modus erzeugt eine Beispielanalyse und ermöglicht Tests der Routen, Formatierung, Kontextdateien und Publish-Logik ohne externe KI. Manuelle Endpunkte und der CLI-Helfer erlauben gezielte Einzelschritte: Process erzeugt Vorschlag und Kontextdatei, Publish liest diese Datei und aktualisiert das Issue.

GitLab-Webhooks können lokal per `curl` oder PowerShell an `/webhooks/gitlab/issues` gesendet werden. `README.md` enthält dafür ein Beispiel mit `object_kind: "issue"`,

Projekt-ID, Issue-IID, Titel und Beschreibung mit `@dev-assist`. Soll GitLab den lokalen Dienst erreichen, kann `src/services/tunnel.ts` mit Cloudflare Tunnel eine öffentliche `trycloudflare.com`-URL erzeugen und den Webhook-Pfad anhängen.

Die automatisierten Tests konzentrieren sich auf deterministische Systemteile:

Testdatei	Abgesicherter Bereich
<code>tests/gitlab.test.ts</code>	Parsing von Issue- und Note-Events, Publish-Kommando, eigene und generierte Kommentare
<code>tests/auth.test.ts</code>	Verhalten ohne Secret, fehlende, gültige und ungültige HMAC-Signaturen
<code>tests/cleanup.test.ts</code>	Auswahl löschrbarer und nicht löschrbarer Kommentare
<code>tests/formatter.test.ts</code>	Rückfragekommentare, Kontextdokumente und Platzhalter für schwache Abschnitte
<code>tests/aiPrompt.test.ts</code>	Repository-Summary, beantwortete Rückfragen und Filterung offener Fragen
<code>tests/aiOpenCode.test.ts</code>	Parsing über OpenCode-Export-Fallback
<code>tests/repositorySummary.test.ts</code>	Secrets, Caching und Extraktion von Repository-Zusammenfassungen
<code>tests/config.test.ts</code> , <code>tests/tunnel.test.ts</code>	Konfigurationsdetails und Cloudflare-Tunnel-Helfer

Diese Teststrategie passt zum Systemcharakter. Das Modell selbst ist nicht deterministisch und wird deshalb nicht wie eine reine Funktion getestet. Stattdessen werden die festen Grenzen um das Modell geprüft: Promptinhalt, erwartete Struktur, Kommentare, Kontextdateien, löschrbare GitLab-Kommentare und Extraktion der OpenCode-Ausgabe. Dadurch entsteht Vertrauen in die Anwendungsteile, die das Modell kontrollieren und begrenzen.

5.13 Zwischenfazit

Die Implementierung setzt die Architektur aus Kapitel 4 als modularen TypeScript-/Express-Dienst um. `server.ts` und `app.ts` bilden den HTTP-Einstieg, `config.ts` kapselt Umgebungsvariablen, der GitLab-Webhook-Router verarbeitet Ereignisse asynchron, und die Prozessschicht trennt Analyse und Publish. Die GitLab-Integration abstrahiert `glab` und REST-Fallback, die KI-Schicht bindet OpenCode über spezialisierte Agents ein, und Kontextdateien bilden den Übergabepunkt zwischen Vorschlag und kontrollierter Übernahme.

Prägend ist die Begrenzung der KI-Komponente. Der Agent erhält vorbereiteten Kontext und feste Analyseanweisungen, muss ein JSON nach definiertem Schema liefern und kann nicht selbst produktive GitLab-Änderungen durchführen. Die Anwendung validiert, rendert, persistiert und veröffentlicht erst nach explizitem Publish-Kommando. Damit zeigt der Prototyp, wie ein KI-gestützter GitLab-Assistent Ticketinformationen

strukturieren kann, ohne menschliche Freigabe und technische Nachvollziehbarkeit aus dem Prozess zu entfernen.

Gleichzeitig bleiben Grenzen sichtbar. Die Mention-Erkennung ist toleranter als die strengere Zielregel aus der Anforderungsanalyse. Die JSON-Validierung prüft Pflichtfelder, aber nicht jedes Detail des Schemas. Bild- und Screenshot-Kontext ist noch keine eigene Implementierung, sondern eine Erweiterungsstelle. Kapitel 6 kann daran anschließen, indem es Tests, verbleibende Risiken und die Aussagekraft der Evaluation systematisch bewertet.

6 Qualitätssicherung und Evaluation

Dieses Kapitel untersucht, in welchem Umfang der implementierte Dev-Assist-Prototyp die in Kapitel 3 formulierten Anforderungen nachweisbar erfüllt und an welchen Stellen die Aussagekraft der bisherigen Prüfung begrenzt bleibt. Qualitätssicherung besitzt bei Dev-Assist zwei unterschiedliche Ebenen. Einerseits müssen die deterministischen Anwendungsteile zuverlässig funktionieren: Webhook-Payloads sind zu parsen, Signaturen und Antwortformate zu prüfen, Kommentare zu filtern, Kontextdateien zu verarbeiten und Publish-Daten korrekt aufzubereiten. Andererseits ist die inhaltliche Qualität einer generativen KI-Ausgabe zu bewerten. Hier geht es nicht nur um syntaktisch gültiges JSON, sondern darum, ob fehlende Informationen erkannt, präzise Rückfragen gestellt und nutzbare, inhaltlich abgesicherte Ticketvorschläge erzeugt werden.

Die beiden Ebenen dürfen nicht gleichgesetzt werden. Eine vollständig grüne Testsuite belegt, dass die geprüften Programmteile für die verwendeten Eingaben das erwartete Verhalten zeigen. Sie belegt jedoch weder die semantische Qualität beliebiger Modellantworten noch die Funktionsfähigkeit mit jeder GitLab- oder Opencode-Version. Easterbrook et al. betonen, dass die Wahl einer empirischen Methode von Forschungsfrage, verfügbaren Ressourcen, Kontrollmöglichkeiten und Kontext abhängt und jede Methode nur begrenzte, qualifizierte Evidenz liefert (vgl. Easterbrook et al., 2008, S. 285 f.). Die Evaluation wird deshalb als Kombination aus reproduzierbarer automatisierter Prüfung, szenariobasierter Komponentensimulation, Anforderungsabgleich und expliziter Diskussion der nicht geprüften Bereiche aufgebaut.

6.1 Zielsetzung und Untersuchungsrahmen

Ziel der Qualitätssicherung ist nicht der Nachweis einer fehlerfreien oder produktionsreifen Anwendung. Untersucht wird vielmehr, ob der Prototyp seinen Kernworkflow technisch kontrolliert abbildet und welche Aussagen aus den vorhandenen Tests über seine Robustheit und fachliche Eignung abgeleitet werden können. Der Untersuchungsgegenstand ist der Repository-Stand vom 21.07.2026. Als ausführbare Prüfbasis dienen der TypeScript-Build und die unter `tests/` vorhandenen automatisierten Tests. Ergänzend werden Implementierung, Tests und Anforderungen statisch miteinander verglichen.

Wohlin et al. beschreiben empirische Untersuchungen in der Softwaretechnik als einen Prozess aus Abgrenzung, Planung, Durchführung, Analyse und Ergebnisdarstellung (vgl. Wohlin et al., 2012, S. 73 ff.). Dieses Kapitel folgt diesem Grundgedanken in einer für den Prototyp angemessenen Form. Zunächst wird festgelegt, welche Quali-

tätsaussagen mit welcher Evidenz geprüft werden. Danach werden Testumgebung und Testfälle beschrieben, die Ergebnisse zusammengeführt und schließlich ihre Grenzen bewertet. Es handelt sich jedoch nicht um ein kontrolliertes Experiment: Es gibt weder Versuchsgruppen noch eine unabhängige Variable, deren kausale Wirkung gemessen wird. Ebenso liegt keine Feldstudie mit Entwicklungsteams vor. Die Untersuchung ist eine verifikationsorientierte Evaluation eines einzelnen Prototyps.

Die verwendeten Evidenzarten beantworten unterschiedliche Fragen:

Evidenzart	Untersuchte Frage	Mögliche Aussage
TypeScript-Build	Ist der untersuchte Quellstand statisch typisierbar und kompilierbar?	Der Quellstand lässt sich ohne TypeScript-Fehler übersetzen.
Automatisierte Unit-Tests	Verhalten sich isolierte Parser-, Validierungs-, Formatierungs- und Hilfsfunktionen für definierte Eingaben korrekt?	Das erwartete deterministische Verhalten ist für die abgedeckten Fälle reproduzierbar.
Komponenten- und Routentests	Funktionieren mehrere reale Anwendungsteile gemeinsam, wenn externe Systeme ersetzt werden?	Schnittstellen zwischen ausgewählten Komponenten sind für die simulierten Fälle konsistent.
Anforderungsabgleich	Decken Implementierung und Tests die Anforderungen FA-1 bis FA-11 sowie NFA-1 bis NFA-10 ab?	Erfüllung, Teilabdeckung und Abweichungen werden nachvollziehbar.
Inhaltsbezogenes Bewertungsraster	Nach welchen Kriterien wären reale KI-Ausgaben zu bewerten?	Semantische Qualität wird operationalisiert; ohne Korpus und Bewertungen entsteht aber noch kein empirischer Wirksamkeitsnachweis.

Diese Trennung verhindert eine Überinterpretation der Testergebnisse. Der Build prüft beispielsweise keine Laufzeitdaten. Ein Schema-Test prüft keine fachliche Wahrheit. Ein simulierter Opencode-Prozess zeigt, dass der Export-Fallback verarbeitet wird, aber nicht, dass ein reales Modell verlässlich gute Tickets erzeugt. Die Evaluation bewertet daher sowohl positive Nachweise als auch Lücken.

6.2 Teststrategie

Die Teststrategie konzentriert sich auf die kontrollierbaren Grenzen um das Sprachmodell. Diese Schwerpunktsetzung folgt aus der Architektur: Das Modell erhält vorbereiteten Kontext und erzeugt eine strukturierte Antwort, während die Anwendung Aktivierung, Datenabruf, Parsing, Formatierung, Persistenz und Publish steuert. Gerade weil Modellantworten nicht vollständig deterministisch sind, müssen die deterministischen Schutzmechanismen zuverlässig geprüft werden.

Die Tests lassen sich in vier Ebenen einordnen:

1. **Unit-Tests für reine oder weitgehend isolierte Funktionen.** Dazu gehören Webhook-Parsing, Kommandoerkennung, Schema-Validierung, Markdown-Formatierung, Kommentarbereinigung, Konfigurationswerte und die Aufbereitung von Publish-Daten. Diese Tests sind schnell, reproduzierbar und benötigen keine Netzwerkverbindung.
2. **Komponententests mit simulierten Abhängigkeiten.** Der Repository-Summary-Service erhält einen simulierten GitLab-Client. Die Opencode-Tests legen temporäre ausführbare Dateien in den Suchpfad und prüfen damit den realen Prozessstart sowie die Verarbeitung von Standardausgabe und Export-Fallback, ohne einen Modellaufruf auszuführen.
3. **HTTP-Routentests.** `tests/webhookRoute.test.ts` startet eine reale Express-Anwendung auf einem zufälligen lokalen Port und sendet HTTP-Anfragen über `fetch`. Damit werden JSON-Middleware, Router, Parser und HTTP-Antwort gemeinsam geprüft. Die bisher abgedeckten Routentests betreffen allerdings nur ignorierte Webhooks, nicht den erfolgreichen asynchronen Process- oder Publish-Pfad.
4. **Szenariobasierte und inhaltliche Bewertung.** Der Mock-Provider und die Prompt-Tests prüfen, ob Tickettext, Repository-Zusammenfassung und beantwortete Rückfragen in die Analyse eingehen. Sie ersetzen jedoch keine Bewertung realer, stochastischer Modellantworten. Für diese Ebene wird deshalb in Abschnitt 6.7 ein Bewertungsraster definiert und der bisherige Nachweisumfang begrenzt ausgewiesen.

Die Abgrenzung zwischen Unit- und Integrationstest ist in diesem Projekt nicht vollständig trennscharf. Ein Test des Formatters ist ein klassischer Unit-Test. Ein Test, der einen echten Express-Server, Parser und Router kombiniert, ist ein Komponenten- oder Integrationstest. Eine vollständige Ende-zu-Ende-Prüfung würde dagegen ein reales GitLab-Projekt, einen echten Webhook, gültige Authentifizierung, den konfigurierten Opencode-Agenten, einen Modellaufruf, Kommentarerstellung, Kontextpersistenz und anschließenden Publish umfassen. Ein solcher Test ist im Repository nicht vorhanden.

6.3 Testumgebung, Durchführung und Gesamtergebnis

Die automatisierte Prüfung wurde am 21.07.2026 unter Windows 11 Pro, 64 Bit, mit Node.js 24.11.0 und npm 11.6.1 durchgeführt. Das Projekt verwendet den in Node.js enthaltenen Test-Runner, der über `tsx -test tests/**/*.test.ts` gestartet wird. Der Build erfolgt mit dem TypeScript-Compiler über `npm run build`.

Die Ausführung ergab folgendes Ergebnis:

Prüfschritt	Ergebnis
<code>npm run build</code>	erfolgreich, keine TypeScript-Fehler
<code>npm test</code>	60 von 60 Tests bestanden
Testsuiten	15 bestanden
Fehlgeschlagene, übersprungene oder offene Tests	0
Laufzeit des dokumentierten Testlaufs	ca. 0,94 Sekunden
Reale GitLab- oder Modellaufrufe	keine; externe Abhängigkeiten wurden simuliert oder umgangen

Die 60 Testfälle verteilen sich wie folgt auf die geprüften Bereiche:

Bereich	Testdateien	Anzahl
GitLab-Parsing, HTTP-Route, Authentifizierung, Cleanup und glab-Hilfen	<code>gitlab.test.ts</code> , <code>webhookRoute.test.ts</code> , <code>auth.test.ts</code> , <code>cleanup.test.ts</code> , <code>glab.test.ts</code>	24
KI-Prompt, Schema, Formatierung und Opencode-Laufzeit	<code>aiPrompt.test.ts</code> , <code>schema.test.ts</code> , <code>formatter.test.ts</code> , <code>aiOpencode.test.ts</code> , <code>opencodeRuntime.test.ts</code>	23
Repository-Kontext, Process- und Publish-Aufbereitung	<code>repositorySummary.test.ts</code> , <code>processor.test.ts</code> , <code>publisher.test.ts</code>	8
Konfiguration und Tunnel-Helfer	<code>config.test.ts</code> , <code>tunnel.test.ts</code>	5
Gesamt	15 Testdateien	60

Das Ergebnis belegt einen stabilen, reproduzierbaren Stand der vorhandenen Tests. Es ist jedoch keine Aussage über eine prozentuale Codeabdeckung, da im Projekt kein Coverage-Werkzeug konfiguriert ist. Insbesondere können 60 erfolgreiche Tests dieselben Pfade mehrfach prüfen, während andere Pfade unberührt bleiben. Deshalb wird im Folgenden nicht die Testanzahl allein, sondern die konkret abgedeckte Funktionalität bewertet.

6.4 Unit- und Komponententests

6.4.1 Webhook-Parsing, Aktivierung und Kommandos

`tests/gitlab.test.ts` prüft Issue- und Note-Payloads. Erfasst werden Projekt-ID, Issue-IID, Titel, Beschreibung oder Kommentar sowie das abgeleitete Kommando. Ein Kommentar mit `@dev-assist publish` wird als Publish-Kommando erkannt. Selbst

verfasste Bot-Kommentare und Kommentare mit bekannten Dev-Assist-Markern werden ignoriert. Diese Tests sind für NFA-7 wichtig, weil sie eine zentrale Ursache für Bot-Endlosschleifen adressieren.

Die Tests machen zugleich eine Abweichung sichtbar. FA-1 fordert, dass der erste inhaltliche Text mit `@dev-assist` beginnt. Die aktuelle Implementierung und die zugehörigen Tests akzeptieren die Mention dagegen auch im Issue-Titel oder an beliebiger Stelle in Beschreibung und Kommentar. Ein Test bestätigt ausdrücklich den Text `Bitte Ticket strukturieren. Danke @dev-assist`; ein weiterer akzeptiert `looks good, @dev-assist publish please`. Damit ist das getestete Verhalten intern konsistent, erfüllt aber die strengere Aktivierungsanforderung nicht. Dieser Befund ist besonders wichtig, weil erfolgreiche Tests andernfalls fälschlich als Nachweis für FA-1 gelesen werden könnten.

Die Routentests prüfen drei negative beziehungsweise ignorierte Fälle über eine reale lokale HTTP-Verbindung: ein Issue ohne Mention, einen Kommentar des Bot-Kontos und einen generierten Dev-Assist-Kommentar. In allen Fällen antwortet die Route schnell mit HTTP 202 und einem maschinenlesbaren Grund. Nicht getestet werden eine erfolgreiche Analyseanforderung, ein erfolgreicher Publish-Aufruf, die asynchrone Fehlerbehandlung oder die Deduplication. Die schnelle Antwortarchitektur aus NFA-8 ist damit für ignorierte Ereignisse belegt, für den positiven Hintergrundprozess aber nur durch Codeinspektion nachvollziehbar.

6.4.2 Webhook-Authentifizierung und Kommentarbereinigung

`tests/auth.test.ts` deckt fünf Fälle ab: Betrieb ohne konfiguriertes Secret, fehlende Signatur bei vorhandenem Secret, gültige zeitgestempelte HMAC, einen direkten HMAC-Fallback und eine ungültige Signatur. Die Verwendung eines konstantzeitlichen Vergleichs wird durch die Implementierung unterstützt. Für den lokal implementierten Signaturvertrag zeigen die Tests ein konsistentes Verhalten.

Die Kompatibilität mit dem aktuellen GitLab-Signing-Token-Verfahren wird dadurch jedoch nicht nachgewiesen. Die aktuelle GitLab-Dokumentation beschreibt die Header `webhook-id`, `webhook-timestamp` und `webhook-signature`, ein Token im Format `whsec_<base64>`, die Nachricht `message_id.timestamp.body` sowie eine Base64-Signatur mit Präfix `v1,`. Außerdem soll die Aktualität des Zeitstempels gegen Replay-Angriffe geprüft werden (vgl. GitLab Docs, o. J.a, Abschnitt "Signing tokens"). Die Implementierung und ihre Tests verwenden dagegen `x-gitlab-signature`, `x-gitlab-timestamp`, eine hexadezimale Signatur und die Nachricht `timestamp:body` beziehungsweise einen direkten Body-HMAC. Die Tests spiegeln somit den eigenen Code, nicht den aktuellen externen Vertrag. NFA-6 ist deshalb für produktive GitLab-Signing-Tokens nicht nachgewiesen. Vor einem produktiven Einsatz sind ein dokumentationskonformer Verifier und Tests mit Payloads aus realen GitLab-Lieferungen erforderlich.

Die sechs Cleanup-Tests prüfen, dass Systemnotes und normale Nutzerkommentare erhalten bleiben, während Kommentare mit `@dev-assist` oder generierten Dev-Assist-Markern löscher sind. Das reduziert das Risiko, beim Publish fachliche Diskussionen unbeabsichtigt zu entfernen. Nicht getestet ist die vollständige Reihenfolge aus Notes

laden, mehrere Kommentare löschen, Einzelfehler tolerieren und anschließend das Issue aktualisieren. Der Testnachweis betrifft den Filter, nicht die reale GitLab-Mutation.

6.4.3 Schema-Validierung und Ausgabeformatierung

Die Schema-Tests gehören zu den stärksten deterministischen Schutzmechanismen. Das aktuelle kompakte Format verlangt exakt die Felder `title`, `description`, `acceptanceCriteria`, `technicalContext`, `proposedSolution` und `openQuestions`. Fehlende Felder, unbekannte Eigenschaften, falsche Feldtypen und nicht textuelle Array-Elemente werden abgelehnt. Außerdem wird JSON innerhalb eines Markdown-Codeblocks korrekt extrahiert. Damit ist belegt, dass beliebiger Freitext oder ein älteres Antwortformat nicht unbemerkt als gültige Analyse weiterverarbeitet wird.

Gleichzeitig besteht eine Abweichung zur Anforderung FA-8 und zur in Kapitel 3 beschriebenen Zielstruktur. Dort werden unter anderem `summary`, `sourceBasis`, ein verschachteltes `implementationTicket`, Risiken und Validierungsschritte gefordert. Der aktuelle Code lehnt dieses ältere, umfangreichere Format ausdrücklich als ungültig ab. Das implementierte Kompaktschema ist strenger validiert als in Kapitel 5 beschrieben, bildet aber weniger fachliche Kategorien ab. Für die Qualitätssicherung bedeutet dies: Die strukturelle Validierung des aktuellen Schemas ist gut nachgewiesen; die Erfüllung der ursprünglichen fachlichen Zielstruktur ist nur teilweise gegeben.

Sieben Formatter-Tests prüfen die vier sichtbaren Abschnitte Description, Acceptance Criteria, Technical Context & Logs und Proposed Solution. Leere Abschnitte erhalten einen eindeutigen Platzhalter. Nummerierungspräfixe werden normalisiert, offene Fragen in den technischen Kontext übernommen und führende Markdown-Syntax wird neutralisiert. Besonders die Tests zu Überschriften, Codezäunen, Blockzitaten und horizontalen Trennlinien verhindern, dass Modelltext die vorgegebene Ticketstruktur unbeabsichtigt aufbricht. Diese Tests belegen die syntaktische Stabilität des gerenderten Markdown. Ob die Inhalte fachlich richtig, vollständig und für Entwicklerinnen und Entwickler hilfreich sind, bleibt davon unberührt.

6.4.4 Prompt, Rückfragen und Opencode-Ausgabe

`tests/aiPrompt.test.ts` prüft sieben Eigenschaften des Analysepfads. Der Prompt fordert ausschließlich das kompakte JSON-Format, die statische Opencode-Anweisung verwendet dieselbe Terminologie, eine Repository-Zusammenfassung wird vor dem Issue-Kontext eingefügt und frühere Dev-Assist-Rückfragen mit späteren Nutzerantworten werden hervorgehoben. Bereits beantwortete oder stark ähnlich umformulierte Fragen werden aus `openQuestions` entfernt. Ein Mock-Szenario prüft außerdem, dass der ursprüngliche Titel erhalten bleibt und keine nicht belegten Implementierungsdetails wie Handler oder Services ergänzt werden.

Diese Tests adressieren ein praktisches Dialogproblem: Rückfragen sollen nicht in Schleifen wiederholt werden. Sie prüfen dafür eine deterministische Ähnlichkeitsheuristik. Nicht abgedeckt sind längere Gesprächsverläufe, widersprüchliche Antworten, mehrsprachige Fragen, Antworten ohne erneute Mention oder Grenzfälle der Ähnlichkeits-

schwelle. Ebenso wird nicht geprüft, ob ein reales Modell trotz Promptanweisung bereits beantwortete Fragen in anderer Form erneut erzeugt.

Der Opencode-Test ersetzt die reale CLI durch ein temporäres Testprogramm. Zunächst liefert der simulierte `opencode run`-Aufruf nur eine Session-ID. Danach stellt `opencode export` eine gültige Analyse bereit. Der Test belegt, dass der Export-Fallback funktioniert und der Prozess ohne unsichere Shell-Verkettung gestartet wird. Weitere Tests prüfen ANSI-Bereinigung, Extraktion verschachtelter Texte, Modellnamen und die Erfassung von Standardausgabe, Fehlerausgabe und Exit-Code. Nicht geprüft werden Timeout und Prozessabbruch, ungültige JSONL-Ereignisse, sehr große Antworten, reale CLI-Versionsunterschiede oder Modellfehler.

6.4.5 Repository-Kontext, Process und Publish

Vier Repository-Summary-Tests verwenden einen simulierten GitLab-Client. Sie prüfen Projektmetadaten, Sprachen, Dateibaum, Schlüsseldateien, Caching sowie die Extraktion von Markdown aus direkter oder JSON-formatierter Opencode-Ausgabe. Zusätzlich wird kontrolliert, dass der Prompt vor der angehängten Kontextdatei an die CLI übergeben wird. Damit ist die Zusammensetzung der Repository-Daten ohne externen Zugriff reproduzierbar.

Die Process- und Publisher-Tests sind enger begrenzt. Ein Process-Test stellt sicher, dass der generierte GitLab-Titel getrennt vom vierteiligen Markdown-Text gespeichert wird. Drei Publisher-Tests prüfen das aktuelle Format sowie die Abwärtskompatibilität zu älteren Kontextdokumenten, bei denen ein eingebetteter Titel entfernt oder aus dem Markdown rekonstruiert wird. Nicht getestet werden das tatsächliche Schreiben und Lesen von `context.md` und `context.json`, fehlende oder beschädigte Kontextdateien im vollständigen Publish-Pfad, GitLab-Fehler beim Löschen oder Aktualisieren und die Garantie, dass ohne vorherigen Process-Schritt keine produktive Änderung erfolgt. Letzteres ergibt sich zwar aus dem Lesezugriff auf die erforderliche Datei, ist aber nicht als Systemtest dokumentiert.

6.5 Simulation ohne reale GitLab- oder Opencode-Abhängigkeit

NFA-2 fordert ausdrücklich, dass Dev-Assist ohne reales GitLab-Projekt und ohne kostenpflichtigen KI-Aufruf testbar ist. Diese Anforderung ist im aktuellen Stand gut umgesetzt. Der Mock-Provider erzeugt eine deterministische Analyse. GitLab-Daten für die Repository-Zusammenfassung werden durch ein lokales Testobjekt bereitgestellt. Opencode wird durch temporäre ausführbare Testprogramme ersetzt. Die HTTP-Route läuft auf einem lokalen, zufällig vergebenen Port. Dadurch benötigt die Testsuite weder Zugangsdaten noch Netzwerkzugriff und kann schnell wiederholt werden.

Die Simulation erfüllt drei Zwecke. Erstens isoliert sie Fehler: Schlägt ein Schema-Test fehl, ist kein schwankendes Modellverhalten die Ursache. Zweitens verhindert sie Kosten und Seiteneffekte, etwa das Erstellen oder Löschen realer GitLab-Kommentare. Drit-

tens verbessert sie die Wiederholbarkeit, weil dieselben Eingaben dieselben Ergebnisse erzeugen.

Eine Simulation kann den externen Vertrag jedoch nur dann zuverlässig vertreten, wenn die Testdoubles dessen Verhalten korrekt abbilden. Genau hier zeigt die Webhook-Signaturprüfung eine Grenze: Ein selbst erzeugter Test-HMAC kann eine intern konsistente, aber gegenüber GitLab falsche Implementierung bestätigen. Auch der simulierte Opencode-Prozess bildet nur die erwarteten Ausgabefälle ab. Änderungen an CLI-Argumenten, JSONL-Ereignissen oder Exportformaten werden erst sichtbar, wenn ein Test gegen eine reale unterstützte Version ausgeführt wird. Die vorhandenen Tests sind deshalb als Komponentenprüfungen zu bewerten, nicht als vollständige Ende-zu-Ende-Nachweise.

6.6 Webhook-Randfälle und Robustheitsbefunde

Die vorhandenen Testfälle decken mehrere für Webhook-Systeme typische Randfälle ab:

Randfall	Erwartetes und getestetes Verhalten	Bewertung
Ereignis ohne Mention	HTTP 202, keine Analyse, Grund <code>no-mention</code>	nachgewiesen
Kommentar des Bot-Kontos	wird als <code>self-authored</code> ignoriert	nachgewiesen
Generierter Dev-Assist-Kommentar	wird anhand von Markern ignoriert	nachgewiesen
Publish-Mention in einem Kommentar	Publish-Kommando wird erkannt	nachgewiesen, aber Aktivierungsregel zu tolerant
Fehlende oder ungültige lokale HMAC	je nach Konfiguration Ablehnung oder Entwicklungsfallback	für den implementierten Vertrag nachgewiesen
Ungültige oder zusätzliche KI-Felder	Schema-Validierung lehnt Antwort ab	nachgewiesen
Leere Ticketabschnitte	sichtbarer Platzhalter statt leerer Struktur	nachgewiesen
Markdown-Steuerzeichen in Modellinhalten	Struktur bleibt auf vier Hauptabschnitte begrenzt	nachgewiesen
Bereits beantwortete Rückfrage	ähnliche offene Frage wird entfernt	für die geprüften Formulierungen nachgewiesen
Opencode-Stream ohne finale Textantwort	Session wird exportiert und Analyse daraus gelesen	nachgewiesen

Randfall	Erwartetes und getestetes Verhalten	Bewertung
Systemnote beim Cleanup	wird nicht gelöscht	nachgewiesen

Mehrere relevante Randfälle bleiben offen. Die In-Memory-Deduplication ist nicht automatisiert getestet. Es gibt keine Tests für zwei gleichzeitige Webhooks, identische Ereignisse ohne Ereignis-ID oder das Verhalten nach Ablauf der TTL. Unvollständige Payloads mit unbekannter Projekt- oder Issue-ID werden zwar defensiv in Strings überführt, aber nicht bis zum Processor verfolgt. Für den positiven Route-Pfad fehlt ein Test, der die schnelle HTTP-Antwort und den nachgelagerten Process-Aufruf gemeinsam beobachtet. Ebenso fehlen Last-, Langzeit- und Ressourcenprüfungen. NFA-3, NFA-7 und NFA-8 sind daher nur teilweise abgesichert.

6.7 Bewertungskriterien für die inhaltliche Ticketqualität

Die automatisierten Tests prüfen vor allem Struktur und Kontrollfluss. Die eigentliche Forschungsfrage betrifft jedoch auch die Nutzbarkeit der erzeugten Tickets. Dafür werden Kriterien benötigt, die über "JSON ist gültig" hinausgehen. Die Forschung aus Kapitel 2 bietet hierfür eine Grundlage.

Bettenburg et al. zeigen, dass Entwicklungsteams unter anderem Reproduktionsschritte, Stack Traces und Testfälle als hilfreich bewerten, während diese Informationen von meldenden Personen häufig nicht bereitgestellt werden (vgl. Bettenburg et al., 2008, S. 308 f.). Breu et al. identifizieren konkrete Informationsbedarfe wie Reproduktionsschritte, Versionen, Betriebssysteme, Beispiele, Ausgaben und Screenshots (vgl. Breu et al., 2010, S. 303). Chaparro et al. heben beobachtetes Verhalten, erwartetes Verhalten und Schritte zur Reproduktion hervor und zeigen, dass gerade diese Bestandteile oft fehlen (vgl. Chaparro et al., 2017, S. 396 f.). Lucassen et al. unterscheiden bei kurzen Anforderungen syntaktische, semantische und pragmatische Qualität (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Aus diesen Arbeiten wird folgendes Bewertungsraster abgeleitet:

Kriterium	Operationalisierung für Dev-Assist	Bewertungsskala
Erkennung fehlender Informationen	Der Vorschlag oder die Rückfrage benennt fehlende Angaben zu Ziel, erwartetem Verhalten, Reproduktion, Randbedingungen oder Akzeptanz.	0 = Lücke übersehen; 1 = allgemein benannt; 2 = konkret und kontextbezogen benannt

Kriterium	Operationalisierung für Dev-Assist	Bewertungsskala
Präzision der Rückfragen	Fragen sind einzeln beantwortbar, beziehen sich auf vorhandenen Kontext und verlangen keine unnötigen internen Implementierungsentscheidungen.	0 = unklar/irrelevant; 1 = teilweise präzise; 2 = präzise und handlungsorientiert
Strukturelle Nutzbarkeit	Titel, Beschreibung, Akzeptanzkriterien, technischer Kontext und Lösungsvorschlag sind getrennt, lesbar und ohne widersprüchliche Duplikate.	0 = unbrauchbar; 1 = nachbearbeitungsbedürftig; 2 = direkt nutzbare Struktur
Prüfbarkeit	Akzeptanzkriterien beschreiben beobachtbare Ergebnisse und relevante Randfälle.	0 = nicht prüfbar; 1 = teilweise prüfbar; 2 = eindeutig prüfbar
Grounding und Unsicherheitskennzeichnung	Der Vorschlag erfindet keine nicht belegten Fakten; Annahmen und offene Fragen bleiben sichtbar.	0 = unbelegte Tatsachen; 1 = gemischt; 2 = konsequent abgesichert
Konsistenz mit dem Kontext	Ticketbeschreibung, Kommentare und gegebenenfalls Repository-Hinweise werden widerspruchsfrei zusammengeführt.	0 = widersprüchlich; 1 = kleinere Inkonsistenzen; 2 = konsistent

Das Raster wäre auf ein vorab festgelegtes Korpus anzuwenden, das mindestens unvollständige Bug Reports, vage Feature-Wünsche, bereits gute Tickets, widersprüchliche Kommentarverläufe und sicherheitskritische Eingaben enthält. Um die Modellvariabilität zu berücksichtigen, müsste jedes Szenario mehrfach und gegebenenfalls mit mehreren Modellkonfigurationen ausgeführt werden. Mindestens zwei fachkundige Bewertende sollten die Ergebnisse unabhängig beurteilen; Unterschiede wären anschließend zu diskutieren. Erst ein solcher Aufbau könnte belastbar beantworten, wie häufig Dev-Assist fehlende Informationen erkennt oder direkt nutzbare Vorschläge erzeugt.

Der aktuelle Teststand liefert dafür nur Teilindikatoren. Der Formatter erzwingt eine stabile Struktur. Das Kompaktschema verhindert freie Zusatzfelder. Das Mock-Szenario vermeidet bestimmte erfundene Implementierungsdetails. Beantwortete Fragen werden nicht erneut ausgegeben. Diese Befunde unterstützen syntaktische Qualität und einzelne Grounding-Regeln. Die semantische und pragmatische Qualität realer Opencode-Modellantworten wurde jedoch weder an einem Ticketkorpus noch durch Entwicklerinnen und Entwickler bewertet. Die Fragen "Erkennt der Assistent fehlende Informationen?", "Sind Rückfragen präzise?" und "Sind Vorschläge unmittelbar nutzbar?" können daher noch nicht allgemein bejaht werden.

NIST weist darauf hin, dass generative KI überzeugend formulierte, aber falsche oder irreführende Inhalte erzeugen kann (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Schema-Validierung reduziert dieses Risiko nicht inhaltlich: Eine Halluzination kann syntaktisch korrekt in einem String stehen. Der kontrollierte

Publish-Schritt bleibt deshalb auch bei guter Testabdeckung notwendig. Er ist kein Ersatz für inhaltliche Evaluation, aber eine wirksame Begrenzung möglicher Folgen.

6.8 Nachverfolgbarkeit der funktionalen Anforderungen

Der Abgleich mit den funktionalen Anforderungen aus Kapitel 3 ergibt ein differenziertes Bild:

Anforderung	Vorhandene Evidenz	Ergebnis
FA-1 Explizite Aktivierung am Textanfang	Parser- und Routentests	Abweichung: Mention wird auch im Titel und mitten im Text akzeptiert.
FA-2 Verarbeitung relevanter GitLab-Ereignisse	Issue-/Note-Parser und lokale Routentests	Teilweise nachgewiesen: Payload-Grundformen sind abgedeckt, aber keine reale GitLab-Lieferung und kein positiver Ende-zu-Ende-Pfad.
FA-3 Analyse-/Publish-Kommando	Parser-Tests für normale Aktivierung und Publish	Nachgewiesen für die getesteten Textformen; Position der Mention bleibt zu tolerant.
FA-4 Abruf des Ticketkontexts	Implementierung und simulierte Repository-Daten	Teilweise nachgewiesen: GitLab-Abruf im Processor ist nicht als Integrationstest abgedeckt.
FA-5 Gemeinsame Analyse von Beschreibung und Kommentaren	Prompt-Tests mit Issue, Kommentaren, Antworten und Repository-Summary	Strukturell nachgewiesen, semantische Zusammenführung durch ein reales Modell nicht bewertet.
FA-6 Gezielte Rückfragen	Formatter- und Clarification-Tests	Teilweise nachgewiesen: Darstellung und Wiederholungsfiler funktionieren; Präzision realer Modellfragen ist offen.
FA-7 Strukturierte Ticketvorschläge	Formatter-, Mock- und Publisher-Hilfstests	Teilweise nachgewiesen: vierteilige Kompaktstruktur ist stabil, aber umfangreichere Zielstruktur und fachliche Nutzbarkeit sind nicht vollständig belegt.
FA-8 Maschinenlesbares Analyseformat	sechs Schema-Tests	Nachgewiesen für das aktuelle Kompaktschema, zugleich Abweichung von der in Kapitel 3 geforderten umfangreicheren Struktur.

Anforderung	Vorhandene Evidenz	Ergebnis
FA-9 Veröffentlichung als Kommentar	Formatter und Implementierung von <code>createNote</code>	Teilweise nachgewiesen: Kommentartext ist getestet, reale GitLab-Veröffentlichung nicht.
FA-10 Persistenz des Kontexts	Implementierung, Trennung von Titel und Markdown	Teilweise nachgewiesen: Dateischreib- und Lese_pfad besitzt keinen eigenen automatisierten Integrationstest.
FA-11 Kontrollierter Publish	Cleanup- und Publisher-Hilfstests	Teilweise nachgewiesen: Aufbereitung und Filter sind geprüft, vollständige Mutation eines realen oder simulierten GitLab-Clients nicht.

Der wichtigste positive Befund ist die klare Validierungs- und Freigabegrenze: Ein aktuelles Analyseobjekt muss exakt dem Kompaktschema entsprechen, und der Titel wird getrennt von der Beschreibung für den Publish vorbereitet. Der wichtigste funktionale Widerspruch betrifft FA-1. Hinzu kommt der Schema-Drift zwischen Zielanforderung und aktuellem Code. Beide Punkte sollten entweder implementierungsseitig korrigiert oder in den vorherigen Kapiteln als bewusst geänderte Anforderungen begründet werden.

6.9 Nachverfolgbarkeit der nicht-funktionalen Anforderungen

Auch die nicht-funktionalen Anforderungen sind unterschiedlich stark abgesichert:

Anforderung	Bewertung
NFA-1 Wartbarkeit	Die Modulgrenzen sind im Quellcode klar und der Build ist erfolgreich. Eine eigenständige Wartbarkeitsmessung wurde nicht durchgeführt.
NFA-2 Testbarkeit ohne externe Abhängigkeiten	Gut nachgewiesen: alle 60 Tests laufen ohne reales GitLab und ohne realen Modellaufruf.
NFA-3 Robustheit	Teilweise nachgewiesen: ungültiges JSON, schwache Inhalte und einige Prozessvarianten sind abgedeckt; externe Fehlerketten, beschädigte Dateien und Parallelität fehlen.
NFA-4 Logging	Umfangreiche Konsolenlogs sind implementiert, ihre Vollständigkeit und Secret-Freiheit werden nicht automatisiert geprüft.
NFA-5 Begrenzung von KI-Risiken	Schema, Promptregeln, Grounding im Mock und expliziter Publish begrenzen Folgen. Prompt-Injection- und Halluzinationsresistenz realer Modelle sind nicht getestet.
NFA-6 Webhook-Sicherheit	Nicht für das aktuelle Signing-Token-Verfahren nachgewiesen: lokaler HMAC-Vertrag weicht von der aktuellen GitLab-Dokumentation ab; Zeitstempelfrische wird nicht geprüft.
NFA-7 Bot-Schleifen und Duplikate	Bot- und Markerfilter sind nachgewiesen; Deduplication und Parallelfälle sind ungetestet.
NFA-8 Schnelle Webhook-Antwort	HTTP 202 ist für ignorierte Ereignisse nachgewiesen; positiver asynchroner Pfad und Lastverhalten sind nicht gemessen.
NFA-9 Konfigurierbarkeit	Einzelne Variablen für Tunnel und Bot-Namen sind getestet; die Gesamtheit der Konfiguration und ungültige Werte sind nicht abgedeckt.
NFA-10 Prozessintegrierte Bedienbarkeit	Die Bedienung ist ohne eigene GUI über GitLab vorgesehen. Eine Usability- oder Feldbewertung mit Nutzenden fehlt.

Insgesamt ist NFA-2 am stärksten belegt. Das Projekt lässt sich schnell und ohne Zugangsdaten testen. Bei Sicherheit, Robustheit unter externen Fehlern und Bedienbarkeit ist der Nachweis deutlich schwächer. Diese Bereiche sollten vor einem produktiven Einsatz priorisiert werden.

6.10 Grenzen und Bedrohungen der Aussagekraft

Easterbrook et al. unterscheiden unter anderem Konstruktvalidität, interne Validität, externe Validität und Reliabilität und empfehlen, Schwächen eines Untersuchungsdesigns explizit offenzulegen (vgl. Easterbrook et al., 2008, S. 302 f.). Auf die vorliegende Evaluation übertragen ergeben sich folgende Grenzen.

Konstruktvalidität. Die Zahl bestandener Tests ist kein direktes Maß für Ticketqualität. Die Tests operationalisieren vor allem technische Korrektheit, etwa Schemaform, Textformat oder Parserentscheidungen. Begriffe wie "präzise Rückfrage" oder "entwicklergerechtes Ticket" werden erst durch das Raster in Abschnitt 6.7 konkretisiert, aber noch nicht empirisch erhoben. Auch der erfolgreiche Build misst nur statische Konsistenz, nicht Laufzeitrobustheit.

Interne Validität. Tests und Implementierung stammen aus demselben Projektkontext. Dadurch können Testdoubles dieselben falschen Annahmen wie der Produktivcode enthalten. Die Signaturprüfung ist ein konkretes Beispiel: Die Tests erzeugen genau das Format, das die Implementierung erwartet, und erkennen deshalb die Abweichung zum aktuellen GitLab-Vertrag nicht. Außerdem können globale Umgebungsvariablen und Modulinitialisierung Testergebnisse beeinflussen, auch wenn die vorhandenen Tests ihre Änderungen überwiegend zurücksetzen.

Externe Validität. Der Testlauf betrifft ein einzelnes TypeScript-Projekt auf einer lokalen Windows-Umgebung. Es wurden weder unterschiedliche GitLab-Versionen noch selbstverwaltete Instanzen, reale Netzwerkfehler, verschiedene Opencode-Versionen oder unterschiedliche Modelle geprüft. Es gibt kein repräsentatives Ticketkorpus und keine Untersuchung mit Entwicklungsteams. Aussagen über andere Projekte, Organisationen, Sprachen oder Domänen sind daher nicht generalisierbar.

Reliabilität. Der deterministische Teil ist durch feste Testdaten, Mock-Objekte und dokumentierte Befehle gut wiederholbar. Der nichtdeterministische Kern wurde aus der Testsuite ausgeklammert. Das erhöht die Reproduzierbarkeit der technischen Tests, verhindert aber Aussagen über Streuung und Stabilität realer Modellantworten. Für eine spätere Replikation müssten Modell, Version, Prompt, Parameter, Zeitstempel, Ticketkorpus und Bewertungsprozess festgehalten werden.

Weitere technische Grenzen sind das Fehlen einer Coverage-Messung, fehlende Last- und Parallelitätstests, keine automatisierte Sicherheitsanalyse, keine Tests des Dateisystems unter Fehlerbedingungen und keine vollständige GitLab-Ende-zu-Ende-Prüfung. Easterbrook et al. weisen allgemein darauf hin, dass empirische Ergebnisse keine sichere Gewissheit erzeugen und durch ergänzende Methoden und Replikationen belastbarer werden (vgl. Easterbrook et al., 2008, S. 305). Für Dev-Assist wären insbesondere ein dokumentationskonformer GitLab-Vertragstest, ein kleines kuratiertes Ticketkorpus, wiederholte reale Modellläufe und eine Bewertung durch mehrere Entwicklerinnen und Entwickler sinnvolle Ergänzungen.

6.11 Zwischenfazit

Die Qualitätssicherung zeigt einen technisch stabilen, aber noch nicht vollständig validierten Prototyp. Der TypeScript-Build ist erfolgreich, und alle 60 automatisierten Tests in 15 Suiten bestehen. Besonders gut abgesichert sind das aktuelle kompakte JSON-Schema, die vierteilige Markdown-Ausgabe, die Erkennung eigener beziehungsweise generierter Kommentare, die Kommentarfilterung, die Verarbeitung beantworteter Rückfragen, Repository-Zusammenfassungen mit simulierten Abhängigkeiten und der Opencode-Export-Fallback. NFA-2, die lokale Testbarkeit ohne reale GitLab- oder KI-Abhängigkeit, wird damit überzeugend erfüllt.

Die Evaluation macht zugleich wesentliche Abweichungen sichtbar. Die Mention-Erkennung ist breiter als FA-1 und akzeptiert Vorkommen im Titel oder mitten im Text. Das aktuelle Kompaktschema weicht von der umfangreicheren Zielstruktur aus Kapitel 3 ab. Die Webhook-Signaturtests bestätigen einen lokalen HMAC-Vertrag, nicht das aktuelle GitLab-Signing-Token-Verfahren. Positive Ende-zu-Ende-Pfade für Process, Persistenz, Kommentarerstellung und Publish fehlen. Vor allem wurde die semantische Qualität realer Modellantworten noch nicht an einem Korpus oder mit Entwicklerinnen und Entwicklern bewertet.

Damit kann für den untersuchten Stand festgehalten werden: Die deterministischen Grenzen um die KI-Komponente sind in wichtigen Teilen testbar und stabil. Ein allgemeiner Nachweis, dass Dev-Assist fehlende Informationen zuverlässig erkennt, stets präzise Rückfragen stellt und unmittelbar nutzbare Ticketvorschläge erzeugt, liegt noch nicht vor. Für einen produktionsnahen nächsten Schritt sind die Korrektur und Prüfung des GitLab-Signaturverfahrens, die Entscheidung über die strikte Mention-Regel, ein konsistentes Zielschema, vollständige Process-/Publish-Integrationstests sowie eine wiederholte inhaltliche Evaluation mit realen Modellläufen erforderlich.

Quellen zu Kapitel 6

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, S. 308-318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports. Improving Cooperation Between Developers and Users“, in Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work, S. 301-310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, S. 396-407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest; Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285-311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

GitLab Docs o. J.a. Webhooks. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 21.07.2026).

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving agile requirements. The Quality User Story framework and tool“, in *Requirements Engineering* 21, 3, S. 383-403. <https://doi.org/10.1007/s00766-016-0250-x>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework. Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

Wohlin, Claes; Runeson, Per; Höst, Martin; Ohlsson, Magnus C.; Regnell, Björn; Wesslén, Anders 2012. *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer. <https://doi.org/10.1007/978-3-642-29044-2>.

Projektinterne Arbeitsgrundlagen: AGENTS.md, README.md, package.json, src/config.ts, src/routes/gitlabWebhooks.ts, src/services/gitlab/parser.ts, src/services/gitlab/mention.ts, src/services/gitlab/auth.ts, src/services/gitlab/cleanup.ts, src/services/gitlab/glab.ts, src/services/ai/instructions.ts, src/services/ai/service.ts, src/services/ai/schema.ts, src/services/ai/formatter.ts, src/services/ai/clarifications.ts, src/services/ai/opencodeRuntime.ts, src/services/repositorySummary.ts, src/services/processing/processor.ts, src/services/processing/publisher.ts, src/services/context/writer.ts, src/services/context/reader.ts, src/services/tunnel.ts, tests/aiOpencode.test.ts, tests/aiPrompt.test.ts, tests/auth.test.ts, tests/cleanup.test.ts, tests/config.test.ts, tests/formatter.test.ts, tests/gitlab.test.ts, tests/glab.test.ts, tests/opencode

Runtime.test.ts, tests/processor.test.ts, tests/publisher.test.ts, tests/repositorySummary.test.ts, tests/schema.test.ts, tests/tunnel.test.ts und tests/webhookRoute.test.ts.

7 Ergebnisse

Dieses Kapitel fasst die Ergebnisse der Konzeption, Implementierung und Qualitätssicherung des Dev-Assist-Prototyps zusammen. Im Mittelpunkt steht nicht mehr die Herleitung einzelner Architektur- oder Implementierungsentscheidungen, sondern der am Ende der Entwicklung tatsächlich vorliegende Funktionsumfang. Dazu werden der finale Prototyp beschrieben, ein beispielhafter Ticketablauf nachvollzogen und der Umsetzungsstand mit den funktionalen sowie nicht-funktionalen Anforderungen aus Kapitel 3 abgeglichen.

Als Ergebnis ist dabei zweierlei zu unterscheiden. Erstens liegt ein ausführbarer TypeScript-Dienst vor, der GitLab-Ereignisse entgegennimmt, Ticketkontext für eine KI-Analyse aufbereitet, Antworten strukturell validiert, Rückfragen oder Vorschläge veröffentlicht und freigegebene Inhalte in ein Issue übernimmt. Zweitens liegt ein begrenzter Nachweis über dieses System vor. Die automatisierten Tests und der erfolgreiche Build zeigen, dass zentrale deterministische Komponenten für die geprüften Fälle funktionieren. Sie zeigen jedoch nicht, dass reale Modellantworten in beliebigen Projekten immer fachlich richtig oder unmittelbar nutzbar sind. Kapitel 7 beschreibt daher sowohl das erzielte Systemergebnis als auch den jeweils erreichten Nachweisstand.

7.1 Ergebnisrahmen

Der finale Repository-Stand vom 21.07.2026 bildet einen durchgängigen prototypischen Workflow ab. GitLab kann den Dienst über Webhooks über Issue- und Kommentaireignisse informieren; solche Webhooks sind als HTTP-basierte Benachrichtigungen externer Systeme vorgesehen (vgl. GitLab Docs, o. J.a, Abschnitt "Webhook events"). Dev-Assist prüft anschließend, ob eine Aktivierung vorliegt, lädt zusätzlichen Kontext, ruft abhängig von der Konfiguration den Mock- oder Opencode-Pfad auf und erzeugt ein maschinenlesbares Analyseobjekt. Aus diesem Objekt entsteht entweder ein Rückfragekommentar oder ein strukturierter Ticketvorschlag. Nach einer ausdrücklichen Publish-Anweisung liest das System den gespeicherten Vorschlag und aktualisiert das GitLab-Issue.

Das Ergebnis lässt sich in vier Ebenen gliedern:

1. **Ausführbarer Dienst.** Eine Express-Anwendung stellt einen Health-Endpunkt, einen GitLab-Webhook-Endpunkt und manuelle Process- und Publish-Endpunkte bereit.
2. **Kontrollierter Analyseablauf.** GitLab-Daten, Kommentare und optionaler Repository-Kontext werden für den Analyseprovider zusammengeführt; dessen Ausgabe muss einem festen Schema entsprechen.

3. **Nachvollziehbare Ergebnisartefakte.** Vorschläge erscheinen als GitLab-Kommentar und werden zusätzlich als `context.md` sowie mit Titelmetadaten in `context.json` gespeichert.
4. **Explizite Freigabegrenze.** Die KI-Komponente verändert das Issue nicht unmittelbar. Erst `@dev-assist publish` beziehungsweise ein manueller Publish-Aufruf löst die produktive Aktualisierung aus.

Diese Ebenen entsprechen dem Kernziel der Arbeit: Dev-Assist ist als Assistenzsystem zur Aufbereitung von Ticketinformationen umgesetzt, nicht als autonomer Entwicklungsagent. Das System erzeugt Anforderungen, Rückfragen und Vorschläge, führt aber keine Implementierung im Zielrepository aus und veröffentlicht Änderungen erst nach einem gesonderten Kommando.

7.2 Beschreibung des finalen Prototyps

Der Prototyp ist als reine Serveranwendung ohne eigene grafische Oberfläche realisiert. GitLab bleibt die Benutzeroberfläche und der zentrale Arbeitsort. Dadurch werden keine zusätzlichen Formulare oder Dashboards benötigt; Aktivierung, Rückfragen, Antworten, Vorschau und Freigabe finden im vorhandenen Issue und dessen Kommentarverlauf statt.

Die wesentlichen Ergebnisbausteine sind:

Baustein	Realisiertes Ergebnis	Zentrale Artefakte
HTTP- und Routing-Schicht	Express-Dienst mit Health-, Webhook-, Process- und Publish-Routen	<code>src/server.ts</code> , <code>src/app.ts</code> , <code>src/routes/</code>
Aktivierungs- und Ereignisschicht	Parsing von Issue- und Note-Payloads, Mention- und Kommandoerkennung, Bot-Filter und In-Memory-Deduplication	<code>src/services/gitlab/parser.ts</code> , <code>mention.ts</code> , <code>commands.ts</code> , <code>cleanup.ts</code>
GitLab-Anbindung	Lesen von Issues und Notes, Erstellen und Löschen von Kommentaren, Aktualisieren von Issues; primär über <code>glab</code> , alternativ über REST	<code>src/services/gitlab/client.ts</code> , <code>glab.ts</code>
Analyse- und Agentenschicht	Mock-Modus oder Opencode-Aufruf, Promptaufbau, Laufzeitparsing und strikte Prüfung eines kompakten JSON-Schemas	<code>src/services/ai/</code> und <code>opencode.json</code>
Repository-Kontext	Abruf von Projektmetadaten, Sprachen, Dateibaum und ausgewählten Schlüsseldateien; Zusammenfassung und Cache pro Projekt	<code>src/services/repositorySummary.ts</code>

Baustein	Realisiertes Ergebnis	Zentrale Artefakte
Ergebnis- und Persistenzschicht	Rückfrage- oder Vorschlagskommentar, vierteilige Markdown-Struktur, <code>context.md</code> und <code>context.json</code>	<code>src/services/ai/formatter.ts</code> , <code>src/services/context/</code>
Prozess- und Publish-Schicht	Orchestrierung der Analyse, Kommentarbereinigung und kontrollierte Aktualisierung von Titel und Beschreibung	<code>src/services/processing/processor.ts</code> , <code>publisher.ts</code>

7.2.1 Schnittstellen und Aktivierung

Der reguläre Einstieg erfolgt über `POST/webhooks/gitlab/issues`. Die Route erfasst den unveränderten Request-Body für die Signaturprüfung, überführt die Payload in ein internes Format und antwortet bei akzeptierten Ereignissen mit HTTP 202. Die eigentliche Analyse oder Veröffentlichung läuft anschließend asynchron weiter. Für lokale Entwicklung und gezielte Wiederholungen stehen zusätzlich `POST/api/issues/:projectId/:issueId/process` und `POST/api/issues/:projectId/:issueId/publish` bereit.

Das interne Webhook-Objekt enthält Ereignisart, Projekt-ID, Issue-IID, Titel, Beschreibung beziehungsweise Kommentartext, Aktion, Kommando und Verarbeitungsentscheidung. Issue- und Note-Ereignisse werden auf unterschiedliche Felder der GitLab-Payload abgebildet. Kommentare des konfigurierten Bot-Kontos und Inhalte mit bekannten Dev-Assist-Markern werden ignoriert. Eine zeitlich begrenzte In-Memory-Deduplication soll außerdem verhindern, dass identische Ereignisse innerhalb eines kurzen Fensters mehrfach verarbeitet werden.

Die aktuelle Aktivierungslogik erkennt `@dev-assist` im Titel, in der Beschreibung oder im auslösenden Kommentar. Das ist funktional breiter als die in Kapitel 3 formulierte Regel, nach der die Mention am Anfang der ersten Inhaltszeile stehen soll. Für den finalen Prototyp bedeutet dies: Eine explizite Mention bleibt erforderlich, ihre Position ist jedoch weniger streng begrenzt als im ursprünglichen Zielbild.

7.2.2 Kontextaufbereitung und Analyse

Nach einem Process-Kommando versucht der Processor, das aktuelle Issue und den Kommentarverlauf über den GitLab-Client abzurufen. Die GitLab Issues API stellt Issue-Daten bereit, während die Notes API den Zugriff auf Issue-Kommentare ermöglicht (vgl. GitLab Docs, o. J.c, o. S.; GitLab Docs, o. J.d, o. S.). Schlägt dieser Abruf fehl, wird mit den Daten aus der Webhook-Payload weitergearbeitet. Der Fehler führt damit nicht automatisch zum Abbruch, verringert aber den verfügbaren Kontext.

Im Opencode-Modus wird zusätzlich eine Repository-Zusammenfassung erzeugt. Dafür sammelt der Prototyp Projektname, Beschreibung, Standardbranch, Programmiersprachen, bis zu 300 Einträge des Dateibaums und höchstens acht ausgewählte Schlüssel-

dateien mit jeweils maximal 8000 Zeichen. Ein eigener **repo-summary**-Agent verdichtet diese Daten. Die Zusammenfassung wird pro Projekt im Speicher gehalten und als `.dev-assist/repo-summary-<projectId>.md` persistiert. Sie ergänzt technische Hinweise, ersetzt aber nicht die fachliche Anforderung aus dem Ticket.

Der Analyseprompt kombiniert die zentral gepflegten Regeln mit Titel und Beschreibung des Issues, optionaler Repository-Zusammenfassung, erkannten Antworten auf frühere Dev-Assist-Rückfragen und bis zu sechs jüngsten Kommentaren. Für reale Analysen startet der Dienst die Opencode-CLI mit dem Agenten **dev-assist-analyzer**; der Agent und das Modell werden über `opencode.json` beziehungsweise Umgebungsvariablen konfiguriert. Opencode stellt Agents und CLI-Aufrufe als konfigurierbare Schnittstelle für spezialisierte Arbeitsabläufe bereit (vgl. OpenCode Docs, o. J.a, Abschnitt "Agents"; OpenCode Docs, o. J.d, o. S.). Für lokale Tests erzeugt der Mock-Provider dagegen eine deterministische Beispiellantwort ohne externen Modellaufruf.

Unabhängig vom Provider akzeptiert die Anwendung nur ein Analyseobjekt mit exakt sechs Feldern:

- `title` als Text,
- `description` als Textliste,
- `acceptanceCriteria` als Textliste,
- `technicalContext` als Textliste,
- `proposedSolution` als Textliste und
- `openQuestions` als Textliste.

Fehlende oder zusätzliche Felder, falsche Feldtypen und nicht textuelle Listenelemente führen zu einer Ablehnung. Die Opencode-Ausgabe kann direkt, aus JSONL-Ereignissen oder über einen Session-Export gelesen werden. In allen Fällen muss am Ende dasselbe Schema erfüllt sein. Damit besitzt der finale Prototyp eine klare technische Grenze zwischen probabilistischer Textgenerierung und deterministischer Weiterverarbeitung.

7.2.3 Rückfragen, Vorschläge und Kontextdateien

Nach der Analyse zählt der Processor die noch vorhandenen substanziellen offenen Fragen. Ab zwei Fragen wird ein Klärungskommentar erzeugt. Dieser erklärt, dass noch keine belastbare Ticketfassung vorliegt, und bittet um Antworten zu Anforderungen, Nutzen, Umfang, Akzeptanzkriterien oder Erfolgskriterien. Bei höchstens einer offenen Frage veröffentlicht Dev-Assist unmittelbar einen vollständigen strukturierten Vorschlag. Die verbleibende Frage wird im technischen Kontext als offener Punkt sichtbar gehalten.

Der sichtbare Vorschlag besitzt vier feste Abschnitte:

1. **Description** beschreibt Ziel und vorhandenen Kontext.

2. **Acceptance Criteria** führt beobachtbare Prüfpunkte auf.
3. **Technical Context & Logs** enthält belegte technische Randbedingungen und verbleibende offene Fragen.
4. **Proposed Solution** beschreibt einen groben, handlungsorientierten Lösungsweg.

Leere Abschnitte werden nicht stillschweigend entfernt, sondern mit `Not enough information available yet.` gekennzeichnet. Nummerierungspräfixe werden normalisiert und führende Markdown-Konstrukte neutralisiert, damit Modelltext die vorgegebene Abschnittsstruktur nicht unbeabsichtigt verändert. Der vorgeschlagene Titel wird nicht in den Beschreibungstext eingebettet, sondern separat als Metadatum behandelt.

Unabhängig davon, ob der GitLab-Kommentar erfolgreich angelegt werden kann, schreibt der Processor die gerenderte Fassung nach `.dev-assist/issues/<projectId>/<issueId>/context.md`. Der Titel wird zusätzlich in `context.json` gespeichert. Diese Dateien bilden eine Brücke zu nachgelagerten Entwicklungswerkzeugen: Ein späterer Agent oder ein manueller Entwicklungsschritt kann den strukturierten Kontext verwenden, ohne den vollständigen GitLab-Kommentarverlauf erneut rekonstruieren zu müssen.

7.2.4 Kontrollierter Publish-Schritt

Das Publish-Kommando ist als eigene Prozessphase umgesetzt. Der Publisher liest zunächst `context.md` und, sofern vorhanden, `context.json`. Danach lädt er den aktuellen Kommentarverlauf, bestimmt anhand des Cleanup-Filters die Dev-Assist-bezogenen Kommentare und versucht, sie einzeln zu löschen. Systemnotes und normale Nutzerkommentare ohne Dev-Assist-Bezug bleiben erhalten. Einzelne Löschfehler werden protokolliert, blockieren die anschließende Issue-Aktualisierung aber nicht.

Aus den Kontextdateien werden der neue Issue-Titel und die neue Beschreibung vorbereitet. Der Titel stammt bevorzugt aus `context.json`; ältere Kontextformate können einen eingebetteten Titel aus Markdown übernehmen. Die Beschreibung enthält nur die vier strukturierten Abschnitte. Anschließend aktualisiert der GitLab-Client das Issue. Manuelle Publish-Aufrufe können zusätzlich Felder wie Status, Labels oder Zuständigkeiten übergeben.

Das resultierende Kontrollmodell besteht somit aus zwei getrennten Schreibvorgängen. Die Analyse darf einen Kommentar und lokale Kontextdateien erzeugen. Die eigentliche Änderung von Titel und Beschreibung erfolgt erst nach einer zweiten, expliziten Aktion. Dadurch bleibt der Vorschlag vor der Übernahme sichtbar und überprüfbar.

7.3 Beispielhafter Ablauf

Der folgende Ablauf zeigt das Verhalten anhand eines fiktiven Issues. Er dient als nachvollziehbares Anwendungsszenario des implementierten Kontrollflusses und nicht

als Messung der Qualität eines bestimmten Sprachmodells.

7.3.1 Erstellung und Aktivierung des Issues

Eine Nutzerin erstellt in Projekt 123 das Issue 42 mit dem Titel Passkey-Login ergänzen und der Beschreibung:

```
@dev-assist Nutzende sollen sich künftig mit Passkeys  
anmelden können. Derzeit stehen Passwort und 2FA zur  
Verfügung.
```

GitLab sendet ein Issue-Ereignis an den Webhook-Endpunkt. Der Parser liest Projekt-ID, Issue-IID, Titel und Beschreibung. Die Mention wird erkannt, das Kommando mangels `publish` als `process` eingeordnet und die Verarbeitung asynchron gestartet. Die HTTP-Antwort bestätigt mit Status 202 lediglich die Annahme des Ereignisses; sie enthält noch kein Analyseergebnis.

7.3.2 Kontextabruf und erste Analyse

Der Processor versucht, das vollständige Issue und die bisherigen Kommentare zu laden. Im Opencode-Modus wird außerdem die Repository-Zusammenfassung des Projekts ermittelt oder aus dem Cache übernommen. Der Agent erhält damit die ursprüngliche Anforderung, vorhandene Diskussionen, bereits beantwortete Rückfragen und gegebenenfalls technische Projekthinweise.

Für das Beispiel seien nach der ersten Analyse zwei fachlich relevante Punkte offen:

- Soll die Passkey-Anmeldung die vorhandenen Verfahren ergänzen oder ersetzen?
- Für welche Nutzergruppen und Oberflächen gilt die Anforderung?

Da mindestens zwei substanzielle Fragen verbleiben, erzeugt der Processor noch keinen freizugebenden Ticketvorschlag. Stattdessen erscheint ein Dev-Assist-Kommentar mit den Fragen und dem Hinweis, bei der Antwort erneut `@dev-assist` zu verwenden.

7.3.3 Antwort und erneute Analyse

Die Nutzerin antwortet:

```
@dev-assist Passkeys ergänzen Passwort und 2FA. Die Funktion  
gilt in der Webanwendung für alle registrierten Nutzenden.  
Wenn kein Passkey verfügbar ist, bleiben die vorhandenen  
Anmelde- und Wiederherstellungsverfahren nutzbar.
```

Das Note-Ereignis startet einen neuen Process-Lauf. Bei der Promptaufbereitung wird die frühere Dev-Assist-Frage zusammen mit der späteren Nutzerantwort als geklärt

Kontext hervorgehoben. Der Nachbearbeitungsschritt entfernt offene Fragen, die bereits beantwortet wurden oder einer beantworteten Frage stark ähneln. Dadurch soll vermieden werden, dass dieselbe Klärungsschleife erneut beginnt.

7.3.4 Strukturierter Vorschlag

Auf Basis der beschriebenen Angaben könnte das resultierende Artefakt inhaltlich wie folgt aussehen. Die Form entspricht dem implementierten Formatter; die Beispielinhalte illustrieren lediglich das Szenario:

Vorgeschlagener Titel: Passkey-Anmeldung als zusätzliche Login-Methode anbieten

Description

Registrierte Nutzende sollen sich in der Webanwendung mit einem vorhandenen Passkey anmelden können. Passkeys ergänzen die bestehenden Verfahren mit Passwort und 2FA; diese bleiben weiterhin verfügbar. Wenn kein verwendbarer Passkey vorhanden ist, können die bestehenden Anmelde- und Wiederherstellungswege genutzt werden.

Acceptance Criteria

- Registrierte Nutzende mit einem verwendbaren Passkey können sich damit erfolgreich anmelden.
- Passwort- und 2FA-Anmeldung bleiben unverändert nutzbar.
- Ohne verfügbaren Passkey bleiben die bestehenden Wiederherstellungsverfahren erreichbar.

Technical Context & Logs

- Die Änderung betrifft die Webanwendung.
- Die konkrete Browser- und Geräteunterstützung ist noch zu bestätigen.

Proposed Solution

1. Den bestehenden Anmeldeablauf um eine Passkey-Option ergänzen.
2. Die bisherigen Anmelde- und Wiederherstellungswege als Alternativen erhalten.
3. Den Ablauf gegen die bestätigten Akzeptanzkriterien und relevante Randfälle prüfen.

Der Vorschlag wird vollständig als GitLab-Kommentar angezeigt und parallel in `context.md` gespeichert. `context.json` enthält den vorgeschlagenen Titel. Die noch offene Browser- und Geräteunterstützung bleibt erkennbar, verhindert bei nur einem offenen Punkt aber nach der implementierten Heuristik nicht die Ausgabe eines Vorschlags.

7.3.5 Freigabe und Übernahme

Nach Prüfung des Kommentars antwortet eine berechtigte Person mit `@dev-assist publish`. Der Parser erkennt das Publish-Kommando. Der Publisher liest den zuvor gespeicherten Kontext, entfernt die Dev-Assist-bezogenen Kommentare soweit möglich und aktualisiert anschließend Titel und Beschreibung des Issues. Das Ergebnis ist kein zusätzlicher Freitext unterhalb der ursprünglichen Beschreibung, sondern ein neu strukturierter Ticketstand mit getrenntem Titel, Beschreibung, Akzeptanzkriterien, technischem Kontext und Lösungsvorschlag.

Der Ablauf erzeugt damit folgende Zustandsfolge:

Phase	Auslöser	Sichtbares Ergebnis	Persistiertes Ergebnis
Ausgangszustand	Issue oder Kommentar mit <code>@dev-assist</code>	ursprünglicher Tickettext	noch kein Kontextartefakt
Klärung	mindestens zwei offene Fragen	Rückfragekommentar	vorläufige strukturierte Analyse
Vorschlag	ausreichender Kontext oder höchstens eine offene Frage	vollständiger Vorschlagskommentar	<code>context.md</code> und <code>context.json</code>
Publish	<code>@dev-assist publish</code>	aktualisierter Titel und strukturierte Beschreibung	derselbe freigegebene Kontext bleibt lokal verfügbar

7.4 Ergebnisartefakte und Übergabepunkte

Der Prototyp erzeugt nicht nur einen einzelnen Kommentar, sondern mehrere aufeinander abgestimmte Artefakte. Der GitLab-Kommentar ist die Vorschau- und Kommunikationsform. Er ermöglicht eine menschliche Prüfung im bestehenden Arbeitskontext. `context.md` ist die menschenlesbare und zugleich werkzeugfreundliche Übergabeform für nachgelagerte Arbeit. `context.json` trennt Metadaten, insbesondere den Titel, von der Beschreibung. Die Repository-Zusammenfassung liefert optional einen wiederverwendbaren technischen Kontext auf Projektebene. Konsolenlogs dokumentieren schließlich die Verarbeitungsschritte und Fehlerzustände.

Diese Trennung verhindert, dass ein einziges Format alle Aufgaben gleichzeitig erfüllen muss. Ein Kommentar ist für Diskussion und Freigabe geeignet, aber nicht ideal als dauerhafte maschinelle Schnittstelle. Ein JSON-Analyseobjekt ist gut validierbar, aber für die unmittelbare GitLab-Kommunikation weniger lesbar. Markdown kann von Menschen geprüft und von Entwicklungswerkzeugen weiterverarbeitet werden. Die getrennte Titelmetadatei erlaubt es, GitLabs eigenes Titelfeld korrekt zu aktualisieren, ohne den Titel in der Beschreibung zu duplizieren.

Der wichtigste Übergabepunkt liegt zwischen Analyse und Publish. Vor diesem Punkt sind alle Inhalte Vorschläge. Nach dem Publish bilden Titel und Beschreibung den freigegebenen Ticketstand. Der zweite Übergabepunkt liegt zwischen Ticketaufbereitung und späterer Implementierung: Dev-Assist schreibt zwar einen technischen Kontext, verändert aber keinen Produktcode. Damit bleibt die Aufbereitung der Anforderung von ihrer Umsetzung getrennt.

7.5 Erfüllung der funktionalen Anforderungen

Kapitel 6 hat den Test- und Nachweisstand detailliert bewertet. Für die Ergebnisdarstellung wird ergänzend zwischen **Umsetzung im Prototyp** und **Nachweis der Umsetzung** unterschieden. Eine Funktion kann im Code vorhanden sein, ohne bereits durch einen vollständigen Integrationstest oder einen realen GitLab-Lauf belegt zu sein.

Anforderung	Ergebnis im finalen Prototyp	Umsetzungs- und Nachweisstand
FA-1 Explizite Aktivierung am Textanfang	Eine Mention ist erforderlich, wird aber auch im Titel oder mitten im Text akzeptiert.	Abweichend umgesetzt. Parser- und Routentests bestätigen das breitere Verhalten.
FA-2 Relevante GitLab-Ereignisse	Issue- und Note-Payloads werden vereinheitlicht und an Process oder Publish weitergeleitet.	Umgesetzt, teilweise nachgewiesen. Grundformen und ignorierte Routenszenarien sind getestet; ein realer positiver GitLab-Ende-zu-Ende-Lauf fehlt.
FA-3 Analyse- und Publish-Kommando	Ohne publish wird analysiert, mit @dev-assist publish wird der gespeicherte Kontext übernommen.	Umgesetzt und auf Parser-Ebene nachgewiesen. Die vollständige Publish-Kette ist nicht integriert getestet.
FA-4 Abruf des Ticketkontexts	Issue und Notes werden über den GitLab-Client geladen; Webhook-Daten dienen als Fallback.	Umgesetzt, teilweise nachgewiesen. Externe Abrufe sind nicht als vollständiger Integrationstest abgedeckt.
FA-5 Gemeinsame Kontextanalyse	Promptaufbau verbindet Issue, Kommentare, frühere Antworten und optionale Repository-Zusammenfassung.	Strukturell umgesetzt und getestet. Die semantische Zusammenführung realer Modellantworten ist nicht bewertet.

Anforderung	Ergebnis im finalen Prototyp	Umsetzungs- und Nachweisstand
FA-6 Gezielte Rückfragen	Ab zwei offenen Fragen wird ein Klärungskommentar erzeugt; beantwortete ähnliche Fragen werden entfernt.	Umgesetzt, teilweise nachgewiesen. Darstellung und Filter sind getestet, fachliche Präzision realer Fragen nicht.
FA-7 Strukturierte Ticketvorschläge	Der Formatter erzeugt vier feste Markdown-Abschnitte und einen getrennten Titel.	Als Kompaktstruktur umgesetzt und getestet. Die umfangreichere Zielstruktur aus Kapitel 3 wird nicht vollständig abgebildet.
FA-8 Maschinenlesbares Analyseformat	Ein exaktes sechsteiliges JSON-Schema wird zur Laufzeit validiert.	Für den aktuellen Vertrag umgesetzt und gut nachgewiesen. Gegenüber dem ursprünglichen umfangreicheren Schema besteht eine Abweichung.
FA-9 Veröffentlichung als GitLab-Kommentar	Rückfragen, Vorschläge und Analysefehler können über <code>createNote</code> veröffentlicht werden.	Umgesetzt, teilweise nachgewiesen. Formatter-Ausgaben sind getestet, eine reale Kommentarerstellung nicht.
FA-10 Persistenz des Kontexts	<code>context.md</code> und optionale Titelmetadaten in <code>context.json</code> werden geschrieben und für Publish gelesen.	Umgesetzt, teilweise nachgewiesen. Ein vollständiger Dateisystem-Integrationstest fehlt.
FA-11 Kontrollierte Übernahme	Der Publisher bereitet Titel und Beschreibung auf, filtert Dev-Assist-Kommentare und aktualisiert erst danach das Issue.	Umgesetzt, teilweise nachgewiesen. Hilfsfunktionen und Filter sind getestet, die vollständige GitLab-Mutation nicht.

Das funktionale Ergebnis ist damit ein nahezu durchgängiger Kernworkflow von der Aktivierung bis zur kontrollierten Übernahme. Zwei fachliche Inkonsistenzen bleiben sichtbar: Die Aktivierungsregel ist toleranter als FA-1, und das aktuelle Kompaktschema ist schmäler als die in Kapitel 3 formulierte Zielstruktur. Darüber hinaus betrifft die Mehrzahl der verbleibenden Einschränkungen nicht das Fehlen einer Codekomponente, sondern die noch unvollständige integrations- oder inhaltsbezogene Validierung.

7.6 Erfüllung der nicht-funktionalen Anforderungen

Bei den nicht-funktionalen Anforderungen ergibt sich ein stärker abgestuftes Ergebnis:

Anforderung	Ergebnis im finalen Prototyp
NFA-1 Wartbarkeit	Die Verantwortlichkeiten sind auf Routing, GitLab, AI, Kontext, Repository-Zusammenfassung und Prozesssteuerung verteilt. Der TypeScript-Build ist erfolgreich. Eine quantitative Wartbarkeitsbewertung liegt nicht vor.
NFA-2 Testbarkeit ohne externe Abhängigkeiten	Mock-Provider, simulierte GitLab-Clients, temporäre Opencode-Programme und lokale HTTP-Tests ermöglichen 60 automatisierte Tests ohne reales GitLab und ohne Modellzugang. Diese Anforderung ist am stärksten erfüllt.
NFA-3 Robustheit	Fallbacks für fehlenden GitLab-Kontext, Schemafehler, leere Abschnitte und Opencode-Export sind vorhanden. Externe Fehlerketten, Dateisystemfehler, Last und Parallelität bleiben unvollständig geprüft.
NFA-4 Nachvollziehbarkeit	Alle wesentlichen Schritte werden über strukturierte Konsolenausgaben protokolliert. Eine automatisierte Prüfung auf Vollständigkeit, Korrelation und Secret-Freiheit fehlt.
NFA-5 Begrenzung von KI-Risiken	Festes Schema, kontrollierte Formatierung, sichtbare offene Fragen und ein separater Publish-Schritt begrenzen die Wirkung fehlerhafter Ausgaben. Die fachliche Richtigkeit realer Modellantworten ist nicht nachgewiesen.
NFA-6 Webhook-Sicherheit	Eine HMAC-Prüfung ist implementiert, entspricht aber nicht dem in Kapitel 6 geprüften aktuellen GitLab-Signing-Token-Vertrag. Für einen produktiven Einsatz ist diese Anforderung nicht ausreichend erfüllt.
NFA-7 Bot-Schleifen und Duplikate	Bot-Namen und Dev-Assist-Marker verhindern mehrere Selbstreaktionen; eine In-Memory-Deduplication reduziert Wiederholungen. Mehrinstanzbetrieb und Parallelfälle sind nicht abgesichert.
NFA-8 Schnelle Webhook-Antwort	Die Route bestätigt akzeptierte oder ignorierte Ereignisse mit HTTP 202 und führt den Kernprozess im Hintergrund aus. Der positive asynchrone Pfad wurde nicht unter Last gemessen.
NFA-9 Konfigurierbarkeit	Port, Provider, Modell, Timeout, GitLab-Zugriff, Signaturpflicht, Tunnel, Kontextpfad und Deduplication-Fenster sind über Umgebungsvariablen steuerbar. Ungültige Kombinationen sind nur teilweise getestet.
NFA-10 Prozessintegrierte Bedienbarkeit	Die Nutzung erfolgt vollständig über GitLab-Issues und Kommentare; eine zusätzliche GUI ist nicht erforderlich. Eine Usability-Untersuchung mit realen Nutzerinnen und Nutzern fehlt.

Das stärkste nicht-funktionale Ergebnis ist die lokale Testbarkeit. Ebenfalls klar rea-

liert sind modulare Zuständigkeiten, Konsolenlogging und die menschliche Freigabegrenze. Die größten produktionsbezogenen Lücken liegen bei der Webhook-Authentifizierung, bei verteilten beziehungsweise parallelen Verarbeitungssituationen, bei Monitoring und bei der inhaltlichen Evaluation realer Modellantworten.

7.7 Verdichtung der Ergebnisse

Die Arbeit erzielt als technisches Ergebnis einen funktionsfähigen Proof of Concept für einen KI-gestützten GitLab-Assistenten. Der Prototyp zeigt, dass ein Ticketworkflow aus Ereigniserkennung, Kontextabruf, agentenbasierter Analyse, strikter Antwortvalidierung, Rückfrage- oder Vorschlagskommunikation, lokaler Persistenz und expliziter Veröffentlichung in einer kleinen TypeScript-/Express-Anwendung zusammengeführt werden kann.

Für die erste in der Einleitung formulierte Fragestellung liefert der Prototyp eine konkrete, wenn auch gegenüber Kapitel 3 reduzierte Antwortstruktur. Ein aufbereitetes Ticket enthält einen eigenständigen Titel, eine Beschreibung, Akzeptanzkriterien, technischen Kontext, einen Lösungsvorschlag und sichtbare offene Fragen. Damit werden die für Verständnis, Umsetzung und Prüfung wesentlichen Informationsarten explizit getrennt. Ob diese Struktur in realen Projekten stets ausreicht und ob die Inhalte zuverlässig korrekt erzeugt werden, ist durch den aktuellen Nachweis noch nicht beantwortet.

Für die zweite Fragestellung liegt ein technischer Integrationsnachweis auf Prototypebene vor. GitLab-Ereignisse können einen Analyseprozess auslösen; der verfügbare Kontext wird in einen Prompt überführt; die Modellantwort wird nicht frei weitergereicht, sondern gegen einen festen Vertrag geprüft; das Ergebnis wird zunächst als Vorschau und Kontextdatei bereitgestellt und erst nach einer zweiten Aktion übernommen. Diese Kette zeigt, wie generative KI in einen kontrollierten Ticketworkflow eingebettet werden kann. Der Nachweis bleibt jedoch lokal und komponentenorientiert, solange ein realer GitLab-Ende-zu-Ende-Test und eine empirische Bewertung der Modellergebnisse fehlen.

7.8 Zwischenfazit

Der finale Dev-Assist-Prototyp bildet den vorgesehenen Kernprozess technisch weitgehend ab. Von einem Issue oder Kommentar mit Mention führt ein nachvollziehbarer Pfad über Kontextaufbereitung und KI-Analyse zu Rückfragen oder einem strukturierten Vorschlag. Der Vorschlag wird als GitLab-Kommentar sichtbar, als Markdown und Metadaten persistiert und erst nach einem expliziten Publish-Kommando in Titel und Beschreibung übernommen. Die Anwendung ist modular aufgebaut, lokal ohne externe Modell- oder GitLab-Abhängigkeiten testbar und besitzt feste Validierungs- und Freigabegrenzen.

Gleichzeitig ist das Ergebnis nicht mit Produktionsreife oder einem allgemeinen Wirksamkeitsnachweis gleichzusetzen. Die Aktivierungsregel und das Zielschema weichen

von Teilen der ursprünglichen Anforderungen ab. Die aktuelle Signaturprüfung ist nicht mit dem aktuellen GitLab-Signing-Token-Verfahren kompatibel. Vollständige positive Integrationspfade und eine Bewertung realer Modellantworten fehlen. Kapitel 8 ordnet diese Ergebnisse deshalb hinsichtlich Nutzen, Risiken, Wartbarkeit und Vergleich mit manueller Ticketpflege ein.

Quellen zu Kapitel 7

GitLab Docs o. J.a. Webhooks. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 21.07.2026).

GitLab Docs o. J.c. Issues API. <https://docs.gitlab.com/api/issues/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.d. Notes API. <https://docs.gitlab.com/api/notes/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.a. Agents. <https://opencode.ai/docs/agents/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.d. CLI. <https://opencode.ai/docs/cli/> (Zugriff vom 08.07.2026).

Projektinterne Arbeitsgrundlagen: AGENTS.md, README.md, package.json, opencode.json, src/server.ts, src/app.ts, src/config.ts, src/routes/gitlabWebhooks.ts, src/routes/issues.ts, src/services/gitlab/parser.ts, src/services/gitlab/mention.ts, src/services/gitlab/commands.ts, src/services/gitlab/auth.ts, src/services/gitlab/client.ts, src/services/gitlab/cleanup.ts, src/services/gitlab/glab.ts, src/services/ai/instructions.ts, src/services/ai/service.ts, src/services/ai/schema.ts, src/services/ai/formatter.ts, src/services/ai/clarifications.ts, src/services/ai/opencodeRuntime.ts, src/services/repositorySummary.ts, src/services/processing/processor.ts, src/services/processing/publisher.ts, src/services/context/writer.ts, src/services/context/reader.ts, src/services/tunnel.ts, src/utils/logger.ts, tests/aiOpencode.test.ts, tests/aiPrompt.test.ts, tests/auth.test.ts, tests/cleanup.test.ts, tests/config.test.ts, tests/formatter.test.ts, tests/gitlab.test.ts, tests/glab.test.ts, tests/opencodeRuntime.test.ts, tests/processor.test.ts, tests/publisher.test.ts, tests/repositorySummary.test.ts, tests/schema.test.ts, tests/tunnel.test.ts und tests/webhookRoute.test.ts.

8 Diskussion

Kapitel 7 hat Dev-Assist als funktionsfähigen Proof of Concept eingeordnet: Der Prototyp verbindet GitLab-Ereignisse, Kontextaufbereitung, KI-Analyse, strukturierte Validierung, Rückfragen, lokale Persistenz und einen kontrollierten Publish-Schritt. Dieses technische Ergebnis beantwortet jedoch noch nicht, welchen tatsächlichen Nutzen das System in einem Entwicklungsteam erzeugt, wie verlässlich KI-generierte Tickervorschläge sind oder unter welchen organisatorischen Bedingungen ein produktiver Einsatz vertretbar wäre. Die nachfolgende Diskussion trennt deshalb zwischen nachgewiesenen Systemeigenschaften, plausiblen Nutzenpotenzialen und bislang offenen Wirkungsannahmen.

Im Mittelpunkt stehen fünf Fragen: Wo kann Dev-Assist die Ticketarbeit sinnvoll unterstützen? Wie unterscheidet sich dieser Ansatz von ausschließlich manueller Ticketpflege? Welche Grenzen entstehen durch generative KI und begrenzten Kontext? Welche Risiken müssen technisch und organisatorisch beherrscht werden? Und wie wartbar beziehungsweise erweiterbar ist die gewählte Architektur? Die Diskussion greift damit die Ergebnisse aus Kapitel 6 und 7 auf, ohne aus einem lokalen Prototyp bereits eine allgemeine Aussage über Produktivität oder Ticketqualität abzuleiten.

8.1 Einordnung des Ergebnisbeitrags

Der wesentliche Beitrag des Prototyps liegt weniger in einer neuen Modellfähigkeit als in der kontrollierten Einbettung einer vorhandenen generativen KI in einen konkreten Arbeitsprozess. Sprachmodelle können natürliche Sprache flexibel strukturieren, doch für eine praktische Nutzung genügt eine plausible Textausgabe nicht. Dev-Assist ergänzt die Generierung daher um Ereignisfilter, Kontextauswahl, einen festen Antwortvertrag, Formatierung, Persistenz und eine menschliche Freigabegrenze. Das Ergebnis ist eine technische Prozesskette, in der die KI nur einen begrenzten Teil der Entscheidung übernimmt.

Diese Einordnung ist für die Bewertung entscheidend. Die 60 automatisierten Tests weisen vor allem nach, dass deterministische Grenzen des Systems im geprüften Stand funktionieren. Dazu gehören Parsing, Schema, Formatierung, Kommentarfilter, Rückfragenlogik und mehrere Fallbacks. Sie weisen nicht nach, dass ein Modell fachliche Absichten stets korrekt erkennt. Ebenso zeigt der beispielhafte Ablauf aus Kapitel 7, wie der Kontrollfluss funktioniert, nicht wie häufig reale Vorschläge von Entwicklungsteams akzeptiert oder korrigiert würden. Der Beitrag ist damit zunächst ein Machbarkeits- und Architekturbeitrag.

Gleichzeitig adressiert der Ansatz ein in der Literatur belegtes Problem. Bettenburg et al. zeigen eine Diskrepanz zwischen Informationen, die Entwicklerinnen und Entwickler

benötigen, und Angaben, die meldende Personen ohne weitere Unterstützung bereitstellen können (vgl. Bettenburg et al., 2008, S. 308 f.). Breu et al. beschreiben Bug Reports als dialogische Artefakte, in denen fehlende Informationen durch Rückfragen und spätere Antworten ergänzt werden; ihre Untersuchung umfasst 600 Berichte aus Mozilla- und Eclipse-Projekten (vgl. Breu et al., 2010, S. 301 und S. 303). Dev-Assist überträgt dieses Muster auf einen GitLab-Workflow: Das System versucht nicht nur, einen Ausgangstext umzuschreiben, sondern bezieht Kommentare ein und kann eine Klärungsschleife anstoßen.

Damit entsteht eine begründete Verbindung zwischen Problem und Lösungsansatz. Ob die konkrete Umsetzung das Problem in realen Teams tatsächlich besser löst als vorhandene Praktiken, bleibt dagegen empirisch offen. Die angemessene Aussage lautet daher: Der Prototyp zeigt einen technisch plausiblen Weg zur assistierten Ticketaufbereitung; einen allgemeinen Wirksamkeitsnachweis liefert er noch nicht.

8.2 Nutzen im Entwicklungsprozess

8.2.1 Strukturierung und frühe Sichtbarkeit von Informationslücken

Ein potenzieller Nutzen liegt in der wiederholbaren Strukturierung heterogener Ticketinformationen. GitLab-Issues können Titel, Beschreibung, Kommentare, technische Hinweise und später getroffene Entscheidungen enthalten. In manuellen Abläufen bleiben diese Informationen häufig über den Verlauf verteilt. Dev-Assist führt den verfügbaren Kontext in einer kompakten Struktur aus Titel, Beschreibung, Akzeptanzkriterien, technischem Kontext, Lösungsvorschlag und offenen Fragen zusammen. Dadurch wird sichtbar, welche Art von Information vorhanden ist und welche noch fehlt.

Die Literatur stützt die Relevanz dieser Strukturierungsaufgabe. Chaparro et al. identifizieren bei Bug Reports insbesondere beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte als zentrale Informationsarten und zeigen, dass solche Bestandteile häufig fehlen (vgl. Chaparro et al., 2017, S. 396 f. und S. 401 ff.). Lucassen et al. unterscheiden bei User Stories syntaktische, semantische und pragmatische Qualität; eine formal lesbare Anforderung kann demnach dennoch inhaltlich unzureichend oder für Beteiligte nicht nutzbar sein (vgl. Lucassen et al., 2016, S. 383 f. und S. 386 f.). Dev-Assist kann vor allem die syntaktische und strukturelle Ebene vereinheitlichen und fehlende Felder sichtbar machen. Semantische Richtigkeit und praktische Nutzbarkeit kann das Format allein nicht garantieren.

Der frühe Zeitpunkt der Analyse ist ebenfalls relevant. Werden fehlende Zielangaben, Akzeptanzkriterien oder Randfälle erst während der Implementierung erkannt, entstehen Rückfragen zu einem Zeitpunkt, an dem bereits Annahmen in technische Entscheidungen eingeflossen sein können. Ein Assistenzsystem kann die Aufmerksamkeit früher auf solche Lücken lenken. Diese Wirkung ist im Prototyp als Mechanismus vorhanden, wurde jedoch nicht durch Durchlaufzeiten, Nacharbeitsquoten oder Teambeobachtungen gemessen. Von einer nachgewiesenen Beschleunigung kann daher nicht gesprochen werden.

8.2.2 Entlastung bei wiederkehrender Aufbereitungsarbeit

Ein weiterer möglicher Nutzen betrifft repetitive Arbeit: Überschriften vereinheitlichen, verstreute Aussagen zusammenführen, Akzeptanzkriterien als Liste formulieren, technische Hinweise abgrenzen und offene Punkte explizit ausweisen. Solche Tätigkeiten erfordern Aufmerksamkeit, sind aber nicht in jedem Fall die wertschöpfende fachliche Entscheidung selbst. Die Automatisierung eines ersten Entwurfs kann Personen einen strukturierten Ausgangspunkt liefern, den sie prüfen und korrigieren.

Die Betonung liegt auf dem Entwurf. Eine automatisch längere Beschreibung ist nicht zwangsläufig besser. Sie kann bestehende Unklarheit lediglich sprachlich glätten oder durch zusätzliche, unbelegte Details verdecken. Der Nutzen entsteht nur, wenn das System gesicherte Informationen, Annahmen und offene Fragen unterscheidet. Die aktuelle Promptregel fordert diese Trennung, und der Formatter hält offene Fragen sichtbar. Dennoch bleibt die Zuordnung selbst modellabhängig. Eine fachlich falsche Annahme kann formal korrekt im technischen Kontext erscheinen und dadurch einen ungerechtfertigten Vertrauenseindruck erzeugen.

Die lokale Persistenz bietet einen zusätzlichen Übergabewert. `context.md` ist sowohl für Menschen lesbar als auch für nachgelagerte Werkzeuge nutzbar; `context.json` trennt den vorgeschlagenen Titel von der Beschreibung. Damit kann derselbe freigegebene Kontext in einer späteren Entwicklungsphase verwendet werden, ohne die Diskussion vollständig neu zu rekonstruieren. Dieser Vorteil setzt allerdings voraus, dass der persistierte Stand aktuell gehalten, eindeutig einem Issue zugeordnet und vor der weiteren Nutzung geprüft wird.

8.2.3 Prozessintegration und kontrollierte Freigabe

Dev-Assist arbeitet innerhalb der bereits verwendeten GitLab-Oberfläche. Rückfragen, Antworten, Vorschau und Freigabe erscheinen im Issue beziehungsweise im Kommentarverlauf. Eine zusätzliche Benutzeroberfläche und ein separater Medienbruch sind für den Kernablauf nicht erforderlich. Diese Einbettung kann die Nutzungsschwelle reduzieren, weil Beteiligte ihren gewohnten Arbeitsort nicht verlassen müssen.

Der getrennte Publish-Schritt unterstützt zugleich die Nachvollziehbarkeit. Vor der Übernahme ist der Vorschlag sichtbar; danach werden Titel und strukturierte Beschreibung gezielt aktualisiert. Das Modell besitzt keine unmittelbare GitLab-Schreibberechtigung. Diese Begrenzung folgt dem Grundsatz, dass ein LLM-gestütztes System nur die für seine Aufgabe notwendigen Funktionen und Berechtigungen erhalten sollte. OWASP beschreibt übermäßige Funktionalität, Berechtigungen oder Autonomie als Risiko "Excessive Agency" (vgl. OWASP, 2025b, Abschnitt "Excessive Agency").

Menschliche Freigabe ist jedoch keine automatische Qualitätsgarantie. Unter Zeitdruck kann ein plausibel formulierter Vorschlag oberflächlich bestätigt werden. Der Prozess sollte deshalb nicht nur einen Publish-Befehl verlangen, sondern eine sinnvolle Prüfung ermöglichen: offene Fragen müssen sichtbar sein, Annahmen dürfen nicht als Fakten erscheinen, Änderungen sollten vor der Übernahme vergleichbar sein und Verantwortlichkeiten für die Freigabe müssen geklärt werden. Der Prototyp realisiert davon die sichtbare Vorschau und den expliziten Befehl, aber noch kein Rollenmodell, keine for-

male Review-Checkliste und keinen Änderungsvergleich.

8.3 Vergleich mit manueller Ticketpflege

Manuelle Ticketpflege besitzt Stärken, die ein generatives System nicht ersetzen kann. Beteiligte kennen fachliche Prioritäten, informelle Entscheidungen, politische Randbedingungen und nicht dokumentiertes Domänenwissen. Sie können Widersprüche aushandeln, Verantwortung übernehmen und entscheiden, ob eine Anforderung überhaupt sinnvoll ist. Gerade bei neuartigen oder konfliktbehafteten Aufgaben entsteht Ticketqualität durch Kommunikation, nicht durch Umformatierung.

Die Schwäche rein manueller Pflege liegt in ihrer Abhängigkeit von Zeit, Erfahrung und individueller Sorgfalt. Unterschiedliche Personen verwenden unterschiedliche Strukturen; wichtige Informationen bleiben in Kommentaren; Akzeptanzkriterien werden nicht konsequent formuliert; Rückfragen erfolgen möglicherweise erst nach Beginn der Umsetzung. Breu et al. zeigen, dass Nachfragen in Bug-Tracking-Systemen ein wesentlicher Teil der Zusammenarbeit sind (vgl. Breu et al., 2010, S. 301 ff.). Werkzeuge wie BEE zeigen zudem, dass automatisierte Strukturanalyse und Rückmeldung direkt in den Ticketprozess eingebunden werden können (vgl. Song/Chaparro, 2020, S. 1551 f.).

Für den Vergleich ergeben sich folgende Schwerpunkte:

Dimension	Ausschließlich manuelle Pflege	Assistierte Pflege mit Dev-Assist
Kontextverständnis	kann Domänenwissen, implizite Ziele und soziale Abstimmung einbeziehen	ist auf bereitgestellten Ticket-, Kommentar- und Repository-Kontext begrenzt
Formkonsistenz	hängt von Erfahrung, Zeit und Teamkonventionen ab	liefert wiederholt dieselbe kompakte Abschnittsstruktur
Lückenerkennung	kann präzise und situationsbezogen sein, erfolgt aber nicht zwingend systematisch	prüft jeden aktivierten Vorgang nach denselben Regeln, kann relevante Lücken übersehen oder falsche Fragen stellen
Verantwortung	liegt unmittelbar bei den beteiligten Personen	bleibt trotz Vorschlag und Publish technisch und fachlich bei den freigebenden Personen
Aufwand	vollständige Strukturierung und Konsolidierung erfolgen manuell	erzeugt einen ersten Entwurf, ergänzt aber Prüf- und Korrekturaufwand

Der sinnvolle Zielzustand ist daher kein Ersatz-, sondern ein Kooperationsmodell. Dev-Assist übernimmt Routineaufbereitung, macht Unsicherheiten sichtbar und bietet eine konsistente Vorschau. Menschen prüfen fachliche Richtigkeit, ergänzen fehlenden Kontext, lösen Konflikte und verantworten die Übernahme. Ob dieses Modell insgesamt

weniger Aufwand verursacht, hängt von der Qualität der Vorschläge ab. Sind viele Korrekturen erforderlich oder werden falsche Annahmen erst spät erkannt, kann der Prüfaufwand den Automatisierungsgewinn aufheben.

Ein belastbarer Vergleich würde reale Tickets benötigen, die sowohl manuell als auch assistiert aufbereitet werden. Bewertet werden könnten Bearbeitungszeit, Zahl und Relevanz der Rückfragen, Vollständigkeit, Korrekturaufwand, Akzeptanzrate der Vorschläge und Übereinstimmung mehrerer Bewertender. Eine solche Untersuchung wurde nicht durchgeführt. Die Diskussion kann deshalb Prozessunterschiede begründen, aber keine Überlegenheit von Dev-Assist behaupten.

8.4 Grenzen KI-generierter Tickervorschläge

8.4.1 Halluzinationen und falsche Annahmen

Das zentrale fachliche Risiko bleibt die Erzeugung plausibler, aber unbelegter Inhalte. NIST bezeichnet solche überzeugend dargestellten falschen oder fehlerhaften Inhalte als Confabulation beziehungsweise Hallucination (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Huang et al. ordnen Halluzinationen als grundlegendes Zuverlässigkeitsproblem von Large Language Models ein (vgl. Huang et al., 2023, S. 1 f.). Für ein Ticketassistenzsystem sind nicht nur frei erfundene Tatsachen problematisch. Auch eine zu konkrete Interpretation einer mehrdeutigen Aussage kann eine falsche Annahme sein.

Der feste JSON-Vertrag begrenzt dieses Risiko nur auf der Formebene. Er stellt sicher, dass erwartete Felder vorhanden sind und Textlisten enthalten. Er kann nicht prüfen, ob ein Akzeptanzkriterium aus dem Ticket ableitbar ist, ob ein technischer Hinweis wirklich zum Repository passt oder ob der Lösungsvorschlag eine fachliche Entscheidung vorwegnimmt. Strukturelle Validität darf daher nicht mit inhaltlicher Wahrheit verwechselt werden.

Besonders kritisch ist die aktuelle Schwelle für Rückfragen. Erst ab zwei substanziellen offenen Fragen erzeugt der Processor zwingend einen Klärungskommentar. Bei einer offenen Frage wird bereits ein vollständiger Vorschlag erstellt und die Unsicherheit im technischen Kontext sichtbar gehalten. Diese Heuristik verbessert den Arbeitsfluss für kleinere Restfragen, kann aber bei einer einzigen entscheidenden Unklarheit zu früh einen scheinbar reifen Vorschlag erzeugen. Die Anzahl offener Fragen ist kein verlässliches Maß für ihre Bedeutung. Produktiv müsste deshalb zusätzlich nach Kritikalität unterschieden werden, etwa zwischen einer optionalen Randfallfrage und einer ungeklärten Kernentscheidung.

Auch der Lösungsvorschlag birgt ein Framing-Risiko. Sobald ein technisch plausibler Ansatz im Ticket steht, kann er die weitere Diskussion verengen, obwohl mehrere Lösungen möglich wären. Ein Assistant sollte deshalb zwischen Anforderung und Implementationsidee klar unterscheiden und technische Vorschläge als vorläufig kennzeichnen. Die aktuellen Promptregeln verlangen, nur belegte technische Details zu verwenden und Annahmen sichtbar zu markieren. Ohne empirische Auswertung realer Antworten ist jedoch nicht belegt, wie zuverlässig dies gelingt.

8.4.2 Begrenzter und potenziell veralteter Kontext

Die Qualität eines Vorschlags ist durch den verfügbaren Kontext begrenzt. Dev-Assist berücksichtigt Issue-Daten, vorhandene Notes, bis zu sechs jüngste Kommentare im unmittelbaren Prompt sowie optional eine gecachte Repository-Zusammenfassung. Der Repository-Kontext ist absichtlich gekürzt: Dateibaum und Schlüsseldateien werden begrenzt, damit große Projekte den Prompt nicht überladen. Diese Auswahl ist für einen Prototyp praktikabel, kann aber relevante Informationen ausschließen.

Mehrere Verluststellen sind möglich. Frühere Kommentare können eine weiterhin gültige Entscheidung enthalten, obwohl sie nicht zu den jüngsten sechs Einträgen gehören. Schlüsseldateien können außerhalb der vordefinierten Auswahl liegen. Eine gecachte Zusammenfassung kann nach Repository-Änderungen veraltet sein. Screenshots und Bildanhänge werden nicht inhaltlich analysiert. Externe Links, Architekturentscheidungen oder organisationsinterne Regeln können fehlen. Das Modell kann daher nur den bereitgestellten Ausschnitt interpretieren und sollte nie so formulieren, als kenne es das gesamte System.

Zusätzlich reduziert das aktuelle kompakte Antwortschema die in Kapitel 3 entworfene umfangreichere Zielstruktur. Die Reduktion verbessert Beherrschbarkeit, Validierung und Lesbarkeit, bildet aber nicht jede Ticketart gleich gut ab. Ein Bug Report benötigt möglicherweise explizite Reproduktionsschritte und beobachtetes Verhalten, während eine neue Funktion Rollen, Nutzen, Abgrenzung und nicht-funktionale Anforderungen stärker betont. Eine einheitliche Kompaktstruktur ist ein brauchbarer gemeinsamer Nenner, aber kein vollständiges domänenspezifisches Modell.

8.4.3 Fehlender semantischer und externer Nachweis

Die Evaluation trennt bewusst deterministische Technik von nichtdeterministischer Modellqualität. Das erhöht die Wiederholbarkeit der Tests, lässt aber die zentrale Wirkungsfrage offen. Es gibt kein kuratiertes Ticketkorpus, keine wiederholten Läufe mit mehreren Modellen, keine Blindbewertung und keine Untersuchung mit Entwicklungsteams. Auch ein realer positiver GitLab-Ende-zu-Ende-Lauf fehlt.

Diese Grenzen betreffen Konstrukt-, interne und externe Validität. Die Zahl bestandener Tests misst nicht direkt Ticketqualität; Testdoubles können dieselben Annahmen wie die Implementierung enthalten; Ergebnisse aus einem lokalen TypeScript-Projekt lassen sich nicht ohne Weiteres auf andere Organisationen, Sprachen oder Domänen übertragen. Easterbrook et al. empfehlen, solche Schwächen des Untersuchungsdesigns explizit zu machen und durch ergänzende Methoden sowie Replikationen zu adressieren (vgl. Easterbrook et al., 2008, S. 302 f. und S. 305).

Folglich bleibt offen, wie stabil die Vorschläge bei gleichen Eingaben sind, ob unterschiedliche Modelle vergleichbare Lücken erkennen und wie oft Entwicklungsteams die Ergebnisse unverändert akzeptieren würden. Diese Unsicherheit ist keine bloße Dokumentationslücke, sondern begrenzt die Reichweite aller Nutzenargumente.

8.5 Risiken und verantwortbarer Einsatz

8.5.1 Prompt Injection und begrenzte Autonomie

Tickettexte und Repository-Inhalte sind für das Modell zugleich Arbeitsdaten und potenzielle Angriffseingaben. Ein Kommentar oder eine Datei kann Text enthalten, der versucht, die Analyseanweisungen zu überschreiben. Liu et al. zeigen anhand untersuchter LLM-integrierter Anwendungen eine hohe grundsätzliche Anfälligkeit gegenüber Prompt-Injection-Angriffen (vgl. Liu et al., 2023, S. 1). OWASP unterscheidet direkte Angriffe über Nutzereingaben und indirekte Angriffe über externe Inhalte wie Dateien oder Webseiten (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection").

Dev-Assist reduziert die mögliche Wirkung durch mehrere Grenzen: Der Agent besitzt keine direkte GitLab-Schreibberechtigung, die Anwendung bestimmt die aufrufbaren Schritte, die Ausgabe muss einem festen Schema entsprechen und produktive Änderungen benötigen ein separates Publish-Kommando. Diese Maßnahmen verhindern nicht, dass ein Angriff die inhaltliche Analyse beeinflusst. Sie begrenzen aber, welche unmittelbaren Systemhandlungen aus einer manipulierten Modellantwort folgen können.

Für einen verantwortbaren Einsatz müssten *untrusted content* und Systemanweisungen technisch und semantisch klar getrennt bleiben. Darüber hinaus wären ein minimierter Werkzeugumfang, restriktive GitLab-Berechtigungen, eine nachvollziehbare Protokollierung, regelmäßige Angriffstests und eine sichere Darstellung verdächtiger Inhalte erforderlich. Ein Schema ist dabei nur eine von mehreren Schutzschichten.

8.5.2 Datenschutz und Vertraulichkeit

Der Analysekontext kann personenbezogene, vertrauliche oder sicherheitsrelevante Informationen enthalten: Namen und Kontaktdaten in Kommentaren, Kundendaten in Logs, interne URLs, Stack Traces, Sicherheitslücken, Zugangsdaten oder proprietären Quellcode. Durch die Repository-Zusammenfassung kann sich der übermittelte Umfang zusätzlich vergrößern. NIST weist darauf hin, dass generative KI Datenschutzrisiken unter anderem durch Verarbeitung persönlicher Daten sowie Offenlegung oder Ableitung sensibler Informationen erzeugt (vgl. National Institute of Standards and Technology, 2024, S. 7).

Für Dev-Assist folgt daraus eine datenminimierende Gestaltung. In den Modellkontext sollten nur Informationen gelangen, die für die Ticketanalyse erforderlich sind. Secrets und sensible Logbestandteile müssten vor der Übermittlung erkannt oder maskiert werden. Modellanbieter, Speicherorte, Aufbewahrungsfristen und Trainingsnutzung wären organisatorisch zu klären. Ebenso braucht es Regeln dafür, welche Projekte und Ticketklassen verarbeitet werden dürfen. Der Prototyp besitzt hierfür noch kein eigenständiges Klassifikations-, Redaktions- oder Löschkonzept.

Lokale Kontextdateien schaffen eine zweite Schutzfläche. `context.md`, `context.json` und Repository-Zusammenfassungen können sensible Inhalte dauerhaft auf dem Dateisystem ablegen. Zugriffsrechte, Lebensdauer, Verschlüsselung, Bereinigung und Backup-Verhalten sind deshalb ebenso relevant wie die externe Modellübertragung. Konsolen-

logging muss zudem verhindern, dass Tokens, vollständige Payloads oder vertrauliche Inhalte unbeabsichtigt in zentralen Logs erscheinen. Diese Anforderungen sind für einen produktiven Betrieb offen.

8.5.3 Technische und organisatorische Betriebsrisiken

Kapitel 6 und 7 haben mehrere konkrete Betriebsrisiken gezeigt. Die aktuelle Signaturprüfung entspricht nicht dem geprüften aktuellen GitLab-Signing-Token-Vertrag. Eine produktive Aktivierung ohne Korrektur könnte legitime Webhooks ablehnen oder ein falsches Sicherheitsgefühl erzeugen. Die In-Memory-Deduplication ist bei Neustarts und mehreren Instanzen nicht ausreichend. Asynchrone Verarbeitung ohne persistente Queue erschwert Wiederaufnahme, Laststeuerung und eindeutige Fehlerzustände. Außerdem fehlt ein Rollenmodell für das Publish-Kommando.

Auch die Kommentarbereinigung ist organisatorisch sensibel. Der Publisher versucht, Dev-Assist-bezogene Kommentare zu entfernen, während normale Nutzer- und Systemkommentare erhalten bleiben. Damit wird der endgültige Ticketstand übersichtlicher, zugleich kann Diskussionskontext verloren gehen, wenn Filterregeln zu breit greifen. Für revisionsrelevante Umgebungen müsste geklärt werden, ob Kommentare überhaupt gelöscht, nur markiert oder in einem Audit-Artefakt archiviert werden sollen.

Ein produktiver Einsatz erfordert daher mehr als einen technisch erreichbaren Dienst. Zuständigkeiten für Betrieb, Freigabe und Fehlersituationen müssen geklärt sein. Monitoring sollte nicht nur Verfügbarkeit, sondern auch Analysefehler, abgelehnte Schemaantworten, lange Laufzeiten, wiederholte Rückfragen und Publish-Fehler sichtbar machen. Ohne diese Prozessverantwortung kann eine grundsätzlich hilfreiche Automatisierung neue Unsicherheit in den Ticketworkflow einführen.

8.6 Wartbarkeit und Erweiterbarkeit

Die Architektur besitzt für einen Proof of Concept eine nachvollziehbare Aufteilung. Routing, GitLab-Zugriff, Authentifizierung, KI-Service, Promptregeln, Laufzeitvalidierung, Formatierung, Kontextpersistenz und Prozesssteuerung liegen in getrennten Modulen. Diese Trennung folgt dem Architekturgedanken, zentrale Elemente und ihre Beziehungen so zu strukturieren, dass Systemeigenschaften wie Änderbarkeit und Testbarkeit unterstützt werden (vgl. Bass et al., 2021, Kap. 1). Der Mock-Provider und simulierte Clients zeigen praktisch, dass viele Komponenten ohne externe Systeme geprüft werden können.

Die Modularität darf dennoch nicht überschätzt werden. Prompt, TypeScript-Typ, Laufzeitvalidator, Formatter, Kontextdatei und Publisher bilden gemeinsam einen Datenvertrag. Ändert sich ein Feld oder eine Abschnittsstruktur, müssen mehrere Stellen konsistent angepasst werden. Die Abweichung zwischen der umfangreichen Zielstruktur aus Kapitel 3, älteren Beschreibungen in Kapitel 5 und dem aktuellen Kompaktschema zeigt dieses Risiko bereits. Langfristig wäre eine einzige formal definierte Schemaquelle sinnvoll, aus der Typen, Validierung und gegebenenfalls Dokumentation abgeleitet werden.

Auch externe Schnittstellen erzeugen Wartungsbedarf. GitLab-Webhook-Verträge, API-Felder, glab-Verhalten, OpenCode-CLI-Ausgaben und Modellkonfigurationen können sich ändern. Die festgestellte Signaturabweichung verdeutlicht, dass erfolgreiche interne Tests eine geänderte externe Spezifikation nicht automatisch erkennen. Vertragstests gegen aktuelle Beispieldaten und eine explizite Versionsstrategie wären daher wichtiger als allein höhere Testzahlen.

Erweiterbarkeit ist an mehreren Stellen vorgesehen. Weitere Analyseprovider können hinter der AI-Schnittstelle eingebunden werden; zusätzliche GitLab-Ereignisse können im Parser ergänzt werden; Bildverarbeitung kann als eigener, abgesicherter Kontextschritt entstehen; persistente Queues oder Speicher können In-Memory-Zustände ersetzen; und das Kompaktschema kann für unterschiedliche Ticketarten profiliert werden. Jede Erweiterung vergrößert jedoch auch Kontextumfang, Berechtigungen und Angriffsfläche. Erweiterbarkeit sollte deshalb nicht nur als technische Offenheit, sondern als kontrollierte Veränderung des Sicherheits- und Validierungsmodells verstanden werden.

Wartbarkeit umfasst schließlich die Verständlichkeit des Betriebs. Zentral gepflegte Konfiguration, strukturierte Logs und dokumentierte manuelle Befehle sind gute Ausgangspunkte. Für mehrere Teams wären zusätzlich Migrationen für Kontextformate, observierbare Jobzustände, dokumentierte Fehlercodes und eine klare Verantwortlichkeit für Prompt- und Schemaänderungen notwendig. Insbesondere Promptänderungen sollten wie Codeänderungen versioniert, getestet und anhand eines festen Ticketkorpus bewertet werden.

8.7 Gesamtbewertung

Dev-Assist ist in seinem aktuellen Stand als Assistenzsystem sinnvoller einzuordnen als als Automatisierung der Anforderungsanalyse. Seine Stärke liegt in der Verbindung dreier Eigenschaften: Es arbeitet im vorhandenen GitLab-Kontext, es erzeugt eine wiederholbare Ticketstruktur, und es trennt Analyse von produktiver Übernahme. Damit adressiert der Prototyp reale Probleme unvollständiger und verteilter Ticketinformationen, ohne dem Modell uneingeschränkte Handlungsfähigkeit zu geben.

Der erwartbare Nutzen ist am größten bei Tickets, deren fachliches Ziel grundsätzlich bekannt ist, deren Informationen aber unstrukturiert oder über Kommentare verteilt vorliegen. Hier kann ein erster Entwurf Konsolidierungsarbeit reduzieren und offene Punkte sichtbar machen. Weniger geeignet ist der aktuelle Ansatz für Aufgaben, bei denen Ziele politisch umstritten, Domänenregeln kaum dokumentiert, Sicherheitsfolgen hoch oder entscheidende Informationen nur in nicht ausgewerteten Anhängen enthalten sind. In solchen Fällen kann die sprachliche Sicherheit eines Vorschlags die tatsächliche Unsicherheit verdecken.

Die zentrale Gestaltungsentscheidung, menschliche Freigabe beizubehalten, ist daher überzeugend. Sie muss produktiv allerdings durch prüfbare Änderungen, Rollen, Datenregeln und Monitoring ergänzt werden. Ebenso muss die technische Validierung um semantische Evaluation erweitert werden. Erst wenn reale Tickets wiederholt bewertet und mit manueller Pflege verglichen wurden, lässt sich entscheiden, ob Dev-Assist nicht nur funktioniert, sondern Entwicklungsteams messbar unterstützt.

8.8 Zwischenfazit

Die Diskussion zeigt ein ausgewogenes Ergebnis. Dev-Assist besitzt plausibles Potenzial, Ticketinformationen früh zu strukturieren, verteilten Kontext zusammenzuführen, wiederkehrende Aufbereitungsarbeit zu unterstützen und Vorschläge kontrolliert in GitLab bereitzustellen. Die Literatur zu Bug Reports und User Stories bestätigt, dass fehlende, unstrukturierte oder schwer nutzbare Informationen ein reales Problem darstellen. Der Prototyp bietet dafür einen nachvollziehbaren technischen Lösungsweg.

Gleichzeitig bleiben die Grenzen substanziell. Schema- und Formvalidierung sichern keine fachliche Wahrheit. Halluzinationen, falsche Annahmen, Prompt Injection, begrenzter oder veralteter Kontext und sensible Daten können die Ergebnisqualität und Sicherheit beeinträchtigen. Die aktuelle Evaluation belegt weder Zeitersparnis noch bessere Ticketqualität im realen Einsatz. Auch Signaturverfahren, verteilte Verarbeitung, Rollen, Monitoring und Datenlebenszyklen sind noch nicht produktionsreif.

Die angemessene Schlussfolgerung ist deshalb kein Ersatz manueller Ticketpflege, sondern ein kontrolliertes Kooperationsmodell: Dev-Assist erzeugt Struktur, Hinweise und Vorschläge; Menschen liefern Domänenwissen, bewerten Unsicherheit und tragen die Verantwortung für die Freigabe. Kapitel 9 fasst auf dieser Grundlage die Arbeit zusammen, beantwortet die Leitfragen und leitet priorisierte Weiterentwicklungen ab.

Quellen zu Kapitel 8

Bass, Len; Clements, Paul; Kazman, Rick 2021. *Software Architecture in Practice*. 4th ed. Addison-Wesley. <https://www.sei.cmu.edu/library/software-architecture-in-practice-fourth-edition/>.

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, S. 308-318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports. Improving Cooperation Between Developers and Users“, in Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work, S. 301-310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, S. 396-407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest; Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285-311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models. Principles, Taxonomy, Challenges, and Open Questions“. <https://arxiv.org/abs/2311.05232>.

Liu, Yi et al. 2023. „Prompt Injection attack against LLM-integrated Applications“. <https://arxiv.org/abs/2306.05499>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving agile requirements. The Quality User Story framework and tool“, in *Requirements Engineering* 21, 3, S. 383-403. <https://doi.org/10.1007/s00766-016-0250-x>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework. Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

OWASP 2025a. LLM01:2025 Prompt Injection. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 25.06.2026).

OWASP 2025b. LLM06:2025 Excessive Agency. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 25.06.2026).

Song, Yang; Chaparro, Oscar 2020. „BEE. A Tool for Structuring and Analyzing Bug Reports“, in Proceedings of the 28th ACM Joint Meeting on European Software En-

gineering Conference and Symposium on the Foundations of Software Engineering, S. 1551-1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

Projektinterne Arbeitsgrundlagen: AGENTS.md, README.md, package.json, opencode.json, src/config.ts, src/routes/gitlabWebhooks.ts, src/routes/issues.ts, src/services/gitlab/auth.ts, src/services/gitlab/parser.ts, src/services/gitlab/mention.ts, src/services/gitlab/commands.ts, src/services/gitlab/cleanup.ts, src/services/gitlab/client.ts, src/services/ai/instructions.ts, src/services/ai/service.ts, src/services/ai/schema.ts, src/services/ai/formatter.ts, src/services/ai/clarifications.ts, src/services/repositorySummary.ts, src/services/processing/processor.ts, src/services/processing/publisher.ts, src/services/context/writer.ts, src/services/context/reader.ts, src/utils/logger.ts sowie die in Kapitel 6 aufgeführten Tests.

9 Fazit und Ausblick

Die vorliegende Arbeit untersuchte, wie unvollständige oder unstrukturierte GitLab-Tickets durch einen KI-gestützten Developer Assistant aufbereitet werden können und wie sich ein solches System kontrolliert in einen bestehenden Entwicklungsworkflow integrieren lässt. Ausgangspunkt war die Beobachtung, dass Tickets nicht nur Aufgabenlisten, sondern zugleich Kommunikations-, Dokumentations- und Übergabeartefakte sind. Ihre Qualität beeinflusst, ob Anforderungen verstanden, umgesetzt und überprüft werden können. Forschung zu Bug Reports und User Stories zeigt, dass benötigte Informationen häufig fehlen, unterschiedlich strukturiert sind oder erst durch Rückfragen im weiteren Verlauf entstehen (vgl. Bettenburg et al., 2008, S. 308 f.; Breu et al., 2010, S. 301 ff.; Lucassen et al., 2016, S. 383 f.).

Zur Bearbeitung dieser Problemstellung wurde Dev-Assist konzipiert, als TypeScript-/Express-Prototyp umgesetzt und technisch bewertet. Das System reagiert auf GitLab-Ereignisse, sammelt Ticket- und Kommentar-Kontext, ergänzt optional eine Repository-Zusammenfassung, ruft einen konfigurierten Analyseagenten auf und akzeptiert nur Antworten nach einem festen Schema. Abhängig von der Informationslage veröffentlicht es Rückfragen oder einen strukturierten Vorschlag. Die Übernahme in Titel und Beschreibung des Issues erfolgt erst nach einem gesonderten Publish-Kommando. Das Schlusskapitel verdichtet die Ergebnisse, beantwortet die Forschungsfragen und priorisiert die nächsten Entwicklungsschritte.

9.1 Zusammenfassung der Ergebnisse

Die theoretischen Grundlagen haben gezeigt, dass Ticketqualität mehrere Ebenen umfasst. Eine Beschreibung kann sprachlich korrekt sein und dennoch fachlich oder praktisch unzureichend bleiben. Für Bug Reports sind unter anderem beobachtetes Verhalten, erwartetes Verhalten und Reproduktionsschritte wichtig; für allgemeine Anforderungen kommen Ziel, Kontext, Abgrenzung, Randbedingungen und überprüfbare Akzeptanzkriterien hinzu (vgl. Chaparro et al., 2017, S. 396 f.; Lucassen et al., 2016, S. 386 f.). Kommentare und Anhänge sind dabei Teil des Arbeitskontexts, weil entscheidende Informationen häufig erst nach der ursprünglichen Ticketanlage entstehen.

Aus diesen Grundlagen wurde ein Anforderungsmodell für Dev-Assist abgeleitet. Der Assistent sollte explizit aktiviert werden, Issue- und Kommentar-Kontext gemeinsam auswerten, Informationslücken erkennen, Rückfragen oder strukturierte Vorschläge erzeugen, Modellantworten zur Laufzeit prüfen, Kontext persistieren und Änderungen nur kontrolliert veröffentlichen. Ergänzende nicht-funktionale Anforderungen betrafen Wartbarkeit, lokale Testbarkeit, Robustheit, Nachvollziehbarkeit, Sicherheit, Schleifenvermeidung und Konfigurierbarkeit.

Die Architektur übersetzt diese Anforderungen in getrennte Verantwortlichkeiten. Routen für HTTP-Anfragen sowie die Webhook-Verarbeitung bilden den Einstieg. GitLab-Module kapseln Parsing, Authentifizierung, Mention- und Kommandoerkennung sowie API-Zugriffe. Die KI-Schicht bündelt Promptaufbau, Mock- und OpenCode-Pfad, JSON-Auswertung, Schema und Formatierung. Repository-Zusammenfassung und Kontextdateien ergänzen die Analyse, während Processor und Publisher die beiden Prozessphasen orchestrieren. Dadurch schreibt das Modell nicht selbst nach GitLab, sondern liefert nur ein strukturiertes Analyseobjekt an deterministische Anwendungsteile.

Der finale Prototyp bildet diesen Kernworkflow technisch weitgehend ab. Ein Issue oder Kommentar mit `@dev-assist` kann einen Process-Lauf auslösen. Der Processor lädt verfügbaren Kontext und entscheidet nach der Analyse zwischen Klärungskommentar und vollständigem Vorschlag. Der Vorschlag enthält einen getrennten Titel sowie vier sichtbare Abschnitte für Beschreibung, Akzeptanzkriterien, technischen Kontext und Lösungsvorschlag; verbleibende offene Fragen werden sichtbar gehalten. `context.md` und `context.json` persistieren Beschreibung und Titel. Erst `@dev-assist publish` beziehungsweise ein manueller Publish-Aufruf aktualisiert das Issue.

Die Qualitätssicherung bestätigt die technische Stabilität wichtiger deterministischer Komponenten. Der TypeScript-Build ist erfolgreich, und 60 automatisierte Tests in 15 Suiten bestehen. Besonders abgesichert sind das aktuelle Kompaktschema, die Ausgabe im Markdown-Format, Parserentscheidungen, Kommentarfilter, beantwortete Rückfragen, Zusammenfassungen des Repositories und OpenCode-Fallbacks. Gleichzeitig hat die Prüfung relevante Abweichungen sichtbar gemacht: Die Mention-Erkennung ist breiter als ursprünglich gefordert, das aktuelle Schema ist kompakter als die Zielstruktur aus Kapitel 3, die Signaturprüfung stimmt nicht mit dem geprüften aktuellen GitLab-Signing-Token-Vertrag überein, und vollständige positive Process-/Publish-Ende-zu-Ende-Tests fehlen.

Der wichtigste nicht erbrachte Nachweis betrifft die inhaltliche Modellqualität. Die Tests zeigen, dass die Anwendung schemawidrige Antworten abweist und formale Grenzen einhält. Sie zeigen nicht, ob reale Modelle fehlende Informationen zuverlässig erkennen, präzise Rückfragen stellen oder fachlich richtige Vorschläge erzeugen. Ebenso wurden weder Zeitersparnis noch Akzeptanzrate, Korrekturaufwand oder Verbesserung der Ticketqualität mit Entwicklungsteams gemessen. Dev-Assist ist deshalb als funktionsfähiger Proof of Concept und nicht als produktionsreifes oder allgemein validiertes System einzuordnen.

9.2 Beantwortung der Forschungsfragen

9.2.1 Erforderliche Informationen für umsetzbare und überprüfbare GitLab-Tickets

Die erste Forschungsfrage lautet, welche Informationen ein GitLab-Ticket enthalten muss, damit es von Entwicklerinnen und Entwicklern nachvollziehbar umgesetzt und überprüft werden kann. Die Ergebnisse zeigen, dass keine einzelne Textschablone für jede Ticketart ausreicht. Es lässt sich jedoch ein gemeinsamer Mindestbestand an In-

formationsarten bestimmen:

1. **Eindeutiger Gegenstand.** Ein prägnanter Titel muss benennen, welche Funktion, Änderung oder Störung behandelt wird.
2. **Problem beziehungsweise Ziel.** Die Beschreibung muss erklären, welcher Bedarf besteht, welcher Ist-Zustand problematisch ist oder welches Ergebnis erreicht werden soll.
3. **Fachlicher Kontext und Abgrenzung.** Betroffene Rollen, Systeme, Abläufe, Voraussetzungen und bewusste Nicht-Ziele müssen erkennbar sein, soweit sie für die Entscheidung relevant sind.
4. **Erwartetes und überprüfbares Verhalten.** Akzeptanzkriterien müssen beschreiben, woran eine korrekte Umsetzung erkannt werden kann. Bei Fehlern gehören insbesondere beobachtetes Verhalten, erwartetes Verhalten und nachvollziehbare Reproduktionsschritte dazu.
5. **Technische Evidenz und Randbedingungen.** Relevante Logs, Fehlermeldungen, Plattformangaben, Abhängigkeiten oder belegte Repository-Hinweise müssen verfügbar sein, ohne fachliche Ziele durch technische Annahmen zu ersetzen.
6. **Unsicherheit und offene Entscheidungen.** Fehlende Angaben, Annahmen, Widersprüche und offene Fragen müssen sichtbar bleiben, statt durch plausiblen Zusatztext verdeckt zu werden.

Der vom Prototyp verwendete Kompaktvertrag bildet diese Mindeststruktur teilweise ab. Die Felder `title` und `description` trennen Gegenstand und Zielbeschreibung. `acceptanceCriteria` enthält die Prüfkriterien, während `technicalContext` technische Hinweise erfasst. Das Feld `proposedSolution` hält einen vorläufigen Lösungsvorschlag fest. Verbleibende Unsicherheit steht in `openQuestions`. Der Lösungsvorschlag ist für die Übergabe hilfreich, gehört aber nicht in gleicher Weise zum fachlichen Mindestbestand wie Ziel und Akzeptanzkriterien. Er sollte als veränderbarer Vorschlag gekennzeichnet bleiben und darf keine ungeklärte Anforderung ersetzen.

Die Antwort auf die erste Forschungsfrage lautet daher: Ein umsetzbares und überprüfbares Ticket braucht nicht möglichst viel Text, sondern eine explizite Trennung von Ziel, Kontext, erwartbarem Verhalten, Prüfkriterien, belegten technischen Informationen und verbleibender Unsicherheit. Ticketartspezifische Profile müssen diese Grundstruktur ergänzen. Ein Bug Report benötigt andere Details als eine neue Funktion oder eine interne technische Aufgabe.

9.2.2 Kontrollierte Integration eines KI-gestützten Assistants in GitLab

Die zweite Forschungsfrage betrifft die technische Integration, sodass Analyse, Rückfragen, Vorschläge und Freigabe kontrolliert ablaufen. Der Prototyp zeigt hierfür eine Prozesskette aus sechs Schritten:

1. **Aktivierung und Ereignisfilter.** GitLab-Webhooks liefern Issue- oder Note-Ereignisse. Parser, Bot-Filter, Mention-Erkennung, die Auswertung von Kommandos und Deduplication bestimmen, ob eine Verarbeitung stattfinden darf.
2. **Kontextaufbereitung.** Issue, Kommentare und optionaler Kontext aus dem Repository werden gesammelt, gekürzt und als Daten für den Analyseprompt aufbereitet. Fehlende externe Daten führen nach Möglichkeit zu einem kontrollierten Fallback.
3. **Gekapselter Modellaufruf.** Ein definierter Provider beziehungsweise Agent erhält zentrale Regeln und den vorbereiteten Kontext. Das Modell bekommt keine unmittelbare Berechtigung zur Änderung des Issues.
4. **Deterministische Antwortgrenze.** Die Ausgabe muss JSON mit exakt festgelegten Feldern enthalten. Parsing und Laufzeitvalidierung trennen probabilistische Textgenerierung von der weiteren Anwendung.
5. **Vorschau, Rückfrage und Persistenz.** Je nach Informationslage veröffentlicht die Anwendung einen Klärungskommentar oder einen strukturierten Vorschlag und speichert den Kontext zusätzlich lokal.
6. **Explizite Freigabe.** Ein zweiter, bewusster Publish-Schritt liest den gespeicherten Vorschlag, bereitet Titel und Beschreibung auf und aktualisiert erst dann das GitLab-Issue.

Diese Architektur begrenzt insbesondere zwei Risiken. Erstens können Halluzinationen oder falsche Annahmen nicht vollständig verhindert werden, ihre unmittelbare Wirkung wird aber durch Schema, sichtbare offene Fragen und menschliche Freigabe reduziert. NIST beschreibt überzeugend formulierte falsche Inhalte als Confabulation beziehungsweise Hallucination (vgl. National Institute of Standards and Technology, 2024, S. 4 und S. 6). Zweitens wird übermäßige Handlungsfähigkeit vermieden, weil das Modell weder frei Werkzeuge auswählen noch selbst veröffentlichen darf; dies entspricht der Begrenzung des von OWASP beschriebenen Risikos "Excessive Agency" (vgl. OWASP, 2025b, Abschnitt "Excessive Agency").

Die Antwort auf die zweite Forschungsfrage lautet damit: Eine kontrollierte Integration entsteht nicht allein durch einen guten Prompt, sondern durch eine deterministische Anwendungshülle um das Modell. Ereignisprüfung, Kontextauswahl, minimale Berechtigungen, ein fester Datenvertrag, sichtbare Vorschau, Persistenz und getrennte Freigabe müssen gemeinsam umgesetzt werden. Prompt Injection, sensible Daten und Änderungen externer GitLab-Verträge erfordern zusätzliche Schutzschichten; ein formal gültiges Modellobjekt ist noch keine fachlich richtige oder sichere Entscheidung.

9.2.3 Übergreifende Antwort

Beide Forschungsfragen führen zu derselben übergreifenden Schlussfolgerung: KI kann die Anforderungsaufbereitung unterstützen, wenn Strukturierung und Prozesskontrolle stärker gewichtet werden als autonome Textproduktion. Dev-Assist ist deshalb sinnvoll als Kooperationssystem. Das System konsolidiert Informationen, erzeugt einen ersten

strukturierten Stand und macht offene Punkte sichtbar. Menschen liefern Domänenwissen, klären Zielkonflikte, bewerten Annahmen und verantworten die Freigabe.

Damit wird das in der Einleitung formulierte Ziel auf Prototypebene erreicht. Es wurde ein System umgesetzt, das wiederkehrende Aufbereitungsaufgaben übernehmen kann, ohne menschliche Abstimmung vollständig zu ersetzen. Nicht erreicht ist der Nachweis, dass diese Unterstützung in realen Teams messbar Zeit spart oder dauerhaft bessere Tickets erzeugt. Die technische Machbarkeit ist belegt; die Prozesswirkung bleibt eine offene empirische Frage.

9.3 Beitrag und Grenzen der Arbeit

Die Arbeit liefert vier zusammenhängende Beiträge. Erstens wurde aus Literatur zu Requirements Engineering, User Stories und Bug Reports ein Qualitätsverständnis für GitLab-Tickets abgeleitet. Zweitens wurde daraus eine Architektur für einen ereignisgesteuerten, KI-gestützten und menschlich kontrollierten Ticketworkflow entwickelt. Drittens liegt mit Dev-Assist eine ausführbare Referenzimplementierung vor. Viertens macht die Evaluation nicht nur bestandene Tests, sondern auch Vertragsabweichungen, Validierungslücken und Grenzen der Aussagekraft transparent.

Gerade die festgestellten Abweichungen sind ein relevantes Ergebnis. Sie zeigen, dass Modularität und automatisierte Tests allein externe Verträge nicht garantieren. Die interne HMAC-Testlogik konnte bestehen, obwohl sie nicht dem geprüften aktuellen GitLab-Verfahren entsprach. Ebenso kann ein striktes JSON-Schema formal zuverlässig sein und dennoch eine gegenüber dem Anforderungsmodell reduzierte Struktur absichern. Qualitätssicherung muss deshalb Implementierung, Anforderungen, externe Spezifikationen und semantische Wirkung gemeinsam betrachten.

Die Aussagekraft bleibt durch das Untersuchungsdesign begrenzt. Der Prototyp wurde in einem einzelnen Projekt und einer lokalen Umgebung untersucht. Modellantworten waren nicht Teil einer wiederholten empirischen Evaluation. Es fehlen reale GitLab-Ende-zu-Ende-Läufe, mehrere Modelle, verschiedene Ticketarten, ein repräsentatives Korpus und Bewertungen durch Entwicklungsteams. Nach Easterbrook et al. sollten solche Einschränkungen in Software-Engineering-Untersuchungen explizit behandelt und durch ergänzende Methoden sowie Replikationen reduziert werden (vgl. Easterbrook et al., 2008, S. 302 f. und S. 305).

9.4 Priorisierter Ausblick

Die Weiterentwicklung sollte nicht mit zusätzlichen Funktionen beginnen, sondern mit der Absicherung des bereits vorhandenen Kernprozesses. Daraus ergeben sich drei aufeinander aufbauende Prioritätsstufen.

9.4.1 Priorität 1: Vertragskorrektheit und belastbare Evaluation

Zuerst muss die Webhook-Authentifizierung an den aktuellen GitLab-Signing-Token-Vertrag angepasst und mit dokumentationsnahen Payloads geprüft werden (vgl. GitLab Docs, o. J.a, Abschnitt "Signing tokens"). Parallel sind die Aktivierungsregel und das Zielschema verbindlich festzulegen. Die Mention sollte entweder bewusst tolerant bleiben oder entsprechend FA-1 auf den Anfang der ersten Inhaltszeile begrenzt werden. Für das Analyseformat sollte eine einzige formale Schemaquelle entstehen, aus der TypeScript-Typen, Laufzeitvalidierung, Formattertests und Dokumentation abgeleitet werden.

Anschließend werden vollständige Integrationstests benötigt. Ein positiver Process-Pfad sollte Webhook, Kontextabruf, Analyse, Kommentar und Dateipersistenz gemeinsam prüfen. Ein positiver Publish-Pfad sollte gespeicherten Kontext, Kommentarfilter und Issue-Aktualisierung abdecken. Fehlerfälle müssen GitLab-Ausfälle, Dateisystemfehler, ungültige Modellantworten, abgebrochene Prozesse und wiederholte Ereignisse umfassen.

Die wichtigste Erweiterung ist eine inhaltliche Evaluation. Dafür sollte ein anonymisiertes und nach Ticketarten ausgewogenes Korpus aufgebaut werden. Mehrere reale Modellläufe müssten mit festgehaltenem Modell, Version, Prompt, Parametern und Zeitstempel ausgeführt werden. Mindestens zwei fachkundige Bewertende sollten die Ergebnisse unabhängig prüfen. So lassen sich Stabilität, Modellunterschiede und Übereinstimmung der Bewertungen erfassen.

9.4.2 Bessere Metriken zur Ticketqualität

Eine spätere Bewertung sollte nicht allein Vollständigkeit oder Textlänge messen. Sinnvoll ist ein mehrdimensionales Raster:

- **Strukturelle Vollständigkeit:** Sind Ziel, Kontext, Akzeptanzkriterien, technische Evidenz und offene Fragen vorhanden?
- **Faktentreue:** Lässt sich jede konkrete Aussage auf Issue, Kommentar, Log oder Repository-Kontext zurückführen?
- **Qualität der Rückfragen:** Sind Fragen relevant, nicht redundant, beantwortbar und auf entscheidende Lücken gerichtet?
- **Prüfbarkeit:** Sind Akzeptanzkriterien beobachtbar, eindeutig und mit vertretbarem Aufwand testbar?
- **Nutzbarkeit:** Können Entwicklerinnen und Entwickler das Ticket verstehen und eine Umsetzung planen, ohne wesentliche Informationen erneut zu suchen?
- **Prozessaufwand:** Wie viel Zeit, Korrekturarbeit und zusätzliche Kommunikation entstehen im Vergleich zur manuellen Aufbereitung?

Aus diesen Dimensionen können konkrete Kennzahlen entstehen: Anteil unbelegter Aus-

sagen, Präzision und Abdeckung erkannter Lücken, Zahl wiederholter Rückfragen, Akzeptanzrate von Vorschlägen, Umfang manueller Änderungen, Zeit bis zu einem freigabefähigen Ticket und Übereinstimmung mehrerer Bewertender. Eine solche Metrik darf fachliche Beurteilung nicht durch einen einzelnen automatischen Score ersetzen. Sie soll nachvollziehbar machen, an welcher Stelle der Assistant unterstützt oder Fehler erzeugt.

9.4.3 Priorität 2: Produktiver Betrieb, Monitoring und Governance

Nach Korrektur und Evaluation des Kernprozesses kann die Betriebsfähigkeit ausgebaut werden. Eine persistente Job-Queue sollte die asynchrone Verarbeitung, Wiederholungen und kontrollierte Fehlerzustände übernehmen. Persistente Schlüssel für Idempotenz und Deduplizierung müssen Neustarts und mehrere Instanzen abdecken. IDs zur Korrelation sollten Webhook, Analyse, Kommentar, Kontextdatei und Publish zu einem nachvollziehbaren Vorgang verbinden.

Monitoring sollte technische und fachliche Signale kombinieren. Dazu gehören Verfügbarkeit, Laufzeit, Providerfehler, Schemaablehnungen, Rückfragehäufigkeit, Publish-Fehler, Queue-Alter und Wiederholungsversuche. Ein Dashboard kann diese Zustände sichtbar machen, sollte aber erst auf stabilen Metriken aufbauen. Alerts sind insbesondere für dauerhaft fehlerhafte Provider, anwachsende Warteschlangen, Authentifizierungsfehler und ungewöhnliche Publish-Muster sinnvoll.

Ein Rechte- und Rollenkonzept muss festlegen, wer Analysen auslösen, Vorschläge prüfen und Issues veröffentlichen darf. GitLab-Projektrollen können eine Grundlage bilden, sollten aber im Publish-Pfad explizit geprüft werden. Für vertrauliche Projekte sind Datenklassifikation, Maskierung sensibler Inhalte, Aufbewahrungsfristen und Löschung lokaler Kontextdateien erforderlich. Prompt- und Schemaänderungen brauchen Eigentümerschaft, Review und Versionierung. OWASPs Hinweise zu Prompt Injection und begrenzter Agency unterstreichen, dass zusätzliche Funktionen und Berechtigungen nur minimal und kontrolliert vergeben werden sollten (vgl. OWASP, 2025a, Abschnitt "LLM01:2025 Prompt Injection"; OWASP, 2025b, Abschnitt "Excessive Agency").

9.4.4 Priorität 3: Funktionale Erweiterungen

Erst auf einem abgesicherten Kern bieten sich funktionale Erweiterungen an. Ticketartspezifische Profile könnten Bug Reports, neue Funktionen und technische Aufgaben unterschiedlich strukturieren. Eine Bild- und Screenshot-Verarbeitung könnte Anhänge kontrolliert abrufen, Dateitypen und Größen prüfen, OCR oder Vision einsetzen und Unsicherheiten sichtbar machen. Bildtext müsste dabei wie anderer externer Kontext als potenziell nicht vertrauenswürdig behandelt werden.

Weitere GitLab-Ereignisse könnten den Einsatzbereich erweitern, etwa Änderungen an Labels, Status oder Merge Requests. Solche Ereignisse sollten nur aufgenommen werden, wenn Trigger, Berechtigungen und Schleifenvermeidung eindeutig definiert sind. Eine optionale Benutzeroberfläche oder ein Dashboard könnte Vergleichsansichten, of-

fene Freigaben, Modellstatus und Qualitätsmetriken anzeigen. Für den Kernprozess bleibt GitLab jedoch der primäre Arbeitsort; eine zusätzliche Oberfläche sollte konkrete Informations- oder Governance-Bedürfnisse lösen und keinen parallelen Schattenprozess erzeugen.

Schließlich könnten mehrere Modelle oder Provider vergleichend eingesetzt werden. Ein Providerwechsel darf nicht nur technisch funktionieren, sondern muss anhand desselben Ticketkorpus bewertet werden. Auch die Übergabe an nachgelagerte Entwicklungsagenten ist denkbar. Sie sollte weiterhin vom freigegebenen Kontext ausgehen und eine zusätzliche Berechtigungs- und Kontrollgrenze besitzen. Aus einem Ticketassistenten darf nicht unbeabsichtigt ein autonomer Änderungsagent mit zu weitreichenden Rechten werden.

9.5 Schlussfolgerung

Dev-Assist zeigt, dass generative KI in einen GitLab-basierten Ticketworkflow eingebettet werden kann, ohne die gesamte Prozesskontrolle an das Modell abzugeben. Der Prototyp erkennt aktivierte Ereignisse, bereitet Kontext auf, erzeugt Rückfragen oder strukturierte Vorschläge, validiert die Ausgabe, persistiert das Ergebnis und trennt Analyse von Veröffentlichung. Damit liegt ein nachvollziehbarer technischer Lösungsweg für die in der Einleitung formulierte Problemstellung vor.

Der entscheidende Erfolgsfaktor ist nicht möglichst autonome KI, sondern die Kombination aus relevanter Informationsstruktur, begrenztem Kontext, deterministischer Validierung und menschlicher Verantwortung. Diese Kombination macht den Ansatz praktisch anschlussfähig, verhindert aber nicht automatisch falsche Inhalte. Der nächste wissenschaftliche und technische Schritt besteht deshalb in der empirischen Bewertung realer Modellergebnisse und in der Absicherung des vollständigen GitLab-Prozesses.

In seinem aktuellen Stand ist Dev-Assist ein belastbarer Ausgangspunkt für weitere Forschung und Entwicklung, aber kein fertiges Produkt. Sein Wert liegt darin, sowohl die Möglichkeiten als auch die notwendigen Grenzen eines KI-gestützten Ticketassistenten konkret sichtbar zu machen. Wird der Kernvertrag korrigiert, die semantische Qualität systematisch gemessen und der Betrieb durch Rollen, Monitoring und Datenregeln ergänzt, kann aus dem Proof of Concept ein verantwortbar einsetzbares Assistenzsystem entstehen.

Quellen zu Kapitel 9

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, S. 308-318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports. Improving Cooperation Between Developers and Users“, in Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work, S. 301-310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, S. 396-407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest; Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285-311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

GitLab Docs o. J.a. Webhooks. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 21.07.2026).

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving agile requirements. The Quality User Story framework and tool“, in *Requirements Engineering* 21, 3, S. 383-403. <https://doi.org/10.1007/s00766-016-0250-x>.

National Institute of Standards and Technology 2024. *Artificial Intelligence Risk Management Framework. Generative Artificial Intelligence Profile*. NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

OWASP 2025a. LLM01:2025 Prompt Injection. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 25.06.2026).

OWASP 2025b. LLM06:2025 Excessive Agency. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 25.06.2026).

Projektinterne Arbeitsgrundlagen: `AGENTS.md`, `README.md`, `package.json`, `opencode.json`, `src/config.ts`, `src/routes/gitlabWebhooks.ts`, `src/routes/issues.ts`, `src/services/gitlab/auth.ts`, `src/services/gitlab/parser.ts`, `src/services/gitlab/mention.ts`, `src/services/gitlab/commands.ts`, `src/services/gitlab/cleanup.ts`, `src/services/gitlab/client.ts`, `src/services/ai/instructions.ts`, `src/services/ai/service.ts`, `src/services/ai/schema.ts`, `src/services/ai/formatter.ts`, `src/services/ai/clarifications.ts`, `src/services/repositorySummary.ts`, `src/services/processing/processor.ts`, `src/services/processing/publisher.ts`, `src/services/context/writer.ts`, `src/services/context/reader.ts`, `src/utils/logger.ts` sowie die in Kapitel 6 aufgeführten Tests.

Quellen

Bettenburg, Nicolas; Just, Sascha; Schröter, Adrian; Weiss, Cathrin; Premraj, Rahul; Zimmermann, Thomas 2008. „What Makes a Good Bug Report?“, in Proceedings of the 16th ACM SIGSOFT International Symposium on Foundations of Software Engineering, S. 308-318. New York: ACM. <https://doi.org/10.1145/1453101.1453146>.

Breu, Silvia; Premraj, Rahul; Sillito, Jonathan; Zimmermann, Thomas 2010. „Information Needs in Bug Reports. Improving Cooperation Between Developers and Users“, in Proceedings of the 2010 ACM Conference on Computer Supported Cooperative Work, S. 301-310. New York: ACM. <https://doi.org/10.1145/1718918.1718973>.

Brown, Tom B. et al. 2020. „Language Models are Few-Shot Learners“, in Advances in Neural Information Processing Systems 33, S. 1877-1901. <https://arxiv.org/abs/2005.14165>.

Chaparro, Oscar; Bernal-Cárdenas, Carlos; Lu, Jing; Moran, Kevin; Marcus, Andrian; Di Penta, Massimiliano; Poshypanyk, Denys; Ng, Vincent 2019. „Assessing the Quality of the Steps to Reproduce in Bug Reports“, in Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, S. 152-163. New York: ACM. <https://doi.org/10.1145/3338906.3338947>.

Chaparro, Oscar; Lu, Jing; Zampetti, Fiorella; Moreno, Laura; Di Penta, Massimiliano; Marcus, Andrian; Bavota, Gabriele; Ng, Vincent 2017. „Detecting Missing Information in Bug Descriptions“, in Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering, S. 396-407. New York: ACM. <https://doi.org/10.1145/3106237.3106285>.

Express Docs o. J. Express.js. Node.js web application framework. <https://expressjs.com/en/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.a. Webhooks. <https://docs.gitlab.com/user/project/integrations/webhooks/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.b. Webhook events. https://docs.gitlab.com/user/project/integrations/webhook_events/ (Zugriff vom 25.06.2026).

GitLab Docs o. J.c. Issues API. <https://docs.gitlab.com/api/issues/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.d. Notes API. <https://docs.gitlab.com/api/notes/> (Zugriff vom 25.06.2026).

GitLab Docs o. J.e. Discussions API. <https://docs.gitlab.com/api/discussions/> (Zugriff vom 25.06.2026).

Huang, Lei et al. 2023. „A Survey on Hallucination in Large Language Models. Principles, Taxonomy, Challenges, and Open Questions“. <https://arxiv.org/abs/2311.05232>.

Inayat, Irum; Salim, Siti Salwah; Marczak, Sabrina; Daneva, Maya; Shamshirband, Shahaboddin 2015. „A systematic literature review on agile requirements engineering practices and challenges“, in Computers in Human Behavior 51, S. 915-929. <https://doi.org/10.1016/j.chb.2014.10.046>.

ISO/IEC 2023. ISO/IEC 25010:2023. Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - Product quality model. Öffentliche Katalogseite. <https://www.iso.org/standard/78176.html>.

ISO/IEC/IEEE 2018. ISO/IEC/IEEE 29148:2018. Systems and software engineering - Life cycle processes - Requirements engineering. Öffentliche Katalogseite. <https://www.iso.org/standard/72089.html>.

Liu, Yi et al. 2023. „Prompt Injection attack against LLM-integrated Applications“. <https://arxiv.org/abs/2306.05499>.

Lucassen, Garm; Dalpiaz, Fabiano; van der Werf, Jan Martijn E. M.; Brinkkemper, Sjaak 2016. „Improving agile requirements. The Quality User Story framework and tool“, in Requirements Engineering 21, 3, S. 383-403. <https://doi.org/10.1007/s00766-016-0250-x>.

National Institute of Standards and Technology 2024. Artificial Intelligence Risk Management Framework. Generative Artificial Intelligence Profile. NIST AI 600-1. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf>.

Node.js Docs o. J.a. Node.js. <https://nodejs.org/en> (Zugriff vom 25.06.2026).

Node.js Docs o. J.b. HTTP. <https://nodejs.org/api/http.html> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.a. Agents. <https://opencode.ai/docs/agents/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.b. SDK. <https://opencode.ai/docs/sdk/> (Zugriff vom 25.06.2026).

OpenCode Docs o. J.c. Tools. <https://opencode.ai/docs/tools/> (Zugriff vom 25.06.2026).

OWASP 2025a. LLM01:2025 Prompt Injection. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/> (Zugriff vom 25.06.2026).

OWASP 2025b. LLM06:2025 Excessive Agency. OWASP Top 10 for LLM Applications 2025. <https://genai.owasp.org/llmrisk/llm062025-excessive-agency/> (Zugriff vom 25.06.2026).

Schick, Timo et al. 2023. „Toolformer. Language Models Can Teach Themselves to Use Tools“, in Advances in Neural Information Processing Systems 36. <https://arxiv.org/abs/2302.04761>.

Song, Yang; Chaparro, Oscar 2020. „BEE. A Tool for Structuring and Analyzing Bug Reports“, in Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, S. 1551-1555. New York: ACM. <https://doi.org/10.1145/3368089.3417928>.

TypeScript Docs o. J. The TypeScript Handbook. <https://www.typescriptlang.org/docs/handbook/intro.html> (Zugriff vom 25.06.2026).

Vaswani, Ashish et al. 2017. „Attention Is All You Need“, in Advances in Neural Information Processing Systems 30. <https://papers.neurips.cc/paper/7181-attention-is-all-you-need>.

Xi, Zhiheng et al. 2023. „The Rise and Potential of Large Language Model Based Agents. A Survey“. <https://arxiv.org/abs/2309.07864>.

Yao, Shunyu et al. 2023. „ReAct. Synergizing Reasoning and Acting in Language Models“, in International Conference on Learning Representations 2023. https://openreview.net/forum?id=WE_vluYUL-X.

Zhao, Wayne Xin et al. 2023. „A Survey of Large Language Models“. <https://arxiv.org/abs/2303.18223>.

Bass, Len; Clements, Paul; Kazman, Rick 2021. *Software Architecture in Practice*. 4th ed. Addison-Wesley. <https://www.sei.cmu.edu/library/software-architecture-in-practice-fourth-edition/>.

Hohpe, Gregor; Woolf, Bobby 2003. *Enterprise Integration Patterns*. Addison-Wesley. <https://www.enterpriseintegrationpatterns.com/>.

Easterbrook, Steve; Singer, Janice; Storey, Margaret-Anne; Damian, Daniela 2008. „Selecting Empirical Methods for Software Engineering Research“, in Shull, Forrest; Singer, Janice; Sjøberg, Dag I. K. (Hrsg.): *Guide to Advanced Empirical Software Engineering*, S. 285-311. London: Springer. https://doi.org/10.1007/978-1-84800-044-5_11.

Wohlin, Claes; Runeson, Per; Höst, Martin; Ohlsson, Magnus C.; Regnell, Björn; Wesslén, Anders 2012. *Experimentation in Software Engineering*. Berlin, Heidelberg: Springer. <https://doi.org/10.1007/978-3-642-29044-2>.

Express Docs o. J.a. Basic routing. <https://expressjs.com/en/starter/basic-routing/> (Zugriff vom 08.07.2026).

Express Docs o. J.b. Writing middleware for use in Express apps. <https://expressjs.com/en/guide/writing-middleware/> (Zugriff vom 08.07.2026).

GitLab Docs o. J.f. GitLab CLI (glab). <https://docs.gitlab.com/cli/> (Zugriff vom 08.07.2026).

OpenCode Docs o. J.d. CLI. <https://opencode.ai/docs/cli/> (Zugriff vom 08.07.2026).

Projektinterne Arbeitsgrundlagen: README.md, descriptions/project_description.txt, AGENTS.md, package.json, tsconfig.json, opencode.json, src/server.ts, src/app.ts, src/config.ts, src/routes/gitlabWebhooks.ts, src/routes/issues.ts, src/routes/health.ts, src/services/gitlab/parser.ts, src/services/g

itlab/mention.ts, src/services/gitlab/commands.ts, src/services/gitlab/auth.ts, src/services/gitlab/client.ts, src/services/gitlab/cleanup.ts, src/services/gitlab/glab.ts, src/services/processing/processor.ts, src/services/processing/publisher.ts, src/services/ai/service.ts, src/services/ai/instructions.ts, src/services/ai/schema.ts, src/services/ai/formatter.ts, src/services/ai/clarifications.ts, src/services/ai/opencodeRuntime.ts, src/services/context/writer.ts, src/services/context/reader.ts, src/services/repositorySummary.ts, src/services/tunnel.ts, src/utils/logger.ts, src/cli/publish-issue.ts, tests/aiOpencode.test.ts, tests/aiPrompt.test.ts, tests/auth.test.ts, tests/cleanup.test.ts, tests/config.test.ts, tests/formatter.test.ts, tests/gitlab.test.ts, tests/glab.test.ts, tests/opencodeRuntime.test.ts, tests/processor.test.ts, tests/publisher.test.ts, tests/repositorySummary.test.ts, tests/schema.test.ts, tests/tunnel.test.ts und tests/webhookRoute.test.ts.